

PHONE FARM

ENGINEERING & OPERATIONS

HANDBOOK

Version 1.7 - Research Freeze

Final adversarial web audit, complete diagram preservation, and bench-gate specification

Research state	Broad web research frozen for this cut; reopen only on upstream change or contradictory bench evidence.
Research coverage	[Inference] 99.95%
Conceptual understanding	[Inference] 99.90%
Actual XYZ/Dim bench validation	74.4% - unchanged; no new local hardware measurements in this run.
Key unresolved gate	Measured 1 -> 2 -> 5 -> 10 -> 20 device stability on the actual Mac, hubs, phones and incoming board.

Evidence rule: first-party Apple/OS documentation outranks vendor claims for operating-system semantics. Vendor and community observations remain labeled and become project facts only after reproducible local testing.

XYZ MGMT - Confidential / Internal Technical Review
Research cut: 27 August 2026

XYZ MGMT Phone Farm Engineering & Learning Handbook — V1.2

Research-expanded Feynman edition: Mac-first iOS lab,
CoreDevice/RSD candidate path, provisioning, physical
plant, networking, recovery, security, validation and scaling

XYZ MGMT internal engineering research

27 August 2026

- Reader Notice
 - Evidence legend
 - Source precedence
 - 27 August 2026 research update — what changed after another web pass
 - 27 August 2026 second web research pass - new corrections and engineering consequences
 - 27 August 2026 third web research pass - V1.2 architecture refinement
- How to learn this handbook — Feynman method
 - The five planes you should never confuse
- Executive Synthesis
 - What this project actually is
 - The most important updated conclusions
- Section Map
- Mac-first bench plan — the work that can start tomorrow
 - Bench objective
 - What waits for the control board
- SECTION 1 - System Model and Architecture Choices
 - Feynman primer — a phone farm is a tiny private cloud
 - 1. Phone farms in OFM
 - 2. Traffic funnel and business model
 - 3. What physical phones solve - and what they do not
 - 4. Platform-policy and authorization boundary
 - 5. Architecture decision: API-first, device-first or hybrid
- SECTION 2 - Physical Plant: Devices, Racks, Power, USB and Thermal
 - Feynman primer — electricity and heat are the farm's plumbing
 - 6. Device selection
 - 7. Rack and mechanical design
 - 8. Power architecture
 - 9. USB topology and cabling
 - 10. Thermal engineering
 - 11. Fire/electrical safety and UPS
 - 12. Host computers and local controllers
 - 13. Inventory, labeling, spares and lifecycle
 - Feynman primer — an iPhone has separate doors for seeing, touching, managing and moving media
 - Why iOS deserves its own section
 - 3.1 Identity and reset model
 - 3.2 Provisioning and supervision
 - 3.3 Display paths
 - 3.4 Input paths
 - 3.5 The Lightning/USB coexistence question
 - 3.6 Media delivery
 - 3.7 Mac mini starting point
 - 3.8 iOS decision table
 - 24. Apple fleet architecture
 - 25. Supervision, enrollment and MDM
 - 26. Android Enterprise architecture
 - 27. Android provisioning and policy
 - 28. ADB for authorized engineering
 - 29. App distribution and configuration
 - 30. OS updates and maintenance rings
 - 31. Health telemetry and remote actions
 - Feynman primer — Android exposes a larger engineering service hatch
 - 4.1 Why Android remains operationally attractive
 - 4.2 Device identity: separate local IDs from backend memory
 - 4.3 ADB and media workflows
 - 4.4 Android Enterprise
 - 4.5 Integrity posture
 - 4.6 iOS versus Android - decision lens

- SECTION 5 - Networking: LAN, Wi-Fi, Cellular, NAT, DNS, Proxy and VPN

- Feynman primer — the network is a road map, not one thing called “the IP”
- 14. Network reference architecture
- 15. Wi-Fi radio and capacity planning
- 16. Ethernet, switches and VLANs
- 17. IPv4, IPv6, NAT and CGNAT
- 18. DNS and resolver design
- 19. Forward proxies and proxy protocols
- 20. VPNs, tunnels and per-app routing
- 21. Cellular, SIM and eSIM concepts
- 22. Egress observability and troubleshooting
- 23. Network failure modes

- SECTION 6 - Client, Account and Credential Separation

- Feynman primer — separation is fireproofing
- 6.1 The security objective
- 6.2 Six independent separation layers
- 6.3 Platform graph and delegated access
- 6.4 Client-owned containers
- 6.5 Credentials, 2FA and recovery
- 6.6 Browser profile is convenience isolation, not workstation security
- 6.7 Billing and vendor concentration
- 6.8 Staff lifecycle
- 6.9 Incident containment

- SECTION 7 - Content Operations and Platform Behavior: Evidence vs

- Feynman primer — treat platform folklore like laboratory rumors
- 7.1 Why this section is separate from infrastructure
- 7.2 The Instagram warmup evidence is contradictory by construction
- 7.3 Content quality versus account age
- 7.4 The five-post/fifth-story observation
- 7.5 Recommendation eligibility and sensitive content
- 7.6 Content pipeline as an engineering object
- 7.7 Attribution
- 42. Creator onboarding and authorization
- 43. Media ingest and rights metadata
- 44. Content processing and variants
- 45. Approval workflow
- 46. Publishing paths and official APIs
- 47. TikTok operational notes
- 48. Reddit, X and Snapchat operational notes
- 49. Instagram/Meta operational notes
- 50. OnlyFans boundary and account responsibility
- 51. Tracking links and attribution
- 52. Employee roles and access model
- 53. Daily/weekly operating cadence
- 54. Quality control and audit
- Feynman primer — the control plane is air-traffic control
- 8.1 Remote desktop is maintenance, not the product UI
- 8.2 RustDesk/GADS/custom roles
- 8.3 macOS permissions and commissioning
- 8.4 FileVault without surrendering unattended recovery
- 8.5 Node agent
- 32. Control-plane architecture
- 33. Device-to-coordinator communications
- 34. Task queue and state machine
- 35. Database and entity model
- 36. Object/media storage
- 37. API design
- 38. Messaging and event transport
- 39. Metrics, logs and traces
- 40. Credential and secret handling

- 41. Availability, backup and disaster recovery
- Feynman primer — build the black box before the crash
- 55. Threat model
- 56. Platform restriction and enforcement handling
- 57. Incident response
- 58. Privacy and data minimization
- 59. Business continuity and creator exit
- Feynman primer — do not build a twenty-lane bridge before testing one span
- 10.1 Experiment discipline
- 10.2 Gate 0 - inventory before engineering
- 10.3 TEST-MAC-CAPTURE-001 - does the proposed Mac USB display path exist programmatically?
- 10.4 TEST-BOARD-001 - inbound control board capability
- 10.5 TEST-IOS-INPUT-001 - characterize event type in an owned test app
- 10.6 TEST-MEDIA-IOS-001 - real Photos ingestion
- 10.7 TEST-COEXIST-001 - display and input simultaneously
- 10.8 TEST-CAPACITY-001 - pod scaling curve
- 10.9 TEST-THERMAL-001 - soak
- 10.10 TEST-REMOTE-001 - unattended recovery
- 10.11 Scale gates
- 10.12 Purchase rule
- 60. Scaling from 5 to 100+ devices
- 61. Cost model and BOM categories
- 62. Pilot build - 10 device reference
- 63. 25-50 device expansion
- 64. 30/60/90 day implementation roadmap
- 65. Acceptance test plan
- Feynman primer — give the coding agent a construction contract, not a wish
- 11.1 What Claude should do first
- 11.2 Repository layout
- 11.3 Domain invariants
- 11.4 Minimum data model
- 11.5 Adapter contract
- 11.6 Task state machine
- 11.7 First implementation backlog
- 11.8 Definition of done for Pilot v1
- 11.9 “Do not infer” list for Claude
- 11.10 Feedback loop
- 11.A - Prior Claude Handoff Deep Reference (updated evidence takes precedence)
- APPENDICES - Reference Configurations, Checklists, Formulas and Glossary
 - APPENDIX A - Example IP/VLAN plan
 - APPENDIX B - Example firewall policy
 - APPENDIX C - Task JSON and API examples
 - APPENDIX D - Example SQL data model
 - APPENDIX E - MQTT topic design
 - APPENDIX F - Device commissioning checklist
 - APPENDIX G - Weekly operations report template
 - APPENDIX H - Incident runbooks
 - APPENDIX I - Capacity formulas
 - APPENDIX J - Glossary
 - APPENDIX K - Research references
- APPENDIX M - 27 August 2026 Web Verification Register
 - Apple - UIApplicationSupportsIndirectInputEvents
 - Apple - DeviceCheck / App Attest WWDC21
 - Apple - App Attest integrity
 - Google - Play Integrity Device Recall
 - Appium - XCUITest real-device preparation
 - Apple - iPhone Mirroring
 - Apple - Apple Configurator supervision
 - Apple - Configurator Blueprints
 - Apple - FileVault remote unlock
 - Apple - Mac mini specifications
 - Google - Chrome profiles
 - Google - Chrome Sync data
 - TikTok - Business Center permissions
 - X - Delegate
 - RFC 6762 - Multicast DNS

- APPENDIX N - Internal Source Register and Authority Notes
 - Phone Farm - Technical Brief (26 August 2026)
 - Client Account Separation - Internal Technical Reference (20 August 2026)
 - IG Account Warmup - Protocol vs Mythology (25 August 2026)
 - iOS Reset Protocol (30 July 2026)
 - Farm Build - Functional Notes (26 August 2026)
 - Phone Farm Handover - Dim to Yigithan (26 August 2026)
- APPENDIX O - New Analytical Insights Derived in This Edition
- APPENDIX P - Final Implementation Path in One Page
- Appendix N — Research challenge ledger: where V1 breaks or refines the earlier brief
 - What V1 still cannot verify
- Appendix O — Dim interview and evidence-capture packet
 - Control/mirroring product
 - Existing hardware
 - Media workflow
 - Remote operation
 - Failure history
 - Evidence template
- Appendix P — Fresh web source register (27 August 2026)
 - Apple / iOS / macOS
 - iOS QA / device-lab tools
 - Android
 - Remote operations / safety
- APPENDIX Q - Second Web Verification Register (27 August 2026)
 - URLs
- APPENDIX R - Research Coverage and Understanding Scorecard
 - How to read the totals
 - What would raise understanding above 98% and validation materially
- APPENDIX S - Third Web Verification Register (V1.2)
 - URLs
- APPENDIX T - Canonical Architecture Diagram Atlas (V1.2)
 - ARCH-IOS-COREDEVICE-001 - Mac mini CoreDevice pod candidate
 - ARCH-IOS-VERSION-001 - iOS version-to-transport map
 - ARCH-COMMISSION-001 - Attach-to-ready commissioning flow
 - ARCH-DEVICE-STATE-001 - Runtime device lifecycle
 - ARCH-BATTERY-001 - Battery and thermal maintenance model
 - ARCH-PHOTOS-001 - Bytes versus a Photos-library asset
 - ARCH-IOS-CHOICE-001 - iOS control-path decision tree
 - ARCH-SLOT-ID-001 - Physical slot to logical history
 - ARCH-PROVISION-001 - Tethered provisioning lane
 - ARCH-RTS-001 - Return to Service lifecycle
 - ARCH-OBS-001 - Pod observability stack

Reader Notice

This handbook is a **technical and operational engineering reference** for an organization-owned physical-device fleet used for authorized creator/social-media operations, testing, content production and related workflows. It incorporates internal research, hands-on observations, vendor documentation, platform/OS documentation, and earlier engineering work.

It deliberately separates **mechanism** from **mythology**. A claim made by an operator is not converted into a fact merely because it is repeated. A vendor feature is not treated as proven until it is tested on the actual hardware. A platform behavior observed once is not treated as a universal algorithm rule. Every architecture-blocking uncertainty becomes an experiment.

This handbook does **not** provide an operational playbook for bypassing platform enforcement, manufacturing fake engagement, mass-creating replacement identities, falsifying geolocation, fingerprint spoofing, scraping data in violation of service terms, or automating prohibited third-party UI behavior. Networking, proxies, device identity and automation are explained for legitimate segmentation, testing, privacy, observability and authorized fleet operation.

Evidence legend

Label	Meaning	Engineering use
VERIFIED-FIRST-PARTY	Supported by current OS/platform documentation	May be used as architecture input, subject to version changes
INTERNAL-MEASURED	Observed by XYZ/Dim on owned hardware	Strong for the exact setup tested; do not overgeneralize
VENDOR-CLAIM	Vendor states the capability	Must be bench tested before procurement/production dependence
OPERATOR-CLAIM	External operator statement	Hypothesis/economic input only unless corroborated
INDUSTRY-AGGREGATE	Repeated community/creator claims	Useful for experiment design, not production constants
INFERENCE	Derived analysis from multiple facts	Useful when assumptions are explicit and falsifiable
OPEN	Not resolved	Architecture cannot depend on it until tested or verified
POLICY	Chosen organizational boundary	Enforced even if a different technical path exists

Source precedence

When sources disagree, use this order:

1. internal bench measurement for the exact hardware/software version being built;
2. current first-party OS/platform documentation;
3. maintained open-source implementation documentation;
4. vendor technical documentation;

5. independent engineering reports;
6. operator testimony;
7. forums/marketing folklore.

This does **not** mean a bench observation proves a universal platform rule. It means it is the most relevant evidence for whether *our exact hardware path works*.

27 August 2026 research update — what changed after another web pass

This V1 edition rechecked the Mac-first path and several architecture claims against current first-party or maintained technical sources. The most important corrections are:

1. **Mac tethered iOS screen capture:** promote from “programmatic mechanism OPEN” to **VERIFIED-MECHANISM / SCALE-OPEN**. QuickTime documents cable-connected iPhone/iPad screen capture; Apple exposes `kCMI0HardwarePropertyAllowScreenCaptureDevices` in `CoreMediaIO`. The unverified part is concurrent capacity and stability at farm scale.
2. **Apple Configurator:** current Apple documentation explicitly allows multiple iPhone/iPad devices connected to Mac ports or powered USB 2.0+ high-speed hubs/carts and modified simultaneously. This supports Configurator as a real bulk-provisioning tool, not a one-phone-at-a-time assumption.
3. **Input semantics:** modern UIKit can expose indirect pointing-device input as `indirectPointer`. Therefore “a mouse/HID click is always indistinguishable from a finger” is incorrect as a general OS-level claim. Whether any specific third-party app uses that information remains unknown.
4. **WDA/Appium:** real-device XCUITest automation requires the WDA helper app, device trust, Developer Mode on iOS/iPadOS 16+, UI Automation, and a valid provisioning profile. It is a supported QA route but not a no-install route.
5. **Developer signing limits:** a free Xcode Personal Team is limited to three devices per platform and seven-day profiles; a paid Apple Developer Program membership is USD 99/year and registered development/ad-hoc devices are capped at 100 per product family per membership year.
6. **iPhone Mirroring:** still ruled out for a farm. Apple documents one Mac and one iPhone at a time, same Apple Account, Bluetooth/Wi-Fi, and a locked nearby phone.
7. **GADS:** useful as a device-lab UI if WDA is accepted, but current iOS streaming is WDA-based and the current repository has a proprietary UI component in addition to AGPL code.
8. **Remote Mac security:** on Apple-silicon Macs with macOS 26+, Apple documents FileVault unlock over SSH after restart when Remote Login and network connectivity are available. This weakens the argument for disabling disk encryption on a remote rack.
9. **Android:** scrcpy remains a strong authorized-lab contrast: USB/TCP mirroring and control, no root, cross-platform. Android Management API provides policy-driven fully managed/dedicated-device provisioning, but its permissible-usage rules must be checked before adopting it for a commercial management product.
10. **Reset-surviving backend state:** Apple DeviceCheck exposes two per-device bits on Apple’s server; Google’s Play Integrity “device recall” beta can recall developer-defined values after reinstall and device reset. Resets therefore cannot be modeled as universal server-state erasure.

Source gap recorded

[OPEN] Concept — Long-Term Organic Device Set.md is referenced by the supplied Technical Brief, but it was **not present among the files supplied in this conversation and was not found by the Library search performed for this task**. This handbook does not claim to have read or incorporated that document. The durable 5–30-device model is therefore derived only from the sources actually available.

27 August 2026 second web research pass - new corrections and engineering consequences

The second research pass concentrated on the places where the first V1 could still mislead a builder even when its high-level architecture was correct. The biggest gains were not new product names. They were **failure semantics, unattended recovery, version-specific host behavior, and licensing/eligibility constraints.**

A. Tethered iOS capture is real, but “stream exists” is not a health check

[INDEPENDENT-ENGINEERING] A current macOS utility built on the same AVFoundation/CoreMediaIO path documents several failure modes that are easy to miss in a rack: a device may still enumerate while delivering no frames; locking can leave a stale last frame; resetting trust can leave an apparently running session with no sample buffers; and some mirroring-state changes can require an iPhone reboot before video resumes.

Feynman version: a camera can stay listed in the computer even when the lens has stopped sending pictures. Therefore the farm must test **freshness of frames**, not merely whether `AVCaptureSession.isRunning` or a device object exists.

Engineering consequence: every iOS capture adapter should expose at least `last_frame_at`, `frame_sequence`, `fps_window`, `stale_for_ms`, `device_locked_or_unknown`, `capture_restart_count`, and a bounded recovery state machine. A screenshot that is two minutes old is not “online.”

B. Powered USB hubs are first-party-supported for Configurator, but the exact farm topology remains empirical

[VERIFIED-FIRST-PARTY] Apple explicitly documents connecting multiple iPhone/iPad devices to a Mac directly or through **powered USB 2.0-or-later high-speed hubs/carts** and modifying them simultaneously with Apple Configurator.

That supports a hub-based pod as a legitimate management topology. It does **not** prove that an arbitrary hub will sustain simultaneous screen capture, charging, provisioning and a third-party control board. Each of those can stress a different USB function or power path.

C. Unattended iPhone racks need an accessory-access policy, not hope

[VERIFIED-FIRST-PARTY] Current iOS exposes Wired Accessories controls. On supervised devices, Apple MDM restrictions include `allowUSBRestrictedMode`; setting it to `false` allows iOS devices to connect to USB accessories while locked, unless Lockdown Mode overrides the setting.

Feynman version: a cable can be physically connected but logically refused. For a remote rack, “USB present” and “USB authorized” are separate states.

Engineering consequence: commissioning must record trust/pairing state, accessory policy, supervision state, Lockdown Mode state, and reconnect behavior after lock, restart and power interruption. Do not weaken this security control fleet-wide until the bench proves it is necessary for the chosen architecture.

D. The 10GbE Mac mini option can buy recovery capability, not just network speed

[VERIFIED-FIRST-PARTY] Apple Lights Out Management supports the 2024 Mac mini **when configured with 10Gb Ethernet**, after device-management enrollment. It can remotely start, shut down and restart managed Macs through another configured Mac controller on the same local Ethernet subnet.

[INFERENCE] For a rack that must be operated remotely, the 10GbE upgrade may be worth evaluating even when normal farm traffic fits comfortably in 1GbE. Its strategic value is the optional Lights Out Management path, not a claim that phone video needs 10Gb/s.

Limits matter: this is not a magic independent BMC. It requires MDM, Ethernet, IPv6, a same-subnet controller Mac, and the documented Apple management workflow.

E. macOS remote recovery now has three distinct layers

[VERIFIED-FIRST-PARTY] Apple documents high-performance Screen Sharing on Apple-silicon Macs, Remote Login/SSH, and - on macOS 26+ Apple silicon - FileVault unlock over SSH after restart when network connectivity and Remote Login are available.

[INFERENCE] The rack should therefore use a **recovery ladder** rather than one remote-desktop product:

1. structured control-plane/node-agent health;
2. SSH for service/process/network repair;
3. remote desktop or Apple Screen Sharing for GUI-only work;
4. optional 10GbE Lights Out Management for power actions;
5. physical break-glass visit when all remote layers fail.

RustDesk can still be useful, but it should be one rung, not the only lifeline.

F. Android Management API had an important eligibility caveat

[VERIFIED-FIRST-PARTY] Google currently restricts Android Management API use to commercial EMM developers, Device Trust solution providers and OEMs under its permissible-use policy, and explicitly lists solutions used exclusively for first-party in-house applications as disallowed.

Correction to a common architecture shortcut: “we own Android phones, so we can just build our internal fleet on Android Management API” is not a safe assumption.

[INFERENCE] For an internal device fleet, evaluate a commercial Android Enterprise EMM, an appropriate Device Policy Controller/dedicated-device architecture, or request clarification from Google before making Android Management API a hard dependency.

G. Android and iOS host tooling must be version-aware

[MAINTAINED-REPO] `usbmuxd` continues to provide multiplexed USB connections to iOS devices. For iOS 17+, however, maintained `pymobiledevice3` documentation records Apple’s shift of many developer services to `CoreDevice/RemoteXPC/RSD` tunnel flows; iOS 17.4+ has a materially simpler USB userspace path than iOS 17.0-17.3.1.

Feynman version: “iOS over USB” is not one protocol. It is a stack whose upper layers changed. A command that works on 17.4+ can fail on 17.1 for transport reasons even though the cable and trust pairing are fine.

Engineering consequence: every node-agent adapter must report iOS version and selected transport path, and the compatibility matrix must be tested by OS family rather than assuming one set of host commands fits every phone.

H. Photo-library ingestion has a clean first-party mechanism when an on-device helper is allowed

[VERIFIED-FIRST-PARTY] Apple’s `PhotoKit PHAssetCreationRequest`, used inside a `PHPhotoLibrary` change block, creates photo/video assets from data resources and adds them to the Photos library.

This clarifies the architectural choice. A helper app can legitimately turn staged bytes into a real Photos asset. A raw file-transfer service is only transport. If the policy forbids a helper app, the media problem remains a separate system constraint rather than something AFC/USB copying magically solves.

I. GADS is useful, but its iOS dependency and source model must be explicit

[MAINTAINED-REPO] Current GADS documentation describes iOS streaming through WebDriverAgent and states that the distribution includes both open-source and proprietary/obfuscated components. It is therefore best evaluated as a **device-lab product candidate**, not treated as a transparent no-phone-app control primitive.

J. 2026 Apple management changes strengthen fleet observability, but are version-gated

[VERIFIED-FIRST-PARTY / PRE-RELEASE-PLATFORM] Apple's WWDC26 management material describes additional declarative status reporting in iOS/iPadOS 27, including device system health information, plus enhanced log-collection workflows and broader declarative management changes.

Engineering consequence: design the fleet telemetry schema so it can ingest richer first-party device-health status when the deployed OS/MDM stack supports it. Do not make a 2026 pre-release feature a Gate-1 dependency for older phones.

What this pass did not prove

The following remain OPEN and are intentionally not replaced with estimates:

- simultaneous **live** tethered capture ceiling on the exact Mac mini at N=5/10/20;
- exact control-board USB role and whether capture + charging + input coexist;
- long-duration stability of the selected powered hub/cable/controller combination;
- whether the vendor control software is acceptable after supply-chain/network inspection;
- the fastest reliable path that imports media into Photos under the final no-helper/helper policy;
- exact operational behavior already observed by Dim but not yet captured in a reproducible test record;
- the missing Concept – Long-Term Organic Device Set.md source.

27 August 2026 third web research pass - V1.2 architecture refinement

This pass concentrated on the remaining architecture questions that can still change the first Mac pod: **CoreDevice/RSD as a no-WDA control candidate, multi-device USB state, deterministic rack-slot identity, automated commissioning, provisioning bandwidth, lifecycle/recommissioning, and headless recovery.**

A. CoreDevice/RSD is now a first-class iOS 17.4+ candidate, but not a production promise

[MAINTAINED-DOC] Current pymobiledevice3 documentation states that iOS 17+ developer services use CoreDevice/RemoteXPC through an RSD tunnel. On iOS 17.4+ over USB, its default path can establish an in-process userspace tunnel on macOS, Windows and Linux without root/admin. The same maintained tool exposes CoreDevice display functions, HEVC streaming, screenshot capture, HID tap/drag/swipe and hardware-button operations.

Feynman version: instead of installing a small robot app on the iPhone and asking that app to press buttons, the Mac can potentially talk to Apple device services that are already part of the development stack. That removes WDA from the candidate path. It does **not** make the path an Apple-supported fleet-control API: pymobiledevice3 is a maintained reverse-engineered implementation of private/semiprivate device protocols, so OS changes can break assumptions.

Engineering consequence: V1.2 promotes CoreDevice/RSD to the **first candidate for modern durable iPhones**, with WDA/GADS and external hardware retained as fallbacks. The go/no-go question is no longer “does control exist?” but “does **simultaneous HEVC + HID remain stable at 1, 2, 5, 10 and 20 devices on our exact Mac/hub/OS matrix?**”

B. iOS 17.x is not one transport class

[MAINTAINED-DOC] The maintained tunnel guide distinguishes iOS 17.0-17.3.1 from iOS 17.4+. The earlier range is routed through privileged tunneld by default; iOS 17.4+ can use CoreDeviceProxy and an in-process userspace USB tunnel without privilege.

Feynman version: “iOS 17” is like saying “Windows 11” when two builds use different network drivers. The label is too broad for the device adapter. The inventory must record the exact OS version and select the transport accordingly.

C. High device counts can expose host-tooling bugs that do not appear on a two-phone bench

[INDEPENDENT-PROJECT-EVIDENCE] A historical closed pymobiledevice3 issue reported device enumeration failure on Windows with more than 30 connected iPhones. Current open issues also include multi-device mounting and concurrent RSD questions. These reports do **not** establish a current 30-device limit or a Mac limit.

Feynman version: a staircase can be perfectly safe on the first three steps and fail on step thirty because a hidden counter, port namespace, queue or resource limit was never exercised. Therefore a “works with two phones” result is evidence for mechanism, not evidence for capacity.

Engineering consequence: the scale test must exercise **enumeration, pairing, DDI state, video, input, reconnect and long-running orchestration simultaneously**, not merely count USB devices.

D. Configurator can become the commissioning engine, not just a manual utility

[VERIFIED-FIRST-PARTY] Apple Configurator Shortcuts can run automatically when devices are attached or detached. Current actions include finding connected devices, installing profiles, preparing supervised devices, restoring/updating, setting device status, waiting for battery level, and assigning port/station numbers with supported USB hardware. Apple Configurator also exposes device information such as OS version, serial number, hardware IDs/addresses, capacity and console logs, and can export much of it to CSV.

Feynman version: the Mac can treat “a phone was plugged into slot 7” like a server rack treats “a disk appeared in bay 7.” The commissioning workflow can identify it, check policy, prepare it, log it and either mark it ready or quarantine it.

Engineering consequence: V1.2 adds a deterministic identity chain:

```
rack_slot -> Configurator port/station -> USB topology/location -> ECID/UDID/serial -> inventory
device_id -> telemetry/task history.
```

The chain must be tested for replug, cable swap, hub replacement and Mac reboot so that a physical phone can never silently become the wrong logical device.

E. Supervision identity and host pairing are security boundaries

[VERIFIED-FIRST-PARTY] Apple documents supervised-device restrictions that can prevent pairing with arbitrary hosts and allow access only from computers holding the appropriate supervision host certificate. Apple also documents that pairing trust records can be cleared by reset/erase and may be removed after long periods of non-use.

Feynman version: USB trust is a key, not a cable. The rack can be physically connected while the Mac is not logically authorized to manage the phone.

Engineering consequence: paired, trusted, supervision_identity, host_pairing_policy, and last_pairing_verified_at become explicit inventory/health fields. The supervision identity must be backed up and controlled as a fleet credential; casually replacing it can force device re-preparation.

F. Content Caching creates a separate commissioning/provisioning lane

[VERIFIED-FIRST-PARTY] macOS Content Caching can serve Apple software/apps to multiple tethered iPhone/iPad devices through a cart or USB hub. Apple also documents tethered Internet sharing from an Ethernet-connected Mac to physically connected iOS devices. This is useful for staging, OS/app updates and managed-device check-in.

Feynman version: if twenty phones all need the same 8 GB update, the Mac can act like a local warehouse instead of making twenty delivery trucks fetch the same box independently.

Engineering consequence: keep **commissioning/provisioning traffic** conceptually separate from **production device egress**. Content caching and tethered Internet are deployment tools, not an identity or enforcement-avoidance mechanism. Measure cache hit ratio, origin bytes, update duration and USB/hub contention during bulk staging.

G. Return to Service belongs in the legitimate lifecycle plane

[VERIFIED-FIRST-PARTY] Apple Device Management provides Return to Service workflows that can erase a managed device and automatically re-enroll it, optionally using supplied Wi-Fi/MDM configuration. Newer OS versions add app-preservation/retry capabilities.

Feynman version: instead of an operator erasing a phone and manually rebuilding it from memory, the fleet can have a documented factory-reset-and-recommission conveyor belt for organization-owned devices.

Engineering consequence: add IN_SERVICE -> RETIRING -> ERASE_REQUESTED -> RETURN_TO_SERVICE -> REENROLLING -> CONFIGURING -> READY to the authorized device lifecycle. This is for normal asset reuse, repair, employee/client reassignment and lab reset; it is not a ban-replacement workflow.

H. The Mac pod has a real remote-recovery stack now

[VERIFIED-FIRST-PARTY] A 2024-or-later Mac mini on macOS 26.5+ can be configured to start automatically when power is connected. Apple Lights Out Management can remotely start, shut down and restart supported Mac minis, but the 2024 Mac mini requires the 10Gb Ethernet option and the documented MDM/network topology.

Feynman version: a remote rack needs recovery below the application layer. Restarting the pod-agent is level 1; rebooting macOS is level 2; recovering after a power outage is level 3.

Engineering consequence: the 10GbE option is evaluated for **resilience**, not because phone traffic requires 10 Gb/s. A cheaper 1GbE M4 Mac mini remains a valid pilot if ordinary remote-access plus automatic power-on meets the recovery requirement.

I. USB topology becomes observable data

[VERIFIED-FIRST-PARTY] Apple's USB host APIs expose device operational speed and port status, and Apple platforms expose USB-associated location identifiers. USB high speed is 480 Mb/s; SuperSpeed classes are 5/10/20 Gb/s. Apple Configurator can also assign logical port/station numbers with supported hardware.

Feynman version: "USB connected" is too vague. We need to know *where* the device is, *how fast* that link negotiated, and whether the same physical branch keeps failing.

Engineering consequence: record the host port/hub path, observed speed, rack slot and logical device identity. This lets the system detect a downgraded cable, dead hub branch, wrong replug or localized disconnect storm.

J. WDA remains a useful fallback, and its parallelism has explicit namespaces

[MAINTAINED-DOC] Appium documents parallel real-device XCTest sessions, with unique device UDIDs and unique `wdaLocalPort`; if MJPEG is used, each session also needs a unique `mjpegServerPort`, and unique derived-data paths are recommended. Current real-device setup still requires a provisioned WDA application, trust and Developer Mode.

[MAINTAINED-REPO] GADS remains actively developed in 2026, but recent releases and issues still contain WDA/DDI-specific fixes. That makes it useful as a reference/control surface while reinforcing that WDA health must be monitored as its own subsystem.

K. Battery and thermal state are lifecycle inputs, not cosmetic telemetry

[VERIFIED-FIRST-PARTY] Apple exposes device battery-health states such as normal, service-recommended and non-genuine. Current iPhone guidance says high temperature can slow/stop charging and damage battery longevity; iPhone 15 and later allow a charge limit between 80% and 100% in 5% steps. Apple also exposes process thermal states from nominal through critical.

Feynman version: an always-plugged phone is a tiny battery-powered computer living next to a heater. If the battery is constantly full and hot, the farm is consuming the battery as a wear item even while the phone “works.”

Engineering consequence: battery health, charging interruptions, ambient temperature, device temperature indicators where available, and maintenance state belong in capacity planning. Model-specific charge-limit settings may help a durable fleet, but V1.2 does not assume they can be centrally enforced unless the chosen management stack documents that capability.

What this third pass still cannot verify

[OPEN] Public documentation still does **not** give a reliable “one M4 Mac mini = N simultaneous iPhones with CoreDevice HEVC + HID” number. It also does not answer whether the specific inbound external control board can coexist with the chosen Mac USB data/display path. Those remain bench gates.

[OPEN] Concept – Long-Term Organic Device Set.md is still not present in the supplied/searchable source set, so V1.2 cannot claim coverage of that document.

How to learn this handbook — Feynman method

This edition is written so a non-specialist can become technically useful without memorizing product names. For every subsystem, answer five questions in your own words:

1. **What job does this subsystem perform?** One sentence, no jargon.
2. **What goes into it and what comes out?** Power, packets, frames, commands, files, permissions or audit events.
3. **What can fail?** Name the physical or software failure, not “it doesn’t work.”
4. **How would we prove it works?** A measurable test with versions and evidence.
5. **What does it *not* solve?** This is the most important question in phone-farm engineering because display, control, management, networking and platform policy are often incorrectly collapsed into one problem.

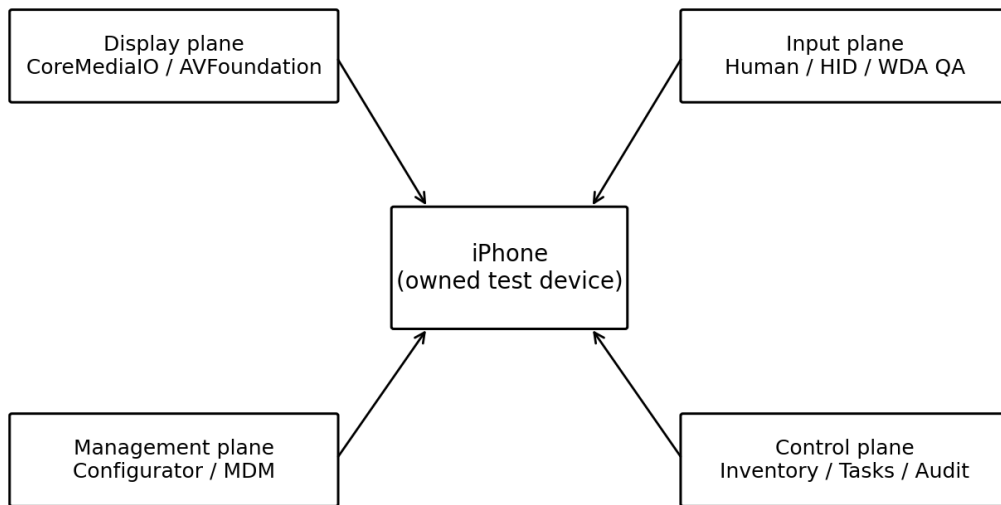
A useful learning loop is: read the plain-English model → sketch it from memory → explain it to someone else → check the detailed section → run the bench test → update the ADR.

The five planes you should never confuse

Plane	Plain-English question	Typical tools	Does not automatically solve
Physical	Is the device powered, cool and connected?	PSU, hub, rack, fan, cable	UI control
Display	Can an operator/software observe the screen?	CoreMediaIO/ AVFoundation, scrcpy, vendor mirror	input
Input	Can an authorized test/ operator send an action?	physical touch, HID, WDA/ XCUITest, ADB	media ingestion
Management	Can IT configure/update/ wipe/inventory the device?	Configurator, MDM, Android Enterprise	arbitrary third-party UI driving
Control plane	Can the organization coordinate devices/tasks/ people/audit?	coordinator, database, node agents, dashboards	underlying device capability

If a vendor says “we control twenty phones,” ask which of these five planes they actually solve.

Do not confuse the four control planes



Seeing $\neq$ touching $\neq$ managing $\neq$ coordinating

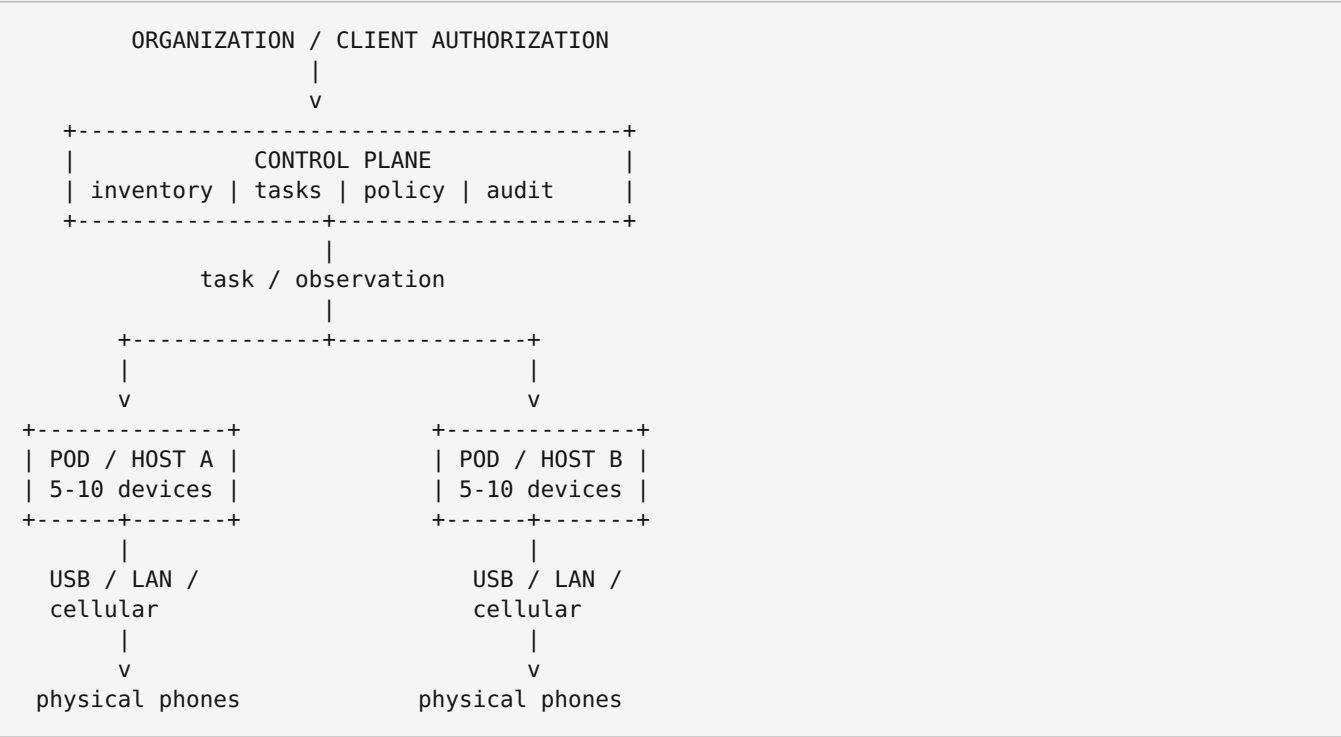
Figure 2 — Display, input, management and control plane are separate jobs.

Executive Synthesis

What this project actually is

[INFERENCE] The phrase “phone farm” hides the real engineering problem. The system is a compact private cloud whose compute nodes happen to be phones. It has power distribution, network interfaces, stateful workloads, identity, storage, scheduling, telemetry, operators and failure domains. Treating it like “20 phones on a hub” is equivalent to treating a server rack as “20 boxes plugged into a switch.”

A useful system model is:



The scaling unit is the **pod**, not the individual phone and not a 100-device monolith. A pod has a bounded power domain, host, hub topology, network policy and failure radius. This makes capacity measurable and incidents recoverable.

Phone farm mental model: a small private cloud

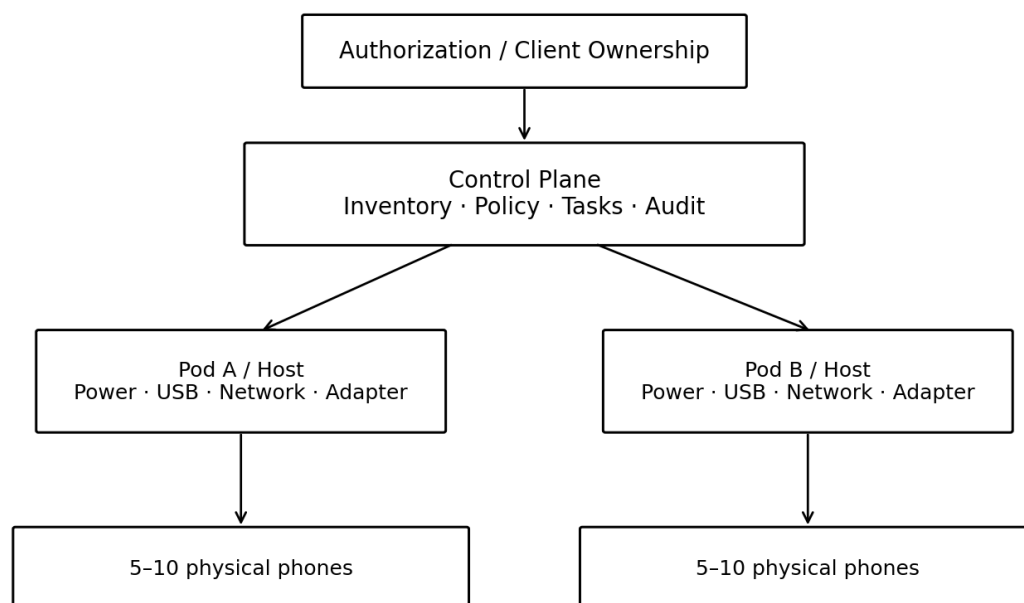


Figure 1 — A phone farm is a small private cloud; pods are the scaling unit.

The most important updated conclusions

1. Mac mini first: the USB screen-capture mechanism is now supported; fleet scale is still OPEN

[VERIFIED-FIRST-PARTY] Apple currently documents that QuickTime Player can capture the screen of an iPhone/iPad connected to a Mac. Apple also publishes the CoreMediaIO property `kCMIOHardwarePropertyAllowScreenCaptureDevices`. This is materially stronger than the earlier position that the programmatic path itself was unproven.

[MAINTAINED-IMPLEMENTATION] Current macOS capture implementations and sample projects use the same CoreMediaIO/AVFoundation device-discovery path to expose tethered iOS screen-capture sources. That is enough to promote the **mechanism** from OPEN to **VERIFIED-MECHANISM**.

[OPEN] What remains unverified is the number that matters to this project: how many simultaneous iPhone screen streams one M4 Mac mini can sustain on the actual phones, cables, hubs and macOS/iOS versions. A feature existing is not the same as a 20-device architecture working. Therefore the status is **VERIFIED-MECHANISM / SCALE-OPEN**.

This makes the client's sequencing correct: put a Mac mini on the bench immediately, measure $1 \rightarrow 2 \rightarrow 5 \rightarrow 10 \rightarrow 20$ streams, and do not buy the second-stage fleet hardware on the assumption that the result will be 20.

2. Display and input must be designed independently

Dim's Xiaowei test is valuable because it proves the conceptual separation: the PC could see the phone only through same-network mirroring; it could not automatically control it. The main input options remain external HID/control hardware or a signed on-device automation component such as WebDriverAgent.

[VERIFIED-FIRST-PARTY] Apple’s UIKit documentation matters here: from iOS 17 onward, an app that has not opted out of indirect input support receives pointing-device clicks as `indirectPointer`, whereas pre-iOS-17 compatibility defaults behaved as direct touch. This proves that pointer input **can be distinguishable at the OS API**. It does not prove any social platform uses the signal.

3. Reset is not a universal identity eraser on either iOS or Android

[VERIFIED-FIRST-PARTY] Apple DeviceCheck persists two developer-defined bits and a timestamp per device across app reinstall, device transfer and Erase All Contents and Settings. [VERIFIED-FIRST-PARTY] Google’s 2026 Play Integrity device-recall beta similarly lets participating apps recall custom per-device state after reinstall and device reset.

Therefore the correct architecture statement is not “iOS is permanently fingerprinted” or “Android resets cleanly.” It is: **ordinary raw identifiers and backend anti-abuse state are different mechanisms**. A system must not base asset economics on the assumption that a reset clears all server-side device state.

4. Client isolation is a first-class architecture problem

The new account-separation research changes the priority order. Shared business containers, credential custody, session bleed, billing entanglement, poor staff offboarding and vendor concentration create a larger practical blast radius than exotic fingerprint theories.

[INFERENCE] The system should model `Client`, `Authorization`, `CredentialReference`, `PlatformAsset`, and `OperatorRole` as core database entities rather than metadata attached later.

5. “Warmup” is not one variable

The Instagram warmup source contains directly contradictory successful practices: immediate posting, several-day delays, 14-day routines, and month-long routines. Proxy and device claims conflict as well. This is evidence that the folk concept “warmup” bundles many different variables.

[INFERENCE] A better model is:

```
observed outcome = f(account provenance,
                      policy/recommendation eligibility,
                      content quality,
                      audience fit,
                      account age,
                      activity history,
                      current platform enforcement,
                      operator behavior,
                      random/recommendation variance)
```

The handbook therefore treats warmup advice as **experiment hypotheses**, not a clock-based SOP.

6. The first milestone is not growth; it is deterministic operation

Before scaling traffic, the system must prove: reliable enumeration, device identity, screen/health observation, controlled input on owned test devices, authorized media delivery, task execution, remote recovery, power/thermal stability, audit and operator separation.

A strong pilot exit criterion is not “it posted once.” It is **twenty consecutive unattended task cycles with correct attribution and no unexplained device loss**, followed by a soak test and recovery test.

Section Map

Section	Topic	Main question answered
0	Orientation, evidence, boundaries	What do we know, and how confident are we?
1	System model & operating choices	What are we actually building?
2	Physical plant	How do the devices stay powered, cool, connected and serviceable?
3	iOS engineering	How do iPhones identify, provision, display, accept input and ingest media?
4	Android engineering	What is easier/different on Android, and what integrity assumptions are unsafe?
5	Networking	How do local control and Internet egress work without conceptual mistakes?
6	Client/account separation	How do we keep one client/staffer/vendor failure from becoming everyone else's?
7	Content & platform evidence	What is documented, what is measured, and what is folklore?
8	Control plane & remote ops	What software coordinates the fleet and humans?
9	Reliability, security, incidents	How does the farm fail safely and observably?
10	Experiments, pilot & scaling	What must be proven before purchasing/scaling?
11	Claude/engineering execution	What exactly should the implementation agent build first?

Mac-first bench plan — the work that can start tomorrow

The client explicitly asked to start with the Mac mini because the inbound control board is roughly two weeks away. This sequencing is technically sound because the Mac path can answer several independent questions before that hardware arrives.

Bench objective

By the end of the first bench cycle, the team should know whether a Mac mini is a good **pod host**, not merely whether one iPhone appears in QuickTime.

Phase A — commission the host

- Record exact Mac model, RAM, SSD, macOS build and network interface.
- Enable encrypted storage and document remote-recovery method; if macOS 26+ is used, test the supported FileVault-over-SSH recovery path before the Mac becomes unattended.
- Configure a named operator/admin model; do not share one personal login.
- Install Apple Configurator, current Xcode only if WDA/QA testing is in scope, the capture probe, monitoring and remote-support tooling.
- Record TCC/privacy permissions granted. A remote farm that cannot be re-commissioned after an OS update is not remote-operable.

Phase B — prove one-phone USB display

1. Connect one known iPhone with a known good data cable.
2. Trust the Mac on the phone.
3. Confirm QuickTime sees the phone's screen.
4. Confirm the programmatic capture probe enumerates the same physical phone as a screen-capture device.
5. Record resolution, frame rate, latency, process CPU/memory and reconnect behavior.
6. Lock/unlock the phone and disconnect/reconnect the cable; record which transitions recover automatically.

Phase C — discover the scaling curve

Repeat at 2, 5, 10 and—only if stable—20 devices. Use powered hubs suitable for simultaneous data, not charge-only splitters. The test stops at the first level that produces unexplained disconnects, unbounded resource growth or unacceptable latency. Fix the reason before adding devices.

Phase D — bulk provisioning

Use Apple Configurator on several dedicated test phones. Verify exactly which preparation/supervision/profile actions the desired fleet configuration can express. Apple documents simultaneous modification through Mac ports or powered high-speed USB hubs/carts, but the team's exact settings still need a local acceptance test.

Phase E — remote-operation rehearsal

Have a team member who is not physically at the bench connect through the chosen host-maintenance channel. They should be able to identify a device, inspect health, recover the node agent and escalate to one live screen without streaming an entire wall of phones through a remote desktop session.

What waits for the control board

The board's arrival unlocks the input-coexistence question. Until it is physically tested, do not assume that its Lightning/USB topology allows simultaneous Mac capture, charging and input. The correct experiment is TEST-COEXIST-001, not a procurement guess.

SECTION 1 - System Model and Architecture Choices

Feynman primer — a phone farm is a tiny private cloud

A beginner often sees “many phones.” An engineer should see **many unreliable computers that share infrastructure**. Each phone has power, a data path, an identity, software state, network state and an owner/authorization context. A farm is the system that makes those devices observable and recoverable. If you cannot answer “which physical phone did this, through which host and network, for whose authorization?” you have a pile of phones, not a fleet.

The pod is the basic Lego brick. Build one brick that survives; then copy it.

A phone farm is only useful when the business process around it is explicit. This part defines the OFM use case, separates legitimate fleet operations from manipulation, and establishes the architecture used throughout the manual.

1. Phone farms in OFM

In an OFM context, a phone farm is a managed collection of physical mobile devices used by an agency or creator team to operate authorized social accounts, prepare and publish content, review notifications, collect performance signals and support traffic acquisition. The physical phones are execution endpoints; the real system is the combination of devices, people, network, content rights, credentials, workflow software and analytics.

Industry usage and terminology

Public operator communities use “phone farm” loosely. It can mean ten old iPhones on a desk, a rack of Android phones, a motherboard rack without screens, a set of remotely controlled devices, or a larger operation with coordinators and staff. These reports are anecdotal and should not be treated as evidence that a particular platform will tolerate the associated behavior.

The legitimate engineering interpretation is closer to a corporate mobile fleet or device lab: each device is inventoried, assigned, secured, connected to controlled networks, monitored and maintained. The difference is that the workload is creator-content operations rather than QA testing or retail kiosk use.

- Define the owner of every device and social account.
- Treat creator authorization and content rights as first-class records.
- Separate device administration from social publishing.
- Measure output with business metrics rather than “number of phones.”

What the farm is not

A physical phone is not an invisibility device. Platforms can evaluate account behavior, content similarity, network characteristics, login history and many other signals. A real handset may make the workflow more operationally convenient, but it does not convert prohibited behavior into permitted behavior.

The architecture should therefore optimize for repeatability, security, operator throughput, reliability and attribution. It should not optimize for hiding the existence of the fleet.

- Do not buy hardware before mapping the traffic funnel.
- Do not assume one physical device must equal one account. The correct mapping depends on authorization, platform rules, staffing and operational risk.

Engineering checklist

- Write down the business goal for each platform.
- List authorized creator accounts and account owners.
- Define which actions are manual, API-driven or scheduler-driven.
- Create a platform-rule register with review dates.

2. Traffic funnel and business model

The phone-farm layer sits in the middle of a larger creator acquisition funnel. Engineering decisions should be evaluated by their effect on measurable funnel stages rather than raw posting volume.

Reference funnel

A typical funnel is: approved creative asset -> platform post -> profile visit -> tracked link or landing page -> creator page -> subscription or other paid action. Depending on the business, there may be additional layers such as a link hub, geofenced landing page, email capture, or CRM attribution. Every added layer creates potential conversion loss and additional privacy/security obligations.

The control plane should attach campaign, creator, platform, account, device, operator and asset identifiers to each publishing task. The analytics plane should connect those identifiers to post-level reach where available and to downstream click/conversion events where lawful and technically possible.

- Primary operational metrics: task completion rate, post success rate, account availability, content approval latency.
- Primary marketing metrics: reach, profile visits, click-through rate, landing conversion, subscriber conversion, revenue per thousand impressions where calculable.
- Primary reliability metrics: device heartbeat, battery/thermal status, network packet loss, API error rate, queue age.

Unit economics

A useful model treats each account or campaign as a cost center. Hardware amortization, software, connectivity, employee time, content production and creator revenue share should be visible. “More phones” only creates value if incremental gross contribution exceeds incremental infrastructure and labor cost.

Attribution is imperfect because not every platform exposes the same analytics and some downstream conversions cannot be deterministically tied to a post. The system should distinguish observed events from modeled or inferred attribution.

Layer	Example measure	Engineering owner
Creative	Approved assets per week; QC rejection rate	Content ops
Publishing	Task success; queue age; retries	Fleet/backend

Layer	Example measure	Engineering owner
Platform	Reach; profile actions	Growth ops
Landing	Unique clicks; conversion	Web/backend
Revenue	Paid conversion; ARPU/LTV	Finance/analytics

3. What physical phones solve - and what they do not

Physical devices are valuable when the workflow actually requires the native mobile app, hardware-bound capabilities, mobile camera/media access, or a standardized operator interface. They are less valuable when an official API or web console already provides the required function.

Figure 3-1. Control-plane and data-plane separation.

Problems physical devices solve

Phones provide the authentic application runtime, camera roll, push notifications, biometric-secured credential storage, mobile-only creation tools and native moderation/status screens. Organization-owned devices can also be physically segmented from employees' personal phones.

For Android, corporate devices can be fully managed or dedicated under Android Enterprise. For Apple fleets, organization-owned supervised devices can receive stronger MDM controls than unsupervised personal devices. [S1][S5]

- Native-app-only features.
- Consistent operator work surface.
- Separation between personal and corporate credentials.
- Physical inventory that can be wiped, reassigned and retired.

Problems they do not solve

Phones do not remove platform terms, content restrictions, API rules or behavioral enforcement. They also do not automatically solve account ownership, credential leakage, employee mistakes, attribution or creator-consent management.

A rack can even make risk worse if the same content is repetitively distributed, credentials are broadly shared, devices are not patched, or network architecture creates one large blast radius.

- Build authorization and access control before scaling hardware.
- Use official publishing APIs where available and suitable.
- Keep humans in approval loops for ambiguous or high-risk actions.

4. Platform-policy and authorization boundary

The most important non-hardware requirement is an explicit rule boundary. Different platforms permit multiple accounts, third-party management, APIs and scheduling to different degrees, but commonly prohibit spam, fake engagement, coordinated inauthentic activity, automated bulk account operation, or circumvention of enforcement. These rules can change.

Policy update rule

Treat all platform-specific operating rules in this manual as a dated snapshot, not a permanent guarantee. Re-check first-party policies and API terms before deploying a workflow change.

Current examples from first-party rules

TikTok's integrity rules distinguish authentic multi-account use from deceptive or spam behavior and prohibit automation used to operate accounts in bulk or manipulate platform systems. TikTok also provides a formal Content Posting API for authorized applications, with scopes and audit requirements. [S22][S23]

Reddit's rules require authentic participation and prohibit spam and manipulation. Snapchat explicitly prohibits spam and artificially inflated engagement. X permits legitimate third-party account management but prohibits coordinated inauthentic activity, duplicate amplification and ban-evasion behavior. [S24][S26][S27]

Authorization records

For every creator and account, retain a signed or otherwise auditable authorization record stating who owns the account, what the agency may do, which employees or systems may access it, what content is approved, and what happens on termination. The agency should not rely on informal chat messages as its only control evidence.

Keep platform credentials separate from legal identity documents and sensitive creator files. The system should store credential references to a vault rather than passwords in application tables or spreadsheets.

- Creator ID and contract/authorization reference.
- Account owner and recovery email/phone owner.
- Permitted actions and publishing platforms.
- Content usage scope and expiration/revocation process.
- Offboarding/wipe/recovery procedure.

5. Architecture decision: API-first, device-first or hybrid

The correct architecture depends on which functions are officially exposed by platform APIs and which require a person in a native application. A hybrid design is usually the most maintainable: APIs or approved schedulers handle supported publishing and analytics; physical phones handle native-only tasks and human review.

Figure 5-1. Hybrid reference architecture.

API-first

Use an API-first workflow when the platform provides a supported publishing or analytics endpoint and the agency can meet app-registration, scope and audit requirements. API-first systems are easier to log, retry, rate-limit, test and attribute. TikTok's Content Posting API is one example of an explicit, authorized posting path. [S22]

- Advantages: deterministic requests, auditability, fewer physical touch points, easier scaling.
- Costs: developer integration, platform app review, scope limitations, changing APIs.

Device-first

Device-first means a human operator performs most social actions on managed phones. This is slower to scale but can be appropriate for native-only workflows. Fleet software should coordinate assignments and record outcomes without pretending to be the human.

If the platform does not permit automated UI operation, do not turn accessibility tools, ADB input commands or computer vision into a covert substitute for an official API.

- Advantages: access to native UX; human judgment.
- Costs: labor, training, ergonomic complexity, more devices, weaker deterministic automation.

Hybrid

Hybrid systems place a coordinator above both paths. A task can route to an official API executor, an approved scheduler, or a human/device queue depending on platform, account and action type. All paths write back a common result schema.

Dimension	API-first	Device-first	Hybrid
Scale	High where API permits	Labor-bound	High for supported actions
Auditability	Strong	Requires operator capture	Strong if unified
Native-only features	Limited	Strong	Strong
Platform dependence	API contracts	App UX changes	Both
Recommended role	Default where supported	Exception/native tasks	Overall architecture

SECTION 2 - Physical Plant: Devices, Racks, Power, USB and Thermal

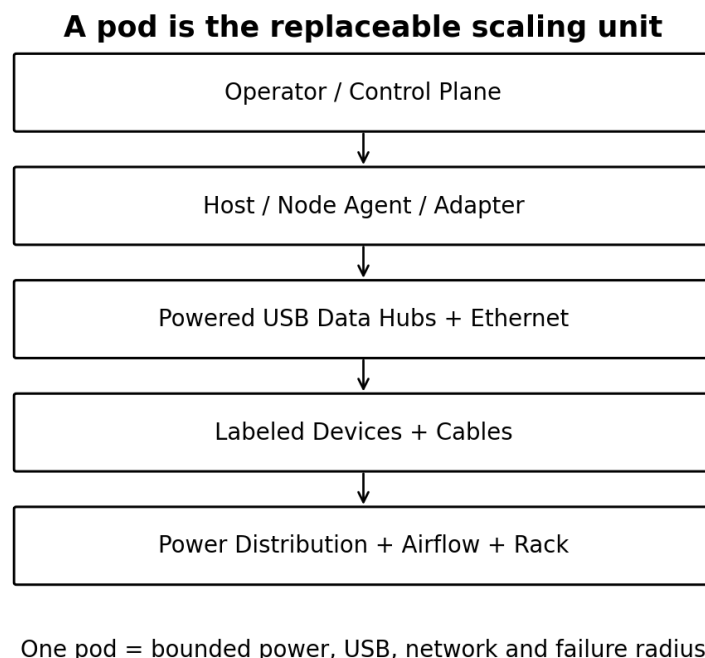


Figure 3 — The pod stack: bounded layers make failures measurable and replaceable.

Feynman primer — electricity and heat are the farm’s plumbing

Software cannot rescue a rack with bad power or bad cooling. Imagine twenty phones as twenty small heaters that also need reliable data cables. Power must be budgeted like water pressure in a building: total demand, branch limits, headroom and failure zones matter. Cooling is the drain: every watt consumed eventually becomes heat. USB is both a data bus and, often, a power path; treating it as “just a cable” is one of the fastest ways to create intermittent failures.

A rack of phones is a continuous electrical and thermal load, not a collection of casual chargers. This part treats the installation as small infrastructure: load planning, serviceability, airflow, cable discipline and lifecycle management.

6. Device selection

Device choice should be driven by supported OS life, battery condition, application compatibility, management support, reparability and total cost rather than the cheapest used handset price.

Selection criteria

Prefer models that remain inside the OS/security support window for the expected service life. Confirm the required social apps still support the OS version. For Android, verify Android Enterprise management capabilities and consider zero-touch eligible procurement for larger deployments. For Apple, organization-owned devices that can be enrolled and supervised through the intended deployment path are easier to administer consistently. [S1][S5]

Used devices can lower capital cost but introduce unknown battery health, replaced parts, activation locks, inconsistent storage and shorter remaining update life. A receiving inspection process is therefore mandatory.

- Supported OS/security updates for planned life.
- 64 GB or more storage if local media caching is heavy; size to actual workflow.
- Healthy battery and charging port.
- Stable Wi-Fi radios and compatible bands.
- Consistent model families to reduce spare-parts and cable variety.
- No activation lock, unknown MDM enrollment or carrier financing lock.

Screened vs headless hardware

Phone-motherboard racks and screenless devices appear in operator communities, but they add maintenance and compliance uncertainty because the hardware may be modified or difficult to recover. For a professional pilot, normal complete phones are easier to inventory, reset, update, visually inspect and hand to an operator.

Only consider custom hardware after the software architecture and workload are proven on standard devices.

Factor	Pilot preference	Scale preference
Model mix	1-2 models max	Standardize by cohort
OS age	Current or recent	Long remaining support
Battery	Known healthy	Replaceable service process
Procurement	Reputable used/new	Enterprise/reseller where possible
Management	Manual + MDM/Enterprise	Automated enrollment

7. Rack and mechanical design

The mechanical objective is not maximum density. It is stable airflow, easy identification, fast replacement, safe cabling and enough access that an operator can service one phone without disturbing ten others.

Figure 7-1. Example serviceable rack layout.

Slot geometry

Use vertical or slightly tilted slots so screens remain visible and warm air can rise between devices. Leave air gaps instead of clamping phones tightly together. The required spacing depends on device power, ambient temperature and fan airflow; validate it with temperature measurements rather than a fixed internet rule.

Every slot should have a stable slot ID. The physical label, inventory database and cable label should use the same identifier. Avoid relying on “third phone from the left.”

- Front access to power button and charging connector.
- No sharp edges or pressure on battery area.
- Cable bend radius and strain relief.

- Fan guards and removable dust filters where appropriate.
- A service aisle or pull-out tray for dense racks.

Material choices

Nonconductive trays simplify electrical isolation. Metal enclosures can provide stronger structure but require proper grounding and careful cable protection. 3D-printed fixtures should be made from suitable materials and kept away from hot power components; do not assume hobby plastics are fire-rated.

Keep chargers, AC distribution and high-current DC distribution in a power zone separate from phone slots. This improves cooling and makes fault isolation easier.

Engineering checklist

- Assign one immutable slot ID to every physical position.
- Provide a safe way to remove one device without pulling adjacent cables.
- Measure inlet and exhaust temperature under full load.
- Document rack weight, power-feed rating and emergency disconnect location.

8. Power architecture

Power should be designed from measured load. Charging power is dynamic: a phone may draw more while recovering from low battery, then taper as it approaches full charge, and thermal controls can reduce charging rate. USB Power Delivery negotiates power between compatible source and sink; USB PD 3.1 supports substantially higher power levels than phones normally require, so charger nameplate wattage is not the same as actual phone draw. [S10]

Figure 8-1. Illustrative power scaling model; use measured values for design.

Load model

For planning, define P_{devices} as the sum of measured average phone power, P_{peak} as the likely coincident peak, and P_{infra} as routers, switches, APs, controller PCs, fans and monitoring. Select supplies, branch circuits and UPS capacity with engineering headroom rather than operating continuously at nameplate limits.

Illustrative example only: if 25 phones average 5 W while connected, device load is about 125 W. Add the actual measured infrastructure load and then add headroom. This example is not a specification for any handset.

- Measure at the AC input with a power meter during representative use.
- Record both idle/maintenance load and recovery-from-low-battery load.
- Do not assume all multi-port chargers can deliver their advertised maximum to every port simultaneously.
- Use chargers/cables from reputable vendors with appropriate certifications for your region.

Power zoning

Split large racks into independently protected zones. A single failed adapter or tripped protection device should not remove the whole fleet. Network, control server and essential monitoring may deserve UPS runtime even if phone charging itself is not fully UPS-backed.

If chargers expose smart power allocation, test how port behavior changes when many devices are connected or disconnected at once.

Engineering checklist

- Create a port-to-slot map.
- Label every power supply with zone and rated output.
- Avoid improvised high-current wiring.
- Document shutdown/startup sequence after utility failure.

9. USB topology and cabling

USB can carry power, data or both. A rack that uses ADB over USB or Apple configuration tooling has different requirements from a charge-only rack. Treat the USB data topology as an engineered bus, not an unlimited fan-out.

Data vs charge

Charge-only cables reduce accidental data exposure but cannot support local USB administration. Data-capable cables are required for USB ADB and some provisioning workflows. Mark cable type visibly because identical connectors can have different capabilities.

Powered USB hubs should have their own adequate supplies. Avoid long chains of hubs; every tier adds connectors, failure points and potential bandwidth or enumeration problems. For large fleets, distribute devices across multiple host USB controllers and powered hubs rather than one massive tree.

- Use short, labeled cables.
- Maintain a spare cable bin by exact type.
- Log recurring disconnects by slot; intermittent cables cause disproportionate troubleshooting time.
- Use strain relief so the phone connector is not supporting cable weight.

Host enumeration

For Android, ADB identifies attached authorized devices by serial and transport. For Apple, management is normally handled through MDM for ongoing fleet operations, with Configurator or local tooling used for specific provisioning/service cases. [S3][S5]

Symptom	Likely class	Check first
Phone charges, no data	Charge-only cable / port mode	Known-good data cable
Random disconnects	Cable/hub/power instability	Hub supply + logs
Many devices disappear together	Shared hub/controller fault	Upstream hub and AC zone
Slow media transfer	Shared USB bandwidth	Controller topology

10. Thermal engineering

Heat is a reliability and battery-lifetime problem. Apple documents a 0-35 C ambient operating range for iPhone/iPad and warns that high temperatures can permanently reduce battery capacity; charging may slow or stop when the device is too warm. [S11] Other manufacturers publish their own limits, which should be checked per model.

Sources of heat

Charging losses, display use, CPU/GPU work, radios and tightly packed devices all add heat. A rack near a window or inside a closed cabinet can run materially hotter than the room. Measure at phone inlet locations, not only at the HVAC thermostat.

Continuous 100% charge is also undesirable for battery aging. Where the manufacturer provides optimized charging or managed battery features, evaluate them. A fleet can be scheduled to maintain adequate charge without forcing every battery to remain at maximum state of charge continuously.

- Ambient intake temperature.
- Rack exhaust temperature.
- Representative battery/device temperature where accessible.
- Fan operation and dust loading.
- Thermal throttling or charging-paused events.

Airflow

Use a predictable front-to-back or bottom-to-top flow. Recirculation is the enemy: if hot exhaust immediately returns to the intake side, adding more fans can increase noise without improving temperature. Baffles or spacing can be more effective than raw fan count.

Design alerts well below absolute device limits so operators have time to react. The correct warning threshold depends on the device model and sensor being monitored.

Engineering checklist

- Baseline temperatures at 25%, 50% and 100% rack occupancy.
- Repeat test with all devices charging from low battery.
- Define warning and critical thresholds.
- Test a failed-fan scenario.
- Keep rack away from direct solar load.

11. Fire/electrical safety and UPS

A phone rack combines many lithium-ion batteries, charging electronics and AC connections. Use commercially rated electrical components, inspect cords and connectors, and follow local electrical/fire rules. OSHA guidance for extension cords emphasizes proper construction, grounding where required, strain relief and inspection; local regulations may impose additional requirements. [S31]

Safety principles

Do not treat consumer power strips as permanent high-density distribution without verifying their ratings and local code suitability. Do not defeat grounding or overcurrent protection. Remove swollen, damaged or unusually hot batteries from service immediately according to manufacturer and local hazardous-waste guidance.

Provide a clearly reachable main disconnect for the rack and an incident procedure for smoke, odor, battery swelling or abnormal heat. Staff should know not to puncture or compress a damaged lithium-ion battery.

- Rated PDU / distribution.
- Cable inspection schedule.

- No blocked ventilation around chargers.
- Smoke detection appropriate to the site.
- Fire response plan aligned to local authority guidance.

UPS strategy

Prioritize graceful operation rather than attempting to run all charging for hours. A practical UPS can keep router, firewall, switch, APs, controller and database online long enough to preserve state and shut down cleanly. Phones already contain batteries and may not need to be on UPS-backed charging circuits unless continuity requirements demand it.

Calculate UPS VA/W load from measured equipment and required runtime. Validate actual runtime with a controlled test after installation.

12. Host computers and local controllers

The local controller is the bridge between the physical fleet and the backend. It may host ADB, local media caching, telemetry collectors, a message broker edge client, a web dashboard, or Apple provisioning utilities.

Sizing

CPU requirements are modest for orchestration but can rise sharply if the host transcodes video, performs local AI analysis or handles many concurrent media transfers. RAM should cover the OS, device tooling, database/cache agents and headroom. Use NVMe storage if the host is a media staging point; otherwise keep large media in object storage or a dedicated NAS.

For a 10-25 device pilot, one reliable workstation or mini-PC may be enough. At higher scale, separate the fleet controller from heavy video processing and from the primary database to reduce blast radius.

- At least two network interfaces if you want clean management/device separation.
- Automatic startup of required services after reboot.
- Disk encryption.
- Admin accounts protected by MFA.
- Regular image/configuration backup.

Operating system choice

Android ADB works across Windows, macOS and Linux. Apple Configurator is a macOS tool, while long-term Apple fleet control should be performed with a suitable MDM rather than requiring every device to remain tethered to one Mac.

Choose the host OS your operators can patch and support reliably. Consistency beats novelty.

13. Inventory, labeling, spares and lifecycle

A fleet becomes manageable when physical identity and database identity are never ambiguous. Build inventory before scale.

Minimum asset record

Each device should have an internal asset ID, manufacturer/model, serial/IMEI where appropriate, purchase source/date, warranty status, OS version, storage, battery-health note, rack/slot, assigned creator or pool, management-enrollment status and retirement date. Sensitive identifiers should be access-controlled.

Do not encode creator names or other sensitive information directly into externally visible device labels. Use opaque asset IDs and resolve them in the management system.

- Asset barcode/QR.
- Cable and charger-port mapping.
- Spare status: ready / repair / quarantine / retired.
- Chain-of-custody when handed to an employee.

Lifecycle

Use a standard state machine: Received -> Inspected -> Enrolled -> Available -> Assigned -> Maintenance -> Quarantined -> Retired -> Wiped/Disposed. Every transition should be timestamped and attributable.

Maintain spare capacity. A fleet that operates at 100% device utilization has no room for OS updates, battery replacement or incident quarantine. # SECTION 3 - iOS Engineering: Identity, Provisioning, Display, Input and Media

Feynman primer — an iPhone has separate doors for seeing, touching, managing and moving media

Four questions that sound like one are actually different: **Can the host see the screen? Can it send input? Can IT configure the phone? Can content become a real Photos asset?** A tool that solves one door does not automatically open the others. QuickTime/CoreMediaIO is about seeing. WDA or external HID is about input. Configurator/MDM is about administration. PhotoKit/import workflows are about the Photos database. Designing these planes separately prevents category errors.

Why iOS deserves its own section

The iOS problem is not simply “Apple is locked down.” It is a set of deliberate platform boundaries that affect different tasks differently. Device management is strong; arbitrary UI injection is intentionally constrained. File transfer exists; Photos-library ingestion is a separate operation. A Mac can be a powerful provisioning/test host; that does not automatically mean it can deliver scalable arbitrary touch input without a phone-side component or attached hardware.

This section gives each mechanism a separate name so the design does not accidentally assume that one capability implies another.

3.1 Identity and reset model

IDFV is useful but not permanent

[VERIFIED-FIRST-PARTY] Apple’s `identifierForVendor` is the same for apps from one vendor on one device and different for other vendors/devices. Apple documents that it changes after all apps from that vendor are deleted and later reinstalled. Therefore it is neither a global cross-vendor hardware serial nor a reliable forever-identifier.

DeviceCheck is persistent coarse state

[VERIFIED-FIRST-PARTY] DeviceCheck lets a developer store two bits and a timestamp associated with a device on Apple servers. Apple explicitly states that this state persists across app reinstall, device transfer between users, and Erase All Contents and Settings.

The design implication is subtle:

- DeviceCheck does **not** expose a universal hardware serial.
- The bits are developer-controlled state, not a descriptive fingerprint.
- The state is useful specifically because ordinary install-local identifiers can change.
- A reset therefore cannot be treated as a guaranteed clean slate from an app backend's perspective.

App Attest is installation-scoped

[VERIFIED-FIRST-PARTY] App Attest uses hardware-backed keys to prove app integrity, but the keys are unique per installation and do not survive reinstall, migration or restoration. Apple also states that attestations expose no hardware identifier.

[INFERENCE] DeviceCheck and App Attest should be mentally modeled as two different anti-abuse primitives: **persistent coarse memory** versus **current app-instance integrity**.

Used-device consequence

[INFERENCE] With any developer-controlled server-side state that survives ownership transfer, used hardware has unknowable historical state relative to a given app. That is an asset-risk consideration, not a reason to invent an “unban” procedure. For a legitimate fleet, record purchase source/IMEI/serial internally for inventory, validate hardware functionality, and treat unexplained platform-level device restriction through official support and asset-retirement processes.

3.2 Provisioning and supervision

Apple Configurator and device management solve **configuration**, not arbitrary third-party app control.

[VERIFIED-FIRST-PARTY] Apple Configurator Blueprints can apply apps, profiles and actions to multiple devices. Supervision unlocks additional restrictions/management features and is best established before activation or on a fully erased device. Unsupervising later erases the device.

This creates a day-zero choice for durable devices:

Choice	Benefit	Cost
Unsupervised	less intrusive setup; closer to consumer default	fewer management controls; more manual setup
Supervised	stronger configuration, restrictions, silent management options	must be planned before production state; reversing it erases device

[INFERENCE] For a five-device engineering pilot, supervision should be tested on **one** dedicated test phone before becoming the fleet default. The decision should be driven by which settings actually need automation, not by the assumption that “more MDM is always better.”

Wired accessory authorization in an unattended rack

[VERIFIED-FIRST-PARTY] Apple separates a physical cable connection from permission to use wired accessories. Current devices expose Wired Accessories policy in Privacy & Security; supervised iOS devices can also receive the `allowUSBRestrictedMode` restriction through device management.

For a rack, test these states separately:

State	What to verify
first connection	Trust/Allow prompts, pairing record creation

State	What to verify
unlocked reconnect	device enumerates and capture/management paths return
locked reconnect	required USB functions remain available under chosen policy
device restart	pairing and accessory authorization recover as expected
host restart	device mapping is stable; no human prompt blocks the pod
Lockdown Mode	document that it can override accessory policy

[POLICY] Do not disable USB Restricted Mode merely for convenience. If a chosen unattended workflow requires a less restrictive setting, document the reason in an ADR, scope it only to supervised organization-owned test devices, and prove the operational requirement first.

3.3 Display paths

Path A - network mirroring

[INTERNAL-MEASURED] Xiaowei mirroring worked when the phone and Windows host were on the same local network. The observed path was network-based, not cable-based. This is a strong fact for that tested product/configuration.

Consequences:

- local service discovery becomes part of availability;
- VLAN segmentation can break discovery;
- cellular-only devices cannot participate in a LAN-only display path;
- a shared WLAN is a control-plane dependency even if Internet egress is different.

Path B - Apple's personal iPhone Mirroring feature

[VERIFIED-FIRST-PARTY] Apple's iPhone Mirroring requires a compatible Mac/iPhone, same Apple Account with 2FA, Bluetooth and Wi-Fi, and a locked nearby iPhone. It is a personal-device feature, not a documented rack-scale automation API. Do not build the fleet around it.

Path C - tethered iPhone screen capture on macOS

[VERIFIED-FIRST-PARTY] Apple QuickTime Player can capture the screen of a cable-connected iPhone/iPad, and Apple publishes the CoreMediaIO switch `kCMI0HardwarePropertyAllowScreenCaptureDevices`. In plain English: **macOS has a real, public screen-capture device path for tethered iOS devices; this is not only a QuickTime trick.**

[MAINTAINED-IMPLEMENTATION] Maintained macOS capture software and public sample code enumerate these iOS screen-capture devices through CoreMediaIO/AVFoundation. This supports programmatic access to the mechanism.

[OPEN] No first-party source found in this research pass gives a supported concurrency ceiling such as 5, 10 or 20 iPhones per Mac. The ceiling can be set by several different things: USB topology, the iOS capture service, host-side decode/copy work, memory pressure, per-device daemon behavior, cable/hub quality, or a software limit. The farm must therefore **measure concurrency rather than infer it from port bandwidth.**

Feynman explanation — why this distinction matters

Imagine a building has an elevator. Proving the elevator exists does not prove it can move twenty people every minute for eight hours. The Mac capture question used to be “is there an elevator?” That part is now substantially answered. The engineering question is now “what is the safe throughput of our elevator under our load?”

A clean capture experiment records, for every opened device:

- stable device identifier and physical slot;
- negotiated resolution and frame rate;
- end-to-end preview latency;
- dropped or stalled frames;
- host CPU and memory pressure;
- USB disconnect/re-enumeration events;
- whether reconnect works without reboot;
- thermal behavior during a one-hour soak;
- behavior after host sleep/restart and device lock/unlock.

TEST-MAC-CAPTURE-001 remains a Gate-1 experiment, but its purpose is now **capacity and reliability validation**, not proving that the public mechanism exists.

Capture health must be based on fresh frames

[INDEPENDENT-ENGINEERING] WireView, a current macOS utility using the same built-in iOS capture path, documents several real AVFoundation/device-state failure modes: device presence without frames, stale/frozen frames after lock, trust changes that leave a session apparently running, and cases where capture recovery requires a device restart.

This is not an Apple guarantee and is not evidence of the frequency of each failure in our rack. It is high-value failure-mode evidence because it tells us what the adapter must be able to detect.

A capture worker should therefore implement a state model similar to:

```
ABSENT
-> ENUMERATED
-> STARTING
-> STREAMING_FRESH
-> STALE
-> RESTARTING_SESSION
-> RECONNECTING_DEVICE
-> REBOOT_REQUIRED
-> QUARANTINED
```

The invariant is simple: **no fresh frame, no healthy display path**. Process liveness and device enumeration are supporting signals only.

iOS 17+ developer-service transport note

[MAINTAINED-REPO] Traditional iOS USB tooling uses the lockdown/usbmux family for many services, but iOS 17 introduced CoreDevice/RemoteXPC transport changes for developer services. Maintained pymobiledevice3 documentation distinguishes iOS 17.0-17.3.1 from iOS 17.4+, where CoreDeviceProxy enables a simpler no-root userspace tunnel over USB for supported operations.

[INFERENCE] Treat host-tool compatibility as a matrix keyed by **iOS version + host OS + service**, not as one generic “USB works” boolean.

3.4 Input paths

External HID/control board

[VENDOR-CLAIM] Niazun and similar products claim to control iPhones with external hardware and vendor host software without jailbreak, developer signing or a custom app on the phone. Published product/API material describes pointer, swipe, keyboard, screenshot/vision and automation functions.

The correct engineering treatment is **adapter + containment + bench verification**:

```
control plane
|
v
vendor adapter service
|
v
vendor host API ----> control board ----> iPhone
```

Never let vendor API shapes leak into business logic. A future board should require one adapter replacement, not a backend rewrite.

WebDriverAgent / XCUITest / Appium

[VERIFIED-FIRST-PARTY/MAINTAINED-DOC] Appium's XCUITest driver requires WDA installed/provisioned on real devices, plus device trust, Developer Mode on current iOS and UI Automation. This is a mature QA path and gives semantic element-level automation.

It conflicts with a strict policy that nothing may be installed for control. That is a **policy trade**, not a technical impossibility.

Indirect input semantics

[VERIFIED-FIRST-PARTY] Apple's UIApplicationSupportsIndirectInputEvents documentation says that when indirect input support is enabled, pointing-device clicks produce `UITouch.TouchType.indirectPointer`; when disabled they are delivered as direct touches. If the key is absent, iOS 17+ defaults to indirect support and pre-iOS-17 systems default to the older direct-touch compatibility behavior.

This corrects two bad simplifications:

1. "Hardware pointer clicks always look exactly like a finger" - false at the UIKit API level on modern defaults.
2. "A platform definitely detects hardware control" - unsupported by the evidence. The OS exposes a distinguishable event; platform use is unknown.

[INFERENCE] Do not create production version-pinning around this. Run a controlled instrumented app on the exact device/OS/control-board combination to characterize event types for engineering knowledge, and treat third-party platform behavior as unknown.

3.5 The Lightning/USB coexistence question

[OPEN] The handover describes a practical concern: an accessory that requires the iPhone to act as USB host can conflict with the phone's simultaneous role as a USB device to a computer. Whether the inbound control board actually forces that topology is not established from the product claims.

This is a simple bench question with enormous architectural impact. If display/data to the host and external input can coexist through the intended board/hub arrangement, a wired pod becomes much simpler. If not, the system may need a network display channel or a different control route.

3.6 Media delivery

A byte copied to an iPhone is not automatically a Photos asset. The production requirement is not “file reached device”; it is “authorized asset is visible in the intended application’s normal media picker with the expected orientation/date/type and can be removed/reconciled later.”

[VERIFIED-FIRST-PARTY] Apple PhotoKit provides PHAssetCreationRequest inside PHPhotoLibrary.performChanges specifically to create image/video assets from underlying data resources and add them to the Photos library. Therefore the architectural distinction is concrete:

```
file transport -> bytes available somewhere
PhotoKit import -> Photos database owns a real asset
```

If an on-device helper app is allowed, PhotoKit is a clean first-party ingestion mechanism. If the no-helper policy remains strict, the team must validate another path that produces the same final Photos-library state; raw AFC/file copy alone is not an acceptance criterion.

A robust media state machine is:

```
APPROVED_IN_CLOUD
-> QUEUED_FOR_DEVICE
-> TRANSFER_STARTED
-> BYTES_PRESENT
-> PHOTO_ASSET_CONFIRMED
-> CONSUMED/POSTED (human or approved path)
-> RETENTION_ACTION
```

[VENDOR-CLAIM] If a control board exposes album/media endpoints, validate the *final Photos-library state* rather than trusting a 200 OK response.

[POLICY] Media provenance, creator authorization and content rights stay attached to the asset. The system may normalize filenames/transcoding/orientation for compatibility; it should not implement metadata fabrication intended to misrepresent origin.

3.7 Mac mini starting point

[VERIFIED-FIRST-PARTY] Apple’s current M4 Mac mini provides five USB-C-class physical ports in total: two front USB-C ports supporting USB 3 up to 10 Gb/s and three rear Thunderbolt 4 ports on the M4 model, plus Ethernet and HDMI. The M4 includes hardware video encode/decode engines and starts with 16 GB unified memory; higher-memory configurations are available.

[INFERENCE] **Pilot purchase recommendation: M4 Mac mini, 24 GB unified memory, 512 GB SSD.** Gigabit Ethernet is sufficient for the ordinary pilot network load. **If this Mac is expected to become a remotely operated rack host and budget permits, evaluate the 10Gb Ethernet configuration because Apple Lights Out Management supports the 2024 Mac mini with the 10GbE option.** The reason to consider 10GbE here is recovery capability, not a claim that phone capture requires 10Gb/s. Twenty-four gigabytes gives more room for several capture sessions, the coordinator/node agent, remote-access software, Configurator/Xcode tools, logs and browser/operator processes while the team measures the real bottleneck.

Do **not** start by buying M4 Pro solely because the target is twenty phones. Thunderbolt bandwidth is only one possible limiter, and iPhone capture may bottleneck elsewhere first. Escalate to M4 Pro or additional Mac minis only when TEST-CAPACITY-001 identifies host CPU/memory/I/O as the limiting resource.

Feynman explanation — why “more expensive Mac” is not automatically “more phones”

Think of a restaurant. A wider front door does not increase meals per hour if the kitchen is the bottleneck. Thunderbolt 5 is the wider door; the iOS capture service, USB hubs, per-device processes or decode pipeline may be the kitchen. Measure the kitchen before paying for a bigger door.

Why the 10GbE option can still matter

[VERIFIED-FIRST-PARTY] Apple Lights Out Management supports Mac mini (2024) models with the 10Gb Ethernet card after enrollment in device management. The managed controller and target Mac must be on the same local Ethernet subnet and the workflow has additional MDM/network prerequisites.

[INFERENCE] If the farm site is difficult to reach physically, buying the 10GbE option can preserve a future remote-start/shutdown/restart path. Treat this as a **resilience feature**. It does not replace UPS design, SSH, remote desktop or a physical break-glass process.

3.8 iOS decision table

Need	Preferred first test	Why	Blocking unknown
Bulk setup	Configurator Blueprint	first-party, repeatable	exact settings coverage
Screen	Mac tethered CoreMediaIO/ AVFoundation capture	removes LAN dependency if stable	concurrency/reliability ceiling at 5/10/20 is not verified
Input without phone-side control app	inbound HID board	aligns with current policy	coexistence, event behavior, stability
Rich semantic automation	WDA/Appium	mature QA tooling	conflicts with no-install policy
Media import	board/Shortcuts/helper workflow test	must create real Photos asset	exact board behavior
Remote human ops	custom/GADS-like control interface	avoids video-of-video	WDA dependency for GADS iOS

Fleet management should make devices boring: repeatable enrollment, predictable settings, known application versions, controlled access and fast recovery. Apple and Android require different approaches.

24. Apple fleet architecture

Apple’s enterprise deployment model centers on organization ownership, Automated Device Enrollment where available, supervision and Mobile Device Management (MDM). Apple Platform Deployment documents the lifecycle from planning and enrollment through configuration and ongoing management. [S5]

Figure 24-1. Simplified Apple MDM control flow.

Core components

Apple Business Manager (or the appropriate Apple organizational service) establishes organization ownership and can assign devices to an MDM service. Supervision provides additional management controls for organization-owned devices. The MDM service sends configuration profiles, app assignments and commands while devices communicate with Apple push infrastructure and the MDM service.

Use a commercial or internally operated MDM that explicitly supports the commands, restrictions and app deployment you require. The phone rack itself should not depend on one operator manually configuring every iPhone.

- Organization account and administrative roles.
- Device enrollment/assignment.
- Supervision.
- MDM server/service.
- APNs connectivity.
- Managed app distribution.

Design principle

MDM is the administrative plane. It is not a generic remote-control framework for simulating user gestures in third-party applications. Keep publishing workflows separate.

25. Supervision, enrollment and MDM

Supervision signals that an iPhone/iPad is organization-controlled and unlocks additional management capabilities. Automated Device Enrollment can supervise eligible organization-owned devices during setup; Apple Configurator can be used in certain manual preparation workflows. [S5]

Enrollment flow

At acquisition, record the serial and organization ownership. Assign the device to the intended MDM, boot/reset it, complete Automated Device Enrollment, verify supervision, apply baseline profiles, install managed applications, test connectivity, then mark the device Ready in inventory.

MDM commands typically use push notification to tell the device that a command is available; the device then contacts the management service over HTTPS, executes supported actions and reports results. [S6][S7]

- Wi-Fi/certificate profiles.
- Passcode and security policy.
- Managed apps.
- Restrictions appropriate to role.
- Lost mode/lock/wipe capability where supported.
- Inventory and compliance reporting.

Recovery

If a supervised device is lost or compromised, use the MDM's supported lock/wipe workflow and revoke relevant credentials. If it is reassigned, wipe and re-enroll rather than merely signing out of one social application.

2026-27 Apple management direction - design for richer status, do not require it yet

[VERIFIED-FIRST-PARTY / PRE-RELEASE-PLATFORM] Apple's WWDC26 device-management material describes additional declarative status in iOS/iPadOS 27, including device system health, enrollment/return-to-service state and enhanced log-collection workflows.

[INFERENCE] The coordinator schema should be able to store first-party health observations when supported, but the pilot must remain functional on older iPhones that cannot provide those new status items. Treat OS-version support as capability negotiation, not a reason to discard otherwise serviceable hardware.

26. Android Enterprise architecture

Android Enterprise provides management modes for personally owned and company-owned devices. For a phone farm, company-owned fully managed devices are normally the cleanest model because the organization can manage the complete work-only device. Dedicated devices are a subset that can be locked to a small application set. [S1]

Figure 26-1. Android Enterprise plus authorized ADB engineering.

Android Management API - capability is real, eligibility is constrained

[VERIFIED-FIRST-PARTY] Android Management API uses Android Device Policy to receive and enforce enterprise policy, with enterprise resources, enrollment tokens, policies and device state.

[VERIFIED-FIRST-PARTY] **Google's current permissible-use policy is a separate constraint.** It restricts the service to commercial EMM developers, Device Trust solution providers and OEMs, and lists solutions used exclusively for first-party in-house applications among disallowed uses.

This means two questions must be kept separate:

1. **Can the API technically manage a fully managed/dedicated phone?** Yes, within its supported model.
2. **Can XYZ use Android Management API as an internal-only fleet backend under Google's current terms?** Do not assume so.

[INFERENCE] For an internal fleet, shortlist a commercial Android Enterprise EMM, a supported DPC/dedicated-device design, or obtain explicit clarification/approval from Google before making Android Management API a production dependency.

The underlying Android Enterprise capabilities still matter: fully managed company-owned devices, dedicated-device/kiosk patterns, application policy, restrictions, network settings, system-update policy and compliance state remain useful design concepts regardless of which compliant EMM implementation supplies them.

Dedicated vs fully managed

Dedicated mode is suitable when the phone should expose only a controlled application set. If operators need several social apps, browser, camera, authenticator and internal tools, a normal fully managed configuration may be more practical than an overly restrictive kiosk.

27. Android provisioning and policy

Google documents QR, zero-touch, NFC and other provisioning methods. For company-owned work-only devices, full management allows the enterprise to manage all applications and enforce broad policy. [S1]

Pilot provisioning

For a small pilot, QR enrollment is straightforward: factory-reset the corporate device, enter the Android provisioning flow, scan the organization-generated enrollment QR, allow Android Device Policy to complete setup, then verify policy/application state. At scale, eligible zero-touch procurement reduces hands-on setup.

Enrollment tokens should be short-lived where practical and protected as administrative secrets because possession can allow a device to enroll into the enterprise configuration.

- Baseline Wi-Fi or setup network.
- Enrollment token handling.
- Policy assignment.
- Required apps.
- VPN client if required.
- Compliance validation.

Policy rings

Use Stable, Canary and Quarantine policies. Canary receives new OS/app/policy changes first. Stable is production. Quarantine removes unnecessary access while preserving enough management connectivity for investigation.

28. ADB for authorized engineering

Android Debug Bridge (ADB) is Google's command-line bridge between a development workstation and Android devices. It can communicate over USB and, on modern Android, paired wireless debugging. [S3] It is appropriate for provisioning support, logs, diagnostics and controlled testing on devices you own or are authorized to administer.

Architecture

ADB consists of a client, local server and device daemon. A workstation can enumerate authorized devices, open a shell, collect logs and transfer files. USB is usually the most predictable for a rack; wireless debugging reduces cables but depends on trusted Wi-Fi and pairing state.

Use a dedicated engineering account and workstation. Disable debugging on production devices if it is not needed, or restrict the workstation/network that can reach it.

- Inventory verification.
- Log collection.
- File push/pull for internal test assets.
- Package/version diagnostics.
- Network diagnostics.

Boundary

ADB can technically generate input events and interact with applications. This manual does not provide a workflow for using ADB to automate third-party social UIs where automation is prohibited or to make automated behavior appear human. Use supported APIs or human operation for those actions.

29. App distribution and configuration

A managed fleet needs predictable application versions. Install apps through MDM/managed app channels where available, define which apps are required, and avoid ad-hoc APK or IPA distribution unless you own the software and have an authorized enterprise development path.

Application catalog

Maintain a catalog with app name, package/bundle ID, platform, source, minimum version, required permissions, owner, data classification and update ring. Social apps, password manager, authenticator, VPN client, internal task app and device-health agent should each have an owner.

Permissions should be the minimum needed. For example, a publishing app may need photo-library access but not contacts. Review permissions after major app updates.

- Required vs optional.
- Version and update policy.
- Permissions.
- Data stored locally.
- SSO support.
- Uninstall/retirement behavior.

Internal companion app

Consider a small internal “operator companion” app that shows device ID, assigned creator/account, task queue, approved asset links, check-in status and incident button. It should not store social-account passwords and should not covertly drive other applications.

30. OS updates and maintenance rings

Ignoring OS updates creates security debt; updating every device immediately creates operational risk. Use rings.

Rings

Canary devices receive updates first and run representative workloads. After a defined observation window, a larger pilot ring updates, then production. Critical security fixes may justify accelerated rollout after compatibility testing.

MDM or Android Enterprise update controls can help schedule or defer updates within platform-supported limits. Do not leave devices indefinitely frozen on unsupported releases.

- Canary: 1-5% of fleet.
- Pilot: 10-20%.
- Production: remaining devices.
- Emergency: accelerated security rollout with explicit approval.

Maintenance window

Drain publishing tasks from the device, confirm battery/power, install update, reboot, verify management heartbeat, verify required apps and network, then return to Ready. Record the new build version automatically where possible.

31. Health telemetry and remote actions

A fleet dashboard should answer “what is healthy right now?” without requiring someone to walk down the rack.

Health model

At minimum track last heartbeat, OS/app version, storage free, battery/charging state, management compliance, network profile, last task, last error and current assignment. Temperature data availability varies by OS/device; use rack sensors even when device-level telemetry is limited.

Status should be derived, not subjective: Ready, Busy, Maintenance, Degraded, Quarantined, Offline.

- Heartbeat age.
- Charging state.
- Free storage.
- Wi-Fi/VPN state.
- Required app versions.
- MDM/Enterprise compliance.
- Task error rate.

Remote actions

Administrative actions can include lock, wipe, restart where supported, reapply policy, request inventory, rotate internal app tokens, or move a device to quarantine. High-impact actions should require stronger authorization and be written to the audit log. # SECTION 4 - Android Engineering: ADB, Enterprise Management and Integrity

Feynman primer — Android exposes a larger engineering service hatch

Android is generally easier for authorized device-lab work because ADB is a documented bridge for device communication and scrcpy can build on that bridge for display/control. Easier does not mean “unrestricted” or “identity-free.” Management policy, Play Integrity and vendor-specific behavior still matter. Treat Android as a different toolchain, not as iOS with different buttons.

4.1 Why Android remains operationally attractive

Android exposes a conventional engineering toolchain for organization-owned devices: ADB for development/debug control, Android Enterprise for managed-device policy, simpler media-file workflows and broad hardware choice. For a device lab, this usually lowers control cost and removes per-phone proprietary control hardware.

The mistake is turning that convenience into the claim “Android always resets cleanly.” Modern Android anti-abuse design no longer supports that certainty.

4.2 Device identity: separate local IDs from backend memory

Modern Android restricts access to stable hardware identifiers for ordinary apps. That weakens old folklore about any app trivially reading IMEI/MAC/serial.

[VERIFIED-FIRST-PARTY] In 2026, Google’s Play Integrity **device recall** is available in beta. A participating developer can store custom per-device state on Google’s servers and retrieve it after reinstall or device reset.

[INFERENCE] The correct conceptual model matches iOS:

local identifiers	may reset / be scoped / be restricted
server-side device state	can persist by platform-supported mechanism
app integrity verdict	evaluates current software/device trust

Therefore a reset is an administrative lifecycle operation, not an identity guarantee.

4.3 ADB and media workflows

ADB is valuable for owned test devices because it gives explicit device addressing, file transfer, shell commands and observability. It also makes media staging materially easier than iOS. The handbook uses ADB for legitimate engineering/QA tasks, not for evading application controls.

4.4 Android Enterprise

Use Android Enterprise/managed-device policy for configuration, app distribution, Wi-Fi/VPN policy, update rings and remote actions. Keep **management** distinct from **third-party UI control** just as on Apple: device policy is a fleet-admin function, not a substitute for an application’s public API.

4.5 Integrity posture

For social/consumer apps that rely on platform integrity, use stock, supported, non-rooted hardware. Root/custom-ROM/emulator techniques create support and integrity incompatibilities even in ordinary QA and should not be the production baseline.

4.6 iOS versus Android - decision lens

Dimension	iOS	Android
Consumer hardware consistency	high	varies by vendor/model
Organization management	strong	strong
Debug/control tool openness	constrained	ADB is powerful
UI automation on real devices	WDA/XCUITest signing path or attached hardware	broader ADB/testing options
Media staging	Photos ingestion is special	filesystem/media-store path simpler
Reset-persistent anti-abuse mechanism exists	DeviceCheck	Play Integrity device recall beta
Proprietary per-device control hardware required for strict no-install policy	possibly	generally not

[INFERENCE] For a durable, authorized multi-platform lab, Android should remain a genuine alternative rather than a cheaper imitation of the iOS plan. The correct choice depends on required target apps, device support horizon and operations cost, not on a myth that one OS is “unfingerprintable.”

SECTION 5 - Networking: LAN, Wi-Fi, Cellular, NAT, DNS, Proxy and VPN

Network zones: local control and Internet egress are different questions

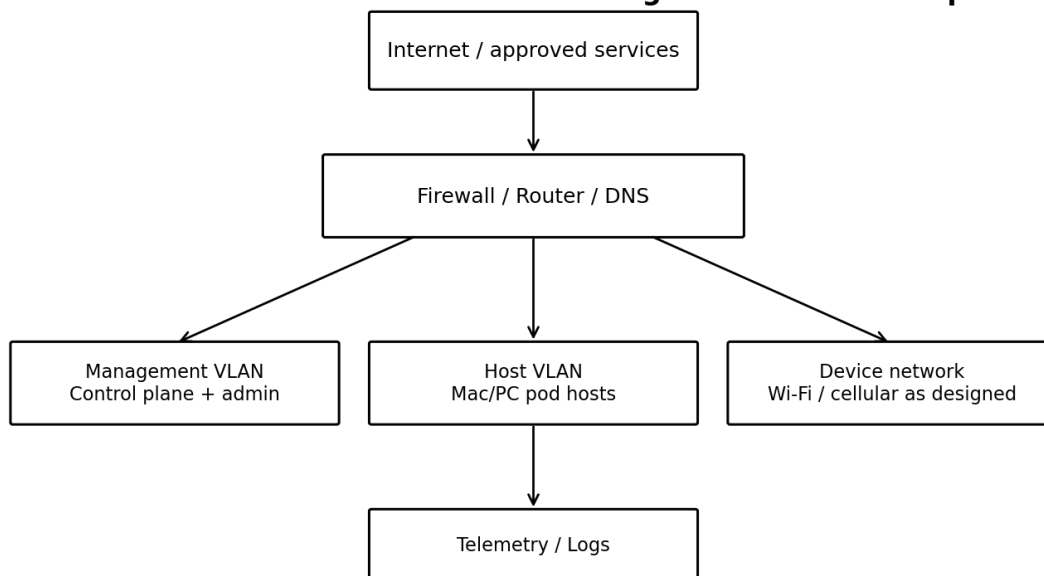


Figure 4 — Reference network zones; exact subnets are implementation choices.

Feynman primer — the network is a road map, not one thing called “the IP”

A phone can be on Wi-Fi for local control while using a different path for Internet traffic; or the two paths can be coupled by the software. VLAN, DNS, NAT, CGNAT, VPN and proxy are different layers. Think of them as roads, address books, gates and tunnels. When troubleshooting, identify the exact hop that failed instead of saying “the network is down.”

Networking is where “phone farm” discussions often become confused. Public IP address, local Wi-Fi address, DNS resolver, proxy, VPN tunnel, SIM carrier and device identity are different concepts. This part separates them and shows how to design reliable, auditable egress without using the network to misrepresent identity or defeat enforcement.

14. Network reference architecture

The recommended baseline is a business firewall/router, managed switching, one or more centrally managed access points, separate VLANs/SSIDs for fleet devices and management, and explicit DNS/egress policy.

Figure 14-1. Segmented network reference.

Zones

The management zone contains administrator workstations, MDM connectors, ADB hosts and monitoring. The device zone contains phones. The data zone contains database, media store and backups. A quarantine/guest zone can isolate new or suspect devices. ACLs should permit only the flows required between zones.

Segmentation reduces the impact of a compromised phone or stolen employee credential. It also makes troubleshooting easier because you know which traffic class a packet belongs to.

- Management VLAN.
- Device VLAN(s).
- Data/storage VLAN.
- Guest/quarantine VLAN.
- Optional VoIP/office user VLAN separate from the farm.

Internet edge

The firewall should log high-level connection and security events without collecting more personal data than necessary. Where a business VPN gateway or proxy is used, document which devices/apps use it and why.

Prefer deterministic configuration to ad-hoc per-phone settings. Configuration drift is one of the main reasons ten devices behave differently despite looking identical.

15. Wi-Fi radio and capacity planning

Phone density is a radio-capacity problem, not merely a “strong signal” problem. High-density wireless design requires enough airtime, channel planning and AP placement for the concurrent workload. Vendor high-density design guides emphasize planning client density and bandwidth rather than relying on maximum advertised AP client counts.

2.4 vs 5/6 GHz

2.4 GHz travels farther but has less clean spectrum and more interference. 5 GHz provides more channels and is usually preferable for dense phone racks when devices and region support it. 6 GHz can add capacity on compatible devices and APs but has different propagation and regulatory constraints.

Do not put a rack of many phones directly against one AP and assume proximity guarantees capacity. Very strong signals can coexist with contention. Use multiple APs only when channel planning supports it; simply adding APs on overlapping channels can make performance worse.

- Measure channel utilization and retry rates.
- Use wired Ethernet backhaul for APs.
- Disable legacy rates/features only after confirming device compatibility.
- Set reasonable transmit power rather than maximum everywhere.
- Perform throughput and latency tests from representative rack positions.

SSID design

Keep SSID count low because management overhead consumes airtime. Separate networks by security/policy need, not by creating one SSID per creator. VLAN assignment can provide segmentation behind a smaller number of SSIDs.

If using ADB wireless debugging, the workstation and device must meet the applicable pairing/network requirements. Android 11+ supports wireless debugging with pairing; use it on trusted networks only. [S3]

Metric	What it indicates	Typical response
Channel utilization	Airtime pressure	Reduce load / add properly planned capacity
Retry rate	Interference / weak link	RF survey, power/channel tuning
RSSI/SNR	Link quality	AP placement / obstacles
Latency/jitter	Congestion or WAN issue	Separate LAN vs WAN tests
Roaming events	AP handoff behavior	Tune only if devices actually move

16. Ethernet, switches and VLANs

All infrastructure that can be wired should generally be wired. APs, controller hosts, storage and servers should use Ethernet so wireless airtime is reserved for phones.

Switching

A managed switch allows VLAN trunks, per-port statistics, PoE power for APs, and port isolation where useful. Maintain a port map so a failed AP or controller can be found quickly.

For larger installations, redundant uplinks and a second switch may be justified, but complexity should follow actual uptime requirements.

- Label patch cables at both ends.
- Use separate management access for switches/APs.
- Back up switch configuration.
- Disable unused physical ports or place them in a restricted VLAN.

VLAN/ACL design

A VLAN is a Layer-2 segmentation mechanism; it is not security by itself. Inter-VLAN routing rules on the firewall or Layer-3 switch enforce which systems can communicate. A device VLAN typically needs DNS, NTP, internet access and access to explicitly approved internal services, not unrestricted access to the management network.

Block unsolicited device-to-device traffic unless the workflow requires it. This limits lateral movement if one handset is compromised.

17. IPv4, IPv6, NAT and CGNAT

Understanding address translation prevents mistaken assumptions about “one IP per phone.” A phone normally receives a private local IP address from DHCP. The router may translate many private IPv4 addresses into one or several public IPv4 addresses using NAT. A mobile carrier may perform another layer of carrier-grade NAT (CGNAT), meaning many subscribers share public IPv4 space.

IPv4/NAT

Private IPv4 addresses such as RFC1918 ranges are not globally routable. The edge router keeps connection state so return traffic is mapped back to the correct phone. NAT changes addressing; it does not inherently anonymize application identity.

Port/address translation also means a platform may see the same public egress IP for many legitimate devices behind a business connection. Conversely, the same phone may appear behind different public addresses as networks change.

CGNAT and mobile networks

CGNAT performs NAT inside the carrier network at large scale. This is common where public IPv4 addresses are scarce. IPv6 can provide globally routable addresses without the same NAT requirement, though firewalls still control inbound/outbound connectivity. [S18]

Do not design identity assumptions around IPv4 alone. Modern applications can use IPv4 and IPv6 depending on network and DNS responses.

- Record whether each segment has IPv6 enabled.
- Test application behavior over IPv4 and IPv6.
- When troubleshooting “wrong IP,” identify the local IP, router WAN IP and externally observed IP separately.

18. DNS and resolver design

DNS converts hostnames into addresses and is part of both reliability and privacy. Traditional DNS can be visible to the local network; DNS over HTTPS (DoH) and DNS over TLS (DoT) encrypt queries between client and resolver. [S17]

Resolver choices

A business can use the firewall as a forwarding resolver, an internal resolver, or a managed external resolver. The goal is consistent policy, fast failure diagnosis and appropriate logging. Do not mix several ad-hoc per-phone resolver settings unless there is a documented requirement.

If a VPN is used, decide whether DNS should also traverse the tunnel. Split DNS may be necessary for internal names. DNS leaks are not merely a “privacy hack” topic; mismatched routing can break corporate resources and make incident analysis confusing.

- Monitor resolver latency and failure rate.
- Use DNSSEC-validating resolvers where suitable.
- Keep internal hostnames out of public DNS.
- Minimize query logging retention to operational/security needs.

Failure diagnosis

Test in layers: can the phone reach the default gateway, can it reach a known IP, can it resolve a hostname, can it establish TLS to the target, and can the application authenticate? This avoids blaming “the proxy” for a DNS or certificate problem.

19. Forward proxies and proxy protocols

A forward proxy is an intermediary that makes outbound requests on behalf of clients. For HTTP(S), a proxy may forward requests directly or use the CONNECT method to create a TCP tunnel for TLS traffic. [S14] A proxy is different from a VPN: a proxy is usually application/protocol-aware, while a VPN normally routes IP packets through a tunnel.

Figure 19-1. Proxy, VPN, NAT and DNS are separate layers.

Why businesses use forward proxies

Legitimate uses include outbound policy enforcement, malware filtering, controlled egress, audit logging, caching in some environments, and allowing specific internal applications to reach the internet through a known gateway.

On mobile devices, proxy support varies by application and OS. A Wi-Fi proxy configured at the network level may not affect every protocol. A VPN-based egress design is often more comprehensive when the requirement is “all traffic from these apps/devices follows this path.”

- HTTP proxy: understands HTTP semantics.
- CONNECT: creates a tunnel commonly used for HTTPS through an HTTP proxy.
- SOCKS-style proxy: relays TCP and sometimes UDP without understanding HTTP content.
- Transparent/intercepting proxy: network redirects traffic; introduces certificate/privacy complexity for TLS inspection.

Proxy provenance and risk

Marketing terms such as “datacenter,” “residential” and “mobile” refer to how the public egress address is sourced. They do not create a compliance exemption. Third-party proxy providers introduce security, privacy, availability and contractual risks because they may observe metadata, alter routes or lose access to address pools.

This manual does not recommend proxy rotation patterns or provider selection for avoiding platform detection. If an agency needs a proxy, choose it for an articulated business purpose such as security, regional testing with authorization, or controlled corporate egress, and perform legal/provider due diligence.

Engineering checklist

- Document why a proxy is needed.
- Confirm which apps/protocols actually use it.
- Test TLS/certificate behavior.
- Measure latency and failure rate.
- Define what the provider can log and retain.
- Create a rollback path to direct egress.

20. VPNs, tunnels and per-app routing

A VPN creates a logical network tunnel between device and gateway. On Android, VpnService creates a virtual TUN interface, allowing a VPN application to read outgoing IP packets and return incoming packets through the interface. Android supports always-on VPN and per-app allow/disallow lists. [S4]

Use cases

VPNs are appropriate for securing traffic over untrusted networks, reaching private corporate services, isolating a fleet behind managed gateways, and applying consistent egress/security policy. WireGuard is one modern protocol designed as a secure network tunnel; enterprise deployments may also use IPsec/IKEv2 or vendor clients. [S19]

Per-app routing can keep only managed applications inside a corporate tunnel while leaving OS update or other traffic direct, or it can do the reverse depending on policy. Test carefully because platform apps may depend on push services, CDNs and media endpoints outside a simple hostname list.

- Always-on mode for high-assurance corporate access.
- Kill/block mode if non-VPN traffic must not bypass policy.
- Per-app lists to scope tunnel use.
- Gateway redundancy for production.
- Key rotation and device revocation.

What a VPN does not do

A VPN changes the network path and may change the public IP address observed by a service. It does not change the account's history, app telemetry, device security state, content, user behavior or the platform's rules. Do not treat it as a method for falsifying identity or location.

21. Cellular, SIM and eSIM concepts

Cellular connectivity can provide redundancy or legitimate mobile testing, but it adds carrier contracts, SIM inventory, signal variation and CGNAT behavior. A mobile plan should be procured and used in accordance with carrier terms and local identity/registration laws.

Physical SIM vs eSIM

Physical SIMs are easy to move between compatible devices but create a physical inventory problem. eSIM can simplify provisioning on supported carriers/devices but may require carrier portals or activation workflows. In either case, map the subscription to the device and cost center in inventory.

Do not confuse the subscriber identity on a SIM with the identity of a social account. Keep telecom records and platform credentials in separate systems.

- Carrier name/plan.
- SIM/eSIM identifier stored with restricted access.
- Assigned device.
- Monthly data cap and roaming rules.
- APN/VPN requirements.
- Suspension/termination procedure.

Redundancy

For business continuity, a cellular router can provide WAN failover for the site. This is often easier to manage than giving every phone its own cellular plan. Individual phone cellular service should exist only where the workflow needs it.

22. Egress observability and troubleshooting

Operators need to answer: which path did this device use, which DNS resolver answered, what public IP was observed, did the VPN/proxy work, how much latency was introduced, and where did the request fail? Build observability for those questions without recording sensitive content unnecessarily.

Telemetry

Collect device ID, network profile, SSID, gateway reachability, resolver status, tunnel status, public egress test result, latency, packet loss and timestamp. Avoid storing credentials, message contents or unnecessary browsing history in network logs.

Use synthetic probes to your own endpoints for health checks. Do not continuously probe third-party platforms in a way that creates abusive load.

- DHCP lease and VLAN.
- VPN/proxy state.
- DNS response time.
- WAN packet loss.
- Gateway CPU/connection utilization.
- AP client count/channel utilization.

Troubleshooting ladder

Always isolate layers: device hardware -> Wi-Fi association -> DHCP -> gateway -> DNS -> tunnel/proxy -> TLS -> application authentication -> platform/API response. A documented ladder reduces random configuration changes that create new faults.

23. Network failure modes

Most fleet networking incidents are ordinary infrastructure failures: overloaded Wi-Fi, dead cables, DNS outages, expired tunnel credentials, saturated WAN links or misapplied ACLs.

Common scenarios

If all devices on one rack fail simultaneously, look for shared AP, switch, VLAN, DHCP or power dependencies. If only one application fails but other internet access works, look above the IP layer. If a subset of devices intermittently disconnects, inspect RF utilization and client-specific power-saving behavior.

When changing proxy/VPN configuration, deploy to a small ring first. A bad egress policy can strand the entire fleet from MDM or management services.

Resilience patterns

Dual WAN, redundant DNS resolvers, spare APs, configuration backups and staged changes provide more benefit than exotic networking. Keep a direct, secure management path so a failed content egress tunnel does not remove your ability to manage the device.

- Configuration versioning.
- Change tickets with rollback.
- Canary device/rack.
- Out-of-band admin path where justified.
- Post-incident review.

Failure	Blast radius clue	First containment
AP failure	One RF cell/rack	Move clients / replace AP

Failure	Blast radius clue	First containment
DHCP failure	New joins fail	Restore DHCP / lease service
DNS failure	IP works, names fail	Fail over resolver
VPN gateway failure	Tunnel users only	Fail over or controlled bypass
WAN outage	Whole site	Secondary ISP/cellular failover

SECTION 6 - Client, Account and Credential Separation

Feynman primer — separation is fireproofing

If every client shares one room, one fire burns the whole building. Separation means putting fire doors at several layers: business container, credentials, staff roles, browser/session, device assignment, network, billing and vendors. A browser profile is only one door. The strongest links often exist in the platform's own database because the platform already knows who administers what.

6.1 The security objective

The goal is not to make authorized clients look unrelated to a platform. The goal is to prevent a mistake, breach, restriction, billing dispute, vendor failure or staff departure affecting clients who were not involved.

The internal account-separation reference ranks the most damaging agency failure modes as: shared business containers, credential spill, session bleed, shared billing, offboarding gaps, enforcement contagion and vendor concentration. Browser fingerprinting is real but lower in practical blast-radius priority.

6.2 Six independent separation layers

6. PLATFORM GRAPH	ownership, partner/admin relationships, billing
5. IDENTITY/CREDENTIAL	password, 2FA, recovery, business identity
4. BROWSER PROFILE	cookies, storage, extension state
3. DEVICE/OS	Windows/macOS user, filesystem, clipboard, backups
2. NETWORK	IP, DNS, NAT, proxy/VPN, network operator
1. BEHAVIOR/CONTENT	timing, assets, link patterns, writing style

A boundary at one layer does not cancel a relationship at another.

[INFERENCE] This is the most useful conceptual tool in the new research because it prevents “proxy thinking” from consuming the whole security design. If the same employee is explicitly an admin in two client Business Centers, a different IP does not erase the platform's own admin relationship. If two clients share one billing instrument, separate phones do not isolate payment risk.

6.3 Platform graph and delegated access

[VERIFIED-FIRST-PARTY] TikTok Business Center provides account/asset-level permissions for members and partners. This allows an agency to request scoped access rather than take custody of the client's primary credential.

[VERIFIED-FIRST-PARTY] X Delegate lets an owner grant admin/contributor access without sharing the owner's password. X explicitly notes that changing the account password does **not** remove delegates; revocation is a separate action.

For every platform, first ask: **does a first-party delegated/partner mechanism exist for the exact asset/action?** If yes, use it by default. If no, document why credential custody is unavoidable.

6.4 Client-owned containers

[POLICY] Where the platform supports business containers, the client owns the container and core asset. The agency receives scoped access. This means the client can revoke the agency without losing its asset, and an agency-level restriction does not automatically take ownership of the client's resources.

This is the difference between *operating an asset* and *owning the asset on the client's behalf*.

6.5 Credentials, 2FA and recovery

Credential custody should be minimized:

- unique vault-generated password where password access is unavoidable;
- no reuse across clients;
- client-owned recovery route;
- organization-managed 2FA with auditable access rather than one employee's personal phone;
- staff never copy credentials into notes/chat;
- rotation on offboarding/security event;
- credential references in the control-plane database, never plaintext secrets.

[INFERENCE] A useful invariant is: **the database may know which secret object authorizes an action; it should not be able to print that secret.** Store a vault reference and authorization scope, not the password/token itself.

6.6 Browser profile is convenience isolation, not workstation security

[VERIFIED-FIRST-PARTY] Google documents that Chrome profiles keep bookmarks, history, passwords and settings separate. Google also warns that someone with access to the device can switch profiles. Chrome sign-in/sync can save passwords/history/tabs/settings to the associated Google Account.

Therefore:

- one browser profile per client is useful for session hygiene;
- separate OS users are stronger for local separation;
- sensitive clients may justify separate workstations/VMs;
- no personal Google Account sync in client profiles;
- extension allowlists matter because a third-party extension becomes another data processor;
- downloads/clipboard/screenshots/shared cloud folders must be included in the separation model.

6.7 Billing and vendor concentration

[POLICY] A billing dispute for Client A should be structurally unable to disable Client B. Use client-owned billing where possible; if the agency fronts spend, isolate instruments/accounting by client.

The same principle applies to schedulers, analytics, link shorteners and media tools. One convenience SaaS account that can access every client is a single breach domain. Document every vendor's client scope and revocation path.

6.8 Staff lifecycle

Every person gets an individual identity and least-privilege role. No generic shared "VA login." The joiner/mover/leaver process should be executable from an inventory:

```
person -> roles -> clients -> platform assets -> vault grants -> workstation/profile -> vendors
```

Offboarding is complete only when every edge is removed. Password rotation alone is insufficient where delegated access persists.

6.9 Incident containment

When one client is compromised/restricted:

1. stop automation/changes for that client only;
2. preserve logs and current state;
3. revoke/rotate relevant access;
4. verify no shared billing/vendor/session path affects other clients;
5. use official platform support/appeal routes as appropriate;
6. do not “test” other clients by repeating the suspicious behavior;
7. produce an incident review describing the actual linkage that created blast radius.

[INFERENCE] The correct success metric is **blast-radius boundedness**: one client incident remains one client incident.

SECTION 7 - Content Operations and Platform Behavior: Evidence vs

Feynman primer — treat platform folklore like laboratory rumors

When operators disagree, do not average their stories into a rule. Write the claim, record the source, decide whether it is measurable, and run a controlled test where allowed. “Warmup works” is not one variable; it mixes account history, content, timing, policy eligibility and random recommendation effects. Engineering starts when those are separated.

Mythology

7.1 Why this section is separate from infrastructure

Infrastructure determines whether a device can be operated reliably. It does not prove how a recommendation system will rank content, whether an account has a hidden risk score, or what exact launch routine a platform prefers. Mixing these categories is how operator folklore becomes “technical fact.”

7.2 The Instagram warmup evidence is contradictory by construction

The supplied industry review aggregated many creators/operator groups and records successful approaches ranging from day-one posting to multi-week warmup. It also records conflicting claims about aged accounts, proxies, anti-detect browsers and physical phones.

That source is valuable **because it preserves disagreement.**

[INFERENCE] If one operator can post immediately and another sees lower failure after delaying, then “number of warmup days” cannot be the only explanatory variable. Likely confounders include account provenance, content originality/quality, policy status, recommendation eligibility, audience fit, activity history, current abuse-enforcement pressure and operator behavior.

What can be operationalized safely

- content rights and creator approval;
- complete and accurate profile/business information;
- consistent posting calendars that match a real content strategy;
- quality/recommendation analytics;
- stories/feed/reels as ordinary platform features used within policy;
- A/B testing content formats on authorized accounts;
- tracking changes over time without claiming an undocumented algorithm rule.

What should remain folklore/risk context

- a universal 3/5/7/14-day “safe” warmup number;
- claims that a proxy class guarantees account survival;
- burn-and-replace account economics;
- anti-detect/browser fingerprints as a magic fix;

- “human-looking” automation designed to conceal automation;
- ban-wave bypass instructions.

7.3 Content quality versus account age

A strong early content history can be correlated with better later performance without proving that “warmup” caused it. Treat content and account age as separate variables.

A good internal experiment table looks like:

Variable	Example controlled values	Keep constant
Account age	new vs established authorized account	creator, content class, audience target
Posting cadence	low/medium ordinary cadence	content quality and format
Format	photo/reel/story	account and daypart
Content set	historically strong vs exploratory	posting cadence
Link placement	approved platform-compliant destination patterns	content set

The point is not to discover a loophole; it is to stop attributing normal recommendation variance to whichever ritual happened before a successful post.

7.4 The five-post/fifth-story observation

[INTERNAL-MEASURED] XYZ has one owned Snapchat observation in which an under-optimized SFW account showed a significant traffic/subscriber change after its fifth public-profile story. Operator A independently described a preference for at least five profile items.

This is interesting convergence, but the correct conclusion is deliberately narrow:

- it justifies a controlled hypothesis;
- it does not prove an exact threshold;
- it does not identify the mechanism;
- timing/account aging/content differences remain confounders;
- one account cannot estimate reliability or expected yield.

A legitimate experiment would compare a small number of authorized test accounts/content schedules with clear logging and stop if platform rules or account health indicate a problem. The result should be written as measured data with confidence intervals/limitations, not a production superstition.

7.5 Recommendation eligibility and sensitive content

Social platforms distinguish content that is allowed to exist from content that is eligible for broad recommendation. That distinction is especially important in adult-adjacent creator marketing: content can be non-violating yet receive restricted recommendation/discovery treatment.

[INFERENCE] For OFM-like operations, the creative/compliance system may have more impact on sustainable traffic than the phone hardware. The device farm cannot engineer around a recommendation-policy boundary. It can only make authorized operations more consistent and measurable.

7.6 Content pipeline as an engineering object

Treat each asset as a versioned object with:

- creator/client owner;
- authorization/consent status;
- source/master checksum;
- content classification;
- allowed platforms/uses;
- derivative/variant lineage;
- approval state;
- expiry/revocation status;
- campaign/tracking metadata;
- device staging history.

The system should be able to answer: “Who approved this exact file, which accounts received it, when was it removed, and what performance metrics were associated with it?”

7.7 Attribution

Use first-party analytics, campaign parameters and owned redirect/landing-page analytics where permitted. Separate:

1. content impression/reach;
2. profile visit;
3. link click;
4. landing-page session;
5. downstream subscription/purchase where the platform and privacy policy permit attribution.

Do not pretend last-click proves causation. Retain channel/account/content identifiers so the team can compare performance without needing invasive cross-client tracking.

The technical fleet exists to support a controlled creator-content workflow. This part focuses on authorization, media, publishing, employees and measurement.

42. Creator onboarding and authorization

Before an account or device is assigned, the creator relationship should be documented and operationally represented in the system.

Onboarding packet

Record legal/business identity as required by the agency, contract reference, platform account ownership, recovery-channel ownership, allowed content categories, geographic/commercial restrictions, approved employees, revenue attribution identifiers and exit procedure. Sensitive identity documents require stricter access than routine content metadata.

The creator should know which agency systems and people will access which accounts. Authorization should be revocable.

- Contract/authorization.
- Account inventory.
- Recovery contacts.

- Content rights/consent.
- Brand rules.
- Escalation contacts.
- Offboarding instruction.

Technical onboarding

Create creator and account records, issue vault entries, assign least-privilege roles, import only approved media, configure tracking links, run one controlled publishing test and verify analytics before scaling.

43. Media ingest and rights metadata

Every media asset should enter through a controlled ingest point instead of being copied through random employee devices.

Figure 43-1. Rights-aware content pipeline.

Ingest

When an asset is received, calculate a checksum, store the original immutably, attach creator/campaign ownership, record source and rights scope, and create a QC status. The ingestion system should not silently strip or rewrite evidence relevant to rights and provenance.

If the agency performs edits, retain an edit lineage so the approved published variant can be traced to the source.

- Asset UUID.
- Checksum.
- Creator/campaign.
- Source.
- Rights/consent reference.
- Import timestamp.
- Classification.
- Retention/expiry.

Sensitive content

Adult material requires strict access control. Avoid unrestricted local synchronization. Use role-based access and watermarking for internal review only if it does not alter source evidence and the creator approves the workflow.

44. Content processing and variants

Content processing should optimize for platform-supported formats and campaign testing without deceptive manipulation.

Variant generation

Create explicit variants for aspect ratio, safe crops, subtitle/caption burn-in where authorized, cover frame, encoding bitrate and clip length. Store the transformation recipe and version ID so results are reproducible.

Use platform-published media specifications and test upload quality. Avoid “metadata spoofing” intended to make repeated content appear unrelated or to deceive integrity systems; that is different from legitimate transcoding or metadata minimization for privacy.

- 9:16 vertical variant.
- Platform-safe crop.
- Captioned variant.
- Thumbnail/cover.
- Source-preserving archival copy.

Quality control

Automated QC can detect resolution, duration, codec, audio presence, file corruption and duplicate checksum. Human QC should confirm brand, consent, visible private information and platform suitability.

45. Approval workflow

Approval is a gate, not a chat reaction. A task should prove which version was approved, by whom, for which account and time window.

Approval object

Approval should reference immutable asset version, creator/campaign, target platforms/accounts or a defined scope, approver identity, timestamp and expiration/revocation if applicable. A later edit creates a new version requiring new approval unless the policy explicitly allows that transformation.

High-risk or ambiguous content can require two-person review: creator/manager plus compliance/operations.

- Asset/version.
- Scope.
- Approver.
- Timestamp.
- Expiry.
- Notes/restrictions.

Revocation

If a creator withdraws authorization, the system should be able to identify queued tasks and previously published references associated with the revoked asset so operators can stop future use and execute contractual/platform removal procedures.

46. Publishing paths and official APIs

Publishing should use the most supportable path available: official API, approved scheduler/integration, or human-operated managed phone. The coordinator records all three the same way.

Official API path

TikTok's Content Posting API allows registered applications to post content on behalf of authorized creators after the app obtains the required product configuration and scope; unaudited clients have restrictions. [S22] This demonstrates the preferred automation pattern: explicit creator authorization, documented API scope and platform-controlled rate/feature constraints.

For other platforms, verify current first-party developer documentation because availability depends on account type, application review and changing product policy.

- OAuth/delegated authorization where supported.
- Rate limiting.
- Idempotency.
- Post/status ID capture.
- Audit of API actor and creator authorization.

Human device path

If an action is mobile-only or API-inaccessible, create an operator task with approved asset, caption guidance and target account. The operator performs the action in the native app and records verification. The coordinator should assist the human, not secretly impersonate the human.

47. TikTok operational notes

TikTok is particularly explicit about integrity and automation. Its current guidelines allow authentic multi-account use but prohibit spam, fake engagement, circumvention and automation used to register or operate accounts in bulk. [S23]

Compliant engineering posture

Use the Content Posting API when the desired workflow is supported and the application can satisfy authorization/audit requirements. Otherwise use trained operators on authorized creator accounts and keep posting cadence/content strategy inside current platform rules.

Build analytics around content quality and conversion rather than account multiplication. A smaller number of healthy, authorized accounts with measurable attribution is easier to govern than a disposable-account model.

- Do not automate bulk account creation.
- Do not buy/sell or misrepresent account identity.
- Do not manufacture views, likes, comments or follows.
- Do not build location-hiding or enforcement-circumvention mechanisms.

Operational telemetry

Record post ID, publishing path, asset version, timestamp, operator/API application, outcome and available reach/click data. If a post is rejected or account restricted, preserve the platform notice and route it to policy review rather than automatically creating replacements.

48. Reddit, X and Snapchat operational notes

Each platform has its own rules, but authenticity and spam controls are common themes.

Reddit

Reddit requires authentic participation and prohibits spam/content manipulation. [S24] Subreddit-specific rules can be stricter than site-wide rules. Build a subreddit rule register if Reddit is part of the creator strategy; do not assume an action allowed in one community is allowed in another.

Where API access is used, comply with current Reddit Data API terms and rate/access conditions. [S25]

- Record subreddit/community rules and last review date.
- Avoid repetitive cross-posting that violates community spam rules.
- Respect NSFW labeling and content restrictions.

X

X's authenticity rules recognize legitimate third-party management but prohibit coordinated inauthentic behavior, duplicate/substantially similar amplification and ban evasion. [S27] This supports a model where the agency manages authorized creator accounts without creating fake networks to manipulate reach.

Snapchat

Snapchat's deceptive-practices policy prohibits spam, artificially inflated engagement, follower-growth schemes and other deceptive behavior. [S26] Native content workflows should therefore remain human or platform-supported rather than driven by hidden engagement bots.

49. Instagram/Meta operational notes

Meta's products have formal business/developer tooling as well as anti-spam and authenticity rules. Exact publishing capabilities and account requirements change, so the deployment should treat Meta integrations as versioned connectors that are validated against current first-party documentation before release.

Source note

Meta policy/API pages can change frequently and were not used here as a step-by-step integration specification. Validate the exact current first-party endpoint, account type and review requirements at implementation time.

Architecture approach

Prefer supported Meta business/account access and approved publishing integrations where available. Maintain separate account ownership, page/profile/business relationships and authorization records in the control plane.

Avoid building the system around large-scale repetitive posting, synthetic engagement or replacement-account churn. Even if a device can technically perform an action, the business should first ask whether the action is permitted and sustainable.

- Verify current Instagram API publishing requirements.
- Use delegated business roles where available.
- Capture post/media IDs for reconciliation.
- Monitor access-token expiry and permission changes.

Native app tasks

Stories, certain editing features or emerging formats may reach the native app before APIs. Route these to a human task and maintain evidence of the approved asset and target account.

50. OnlyFans boundary and account responsibility

OnlyFans is the monetization endpoint in many OFM funnels, but the social phone farm and the OnlyFans account should be treated as separate systems. Agency assistance does not remove the creator's legal or contractual responsibility for the account. Publicly archived terms and UK parliamentary evidence have historically emphasized creator control/responsibility; verify current OnlyFans Terms directly before implementation. [S28][S29]

Verification warning

OnlyFans terms are especially important to verify at the time of deployment. Some accessible web sources are archived or secondary, so do not rely on this chapter as a current legal opinion.

Access

Do not assume that a social-media operations employee needs OnlyFans credentials. Separate traffic acquisition roles from subscriber/chat or account-administration roles. This reduces both privacy exposure and credential blast radius.

If third-party account assistance is permitted under current terms and creator contract, document the authority, employee scope, authentication method and revocation process.

- Creator retains required ownership/control.
- Agency access explicitly authorized.
- Separate roles and least privilege.
- MFA/recovery ownership documented.
- Offboarding and session revocation tested.

Data separation

Keep public social analytics, creator media and subscriber/private customer data in separate access domains. Do not copy subscriber conversations into a growth dashboard unless there is a documented lawful/business need and appropriate protection.

51. Tracking links and attribution

Without attribution, a phone farm can produce activity without knowing what generates revenue. Use campaign-aware links and first-party landing analytics where lawful.

Link structure

Create a campaign/link ID that resolves to the intended destination. Store creator, platform, account, campaign, asset and optional post reference. The redirect service records only necessary event data and should respect privacy/consent requirements.

UTM parameters are useful for general analytics, but an internal opaque link ID can be easier to rotate or update without exposing internal naming.

- Link ID.
- Creator/campaign.
- Source platform/account.
- Asset/post reference.
- Created/expired dates.

- Destination.

Attribution levels

Deterministic attribution uses a directly observed click and conversion identifier. Probabilistic or modeled attribution estimates influence when deterministic linking is unavailable. Label modeled results clearly; do not present them as exact facts.

Use cohort comparisons and revenue per campaign rather than optimizing solely for impressions.

52. Employee roles and access model

Employees are part of the system. Define roles so a new hire does not automatically receive every creator credential and media library.

Example roles

Fleet technician manages devices/network but does not need creator social passwords. Content operator sees assigned approved assets and target accounts. Growth manager sees analytics and campaign configuration. Security/admin manages access and incidents. Finance sees revenue/cost summaries but not raw sensitive content unless required.

Role-based access should be supplemented by creator/account scope. A content operator may be authorized for Creator A but not Creator B.

- Fleet technician.
- Content operator.
- Growth manager.
- Creator manager.
- Security/admin.
- Finance/analytics.

Joiner/mover/leaver

Provision named accounts, train the employee, issue hardware/keys, review access when role changes, and revoke access immediately at departure. Rotate shared recovery secrets if any existed, and reconcile devices/keys physically returned.

Role	Devices	Creator media	Social credentials	Analytics
Fleet tech	Admin	No/minimal	No	Health only
Content operator	Assigned	Assigned only	Delegated/assigned	Task-level
Growth manager	View	Campaign	Usually no password	Broad
Security admin	Admin	As incident requires	Vault admin only	Audit
Finance	No	No	No	Revenue/cost

53. Daily/weekly operating cadence

A phone farm should be run as an operations function with handoffs, not as continuous improvisation.

Daily

Start with health: offline devices, failed backups, full storage, expired credentials, network alerts and platform notices. Then review the approved publishing queue. End the shift with unresolved incidents and blocked tasks explicitly handed over.

Avoid measuring employees purely by number of posts; that incentivizes spam and hides quality or conversion problems.

- Morning fleet health check.
- Approval/queue review.
- Execute tasks.
- Verify and reconcile post IDs.
- Review alerts/restrictions.
- Shift handoff.

Weekly

Report output, funnel metrics, account health, device/network reliability, policy incidents, top/bottom content, labor usage and next-week experiments. The report should say what is blocked and who owns resolution.

54. Quality control and audit

Quality control verifies content, account and task correctness; audit verifies who changed what.

QC sampling

Randomly sample completed tasks and compare the published post with the approved asset/caption/target. Verify links resolve correctly and attribution metadata is present. Sample failed tasks to ensure error classification is honest rather than simply retried until “success.”

Periodically inventory physical devices against the database and MDM/Android Enterprise records.

- Published asset matches approval.
- Correct creator/account.
- Correct tracking link.
- Post ID captured.
- No unauthorized local asset remains.
- Device assignment matches physical rack.

Audit log

Write immutable or append-only records for login, role changes, credential access, creator authorization changes, task approvals, high-impact device commands and deletions. Retain according to legal/business requirements, not indefinitely by default. # SECTION 8 - Control Plane and Remote Operations

Feynman primer — the control plane is air-traffic control

The phones are airplanes. The control plane should know which ones exist, whether they are healthy, what authorized task is assigned, who approved it and what happened. It should not be the mechanism that physically “flies” every phone; adapters do that. This separation lets you replace a vendor control board, WDA or a host without rewriting the business brain.

8.1 Remote desktop is maintenance, not the product UI

Dim’s handover correctly warns against “video of video”: a remote user watching a Mac/PC desktop that itself shows 20 live mirrored phones compounds compression, bandwidth and latency.

[INFERENCE] The operator UI should therefore render **structured device cards**:

Device 07		online		31 C		74% battery		USB ok		net ok
Account		authorized asset reference								
Task		media-stage #884 step 3/5								
Preview		screenshot age 2.1s [refresh]								
Actions		inspect		retry		quarantine		open live session		

Live video is an escalation tool for one device, not the default dashboard for all devices.

8.2 RustDesk/GADS/custom roles

- **RustDesk/remote desktop**: host maintenance, emergency manual work, software installation, OS troubleshooting.
- **GADS**: reservations, per-device screen/control and multi-user lab workflow. [MAINTAINED-DOC] Current GADS iOS streaming uses WebDriverAgent; the current repository is also dual-licensed, with AGPL-3.0 open components and a proprietary hub-ui component. Therefore “GADS is fully open source” is not precise. WDA also collides with a strict no-phone-side-control-app policy and must be accepted as a deliberate architecture decision.
- **Custom control plane**: stable inventory/task/audit layer that can call vendor adapters or a WDA adapter without coupling business workflow to a single UI tool.

[INFERENCE] These are complementary layers, not mutually exclusive products.

8.3 macOS permissions and commissioning

Remote-control software on macOS depends on privacy/TCC permissions such as Screen Recording and Accessibility/Input Monitoring. Commission the host physically once, record the required grants, and create a recovery runbook for the case where an OS update/reinstall removes a grant.

The commissioning record should include:

- macOS version/build;
- remote-access client version;
- which permissions were granted;
- FileVault state/recovery method;
- Remote Login state and firewall scope;
- local break-glass admin identity;
- hostname/static reservation;
- time sync;
- automatic restart after power failure if appropriate;
- monitoring agent state.

8.4 FileVault without surrendering unattended recovery

[VERIFIED-FIRST-PARTY] Apple now documents that on Apple-silicon Macs with macOS 26 or later, FileVault can be unlocked over SSH after restart when Remote Login and network connectivity are available.

[INFERENCE] This changes the security tradeoff materially. The default should not be “turn FileVault off because the rack is remote.” Instead, test the supported SSH-unlock workflow in the exact network/firewall environment, protect the credentials/keys, and maintain a physical break-glass procedure.

Remote recovery ladder

[VERIFIED-FIRST-PARTY] Apple also documents high-performance Screen Sharing on Apple-silicon Macs. [VERIFIED-FIRST-PARTY] For supported 10GbE Mac mini configurations, Lights Out Management can add remote power actions under documented MDM/same-subnet requirements.

[INFERENCE] Operate the host with multiple independent recovery paths:

Layer	Purpose	Typical failure it can recover
node agent/control plane	normal service restart/reconciliation	one adapter/process wedged
SSH/Remote Login	command-line repair	daemon, config, routing, disk/log issue
Screen Sharing/RustDesk	GUI repair	TCC permission, Xcode/Configurator, desktop-only action
Lights Out Management (optional)	power-cycle/start/shutdown	host OS unreachable but management path available
physical break-glass	last resort	power/network/hardware failure

A rack is not remotely operable merely because RustDesk connected once during setup.

8.5 Node agent

Each host runs a node agent that:

- enumerates local devices;
- maps stable internal device UUIDs to transient USB/vendor IDs;
- reports heartbeat/resources;
- receives tasks for devices it owns;
- invokes a vendor-specific adapter;
- captures observations/screenshots/logs;
- enforces local concurrency/resource limits;
- survives coordinator interruption and reconciles afterward.

The coordinator never assumes a device exists because a database row says it exists. The node agent proves liveness.

The coordinator makes the phone farm a system rather than a pile of devices. It should model devices, creators, accounts, assets, approvals, tasks, results, networks and employees as separate entities with an auditable workflow.

32. Control-plane architecture

A practical control plane consists of an API service, relational database, object/media storage, task queue or broker, device/fleet connectors, authentication/RBAC, metrics/logging and an operator web interface.

Figure 32-1. Control plane and execution layers.

Service boundaries

The API service owns business rules. The relational database stores durable entities and task state. Object storage holds media and large artifacts. A broker or queue delivers asynchronous work. Device connectors translate generic management intents into Apple MDM, Android Enterprise, internal companion-app or other supported actions.

Keep platform publishing connectors separate from device-management connectors. This prevents a social API outage from breaking fleet administration and makes authorization boundaries clearer.

- API/auth service.
- PostgreSQL or equivalent relational store.
- Object storage.
- Queue/broker.
- Fleet connectors.
- Publishing connectors.
- Analytics/event pipeline.
- Web dashboard.

Deployment

For a pilot, these can run on one secured server using containers. For production, separate stateful services, backups and secrets. Avoid unnecessary microservices: operational complexity grows faster than device count.

33. Device-to-coordinator communications

There are two broad channels: enterprise management channels provided by Apple/Google, and a custom internal agent/companion channel controlled by the agency. Do not reinvent MDM for functions MDM already performs.

HTTPS model

An internal companion app can periodically call an HTTPS endpoint to register heartbeat, fetch permitted task metadata and report completion. Use short-lived OAuth-style tokens or device certificates rather than embedding a shared static API key in every phone.

For near-real-time updates, WebSocket or server-sent events can be added, but simple polling is often easier to operate at 10-100 devices. Poll intervals should use jitter so all devices do not hit the server simultaneously.

- TLS for all remote traffic.
- Unique device identity.
- Token rotation/revocation.

- Idempotent task receipt.
- Clock synchronization.
- Backoff on failure.

Message broker

MQTT is a lightweight publish/subscribe protocol widely used for constrained devices and IoT-style messaging. [S20] A companion app could publish health telemetry and subscribe to a narrow command topic, but do not expose a broker directly to the public internet without strong authentication, authorization and TLS.

A queue/broker is transport, not the source of truth. Durable task state belongs in the database.

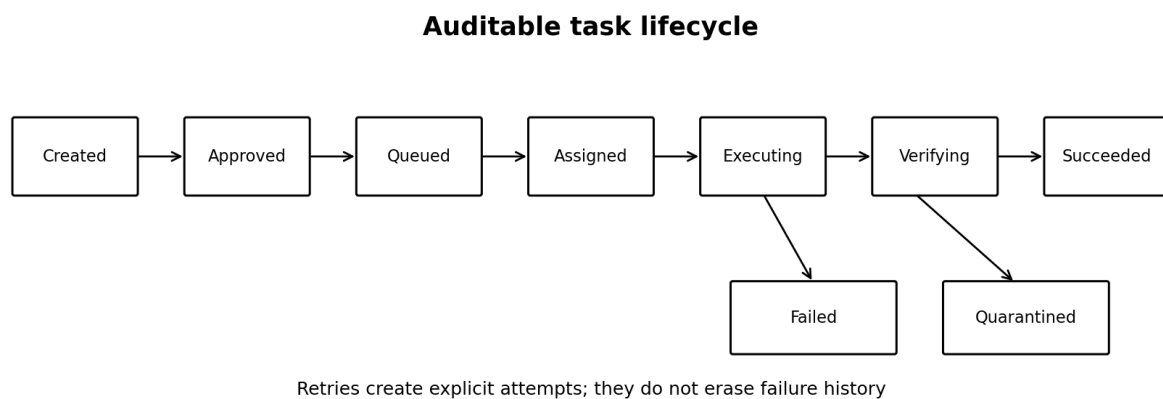


Figure 5 — Auditable task lifecycle.

34. Task queue and state machine

Every action should move through explicit states so operators know whether it is waiting, running, verified or failed. Silent retries hide problems and can create duplicate posts.

Reference states

Draft: not yet approved. Approved: content/action permitted. Queued: eligible for scheduling. Assigned: bound to executor/device/operator. Executing: in progress. Verified: expected result observed. Closed: complete and immutable except for annotations. Failed tasks enter a review/retry branch with a reason.

Use idempotency keys for API publishing so a network retry does not create a second post. Human tasks should require the operator to record the platform post ID or a verification artifact before closure.

- Maximum retry count.
- Retryable vs terminal error classes.
- Deadline/SLA.
- Priority.
- Executor type.
- Required approval IDs.
- Verification method.

Scheduling

Scheduler logic should consider platform/account restrictions, operator shift, device readiness, campaign windows and content approvals. It should not create prohibited spam by blindly maximizing throughput.

35. Database and entity model

The database should model real business entities instead of embedding everything in a spreadsheet row. This supports permissions, history and reporting.

Figure 35-1. Core entity separation.

Core entities

Creator represents the contracted person/business. Account represents one platform account and stores ownership/authorization metadata, not plaintext credentials. Device represents a physical asset. NetworkProfile describes approved egress configuration. MediaAsset stores immutable asset metadata and storage URI. Approval links a person to a permitted action/asset. Task describes work. Event/Metric stores observed outcomes.

Use stable UUIDs internally. Human-readable labels can change; primary identity should not.

- Creator.
- PlatformAccount.
- Device.
- NetworkProfile.
- MediaAsset / AssetVersion.
- Approval.
- Task / TaskAttempt.
- Operator.
- AuditEvent.
- MetricEvent.

History

Use append-only audit records for security-relevant changes. For mutable business records, store updated_at and updated_by; for especially sensitive fields keep version history or event-sourced change records.

36. Object/media storage

Creator media is valuable and sensitive. Centralize it in access-controlled object storage rather than uncontrolled phone camera rolls and employee downloads.

Storage model

Use immutable source assets plus derived versions. Store SHA-256 or another strong checksum for integrity, creator ID, usage authorization, source, creation/import timestamp, classification and retention/expiration. A derived version should point back to its source and transformation recipe.

Phones should download only the assets required for assigned tasks, ideally through time-limited signed links. Clear local staging areas on reassignment/offboarding where technically possible.

- Server-side encryption.
- Restricted buckets/containers.
- Versioning where useful.
- Lifecycle/retention rules.
- Backup/replication.
- Access logging.

Content safety

Only store adult creator content for verified adults with documented authorization and applicable consent/releases. Build a takedown/revocation process that can locate every derived copy from an asset ID.

37. API design

The internal API should expose business intentions, not raw device-control primitives. “Assign approved asset X to account Y” is safer than “tap coordinate 400,900.”

Resource examples

Use REST or another consistent RPC model with resources such as `/devices`, `/accounts`, `/assets`, `/approvals`, `/tasks`, `/events` and `/metrics`. Every write should authenticate the caller and enforce role/creator scope.

Use request IDs and idempotency keys. Return machine-readable error codes. Record actor, source IP/device, target object and result in audit logs for security-relevant writes.

- GET `/devices/{id}`.
- POST `/tasks`.
- POST `/tasks/{id}/approve`.
- POST `/tasks/{id}/assign`.
- POST `/tasks/{id}/result`.
- GET `/accounts/{id}/metrics`.

Versioning

Version the API contract before external clients depend on it. Internal rapid changes are easier if the companion app and backend can tolerate one previous schema version during rolling updates.

38. Messaging and event transport

Asynchronous messaging decouples slow device/network actions from interactive web requests. A user should not keep an HTTP request open while waiting for a phone task to finish.

Queue semantics

At-least-once delivery is common and requires idempotent consumers. Exactly-once behavior is usually implemented at the application level using unique keys and database constraints rather than assumed from the broker.

Use separate queues/topics for device commands, publishing jobs, analytics ingestion and low-priority maintenance so one workload cannot starve another.

- Dead-letter queue.
- Retry delay/backoff.
- Maximum age.
- Consumer concurrency.
- Per-account/device serialization where necessary.

MQTT topic example

For telemetry, a namespace such as `fleet/{device_id}/health` and `fleet/{device_id}/events` is easy to authorize. Commands should be narrow and typed. Do not create a universal remote-shell topic in a production broker.

39. Metrics, logs and traces

Prometheus is a common metrics and alerting system built around time-series data and label dimensions. [S21] Whether using Prometheus or another stack, distinguish metrics, logs and traces.

Metrics

Metrics answer aggregate questions: online device count, task success ratio, queue depth, API latency, Wi-Fi loss, VPN tunnel availability. Keep label cardinality controlled; do not use a unique post URL as a Prometheus label if it creates millions of series.

Logs answer event details: device enrolled, task failed, credential reference rotated. Traces connect calls across services when debugging distributed workflows.

- Fleet online %.
- Task success/failure by executor.
- Queue age P50/P95.
- Publishing connector error rate.
- Database latency.
- Storage capacity.
- Network health.

Alerting

Alerts should be actionable: “>20% of devices in rack B missed three heartbeats” is useful; “CPU 71%” may not be. Add runbook links to alerts.

40. Credential and secret handling

Credentials are one of the highest-value assets in an OFM system. Do not store platform passwords, session cookies, MFA seeds or API keys in source code, shared spreadsheets, task descriptions or chat channels.

Vault model

Use a password/secret manager that supports per-user access, audit logs, MFA and revocation. The application database stores a reference such as `vault://creator/123/platform/x` rather than the secret value. Backend services receive only the secret they need, ideally for a limited time.

Where a platform supports delegated access, teams/business accounts or OAuth, prefer that over sharing the creator's primary password.

- Unique operator accounts.
- MFA/hardware keys for administrators.
- No shared root credential.
- Recovery codes stored securely.
- Immediate revocation on offboarding.
- Periodic access review.

Session management

Social apps may maintain long-lived sessions on phones. Treat the device itself as a credential-bearing asset: lock it, encrypt it, inventory it, and wipe/revoke on loss or reassignment.

41. Availability, backup and disaster recovery

The control plane should survive ordinary failures without losing authorization records, task history or media.

Backup scope

Back up relational databases, configuration, audit records, object-storage metadata and essential media according to business/legal requirements. Secrets should be backed up through the vault's supported recovery mechanism, not exported into general backups.

Test restore. A backup that has never been restored is an assumption, not a recovery plan.

- Define RPO: acceptable data loss window.
- Define RTO: acceptable service recovery time.
- Keep at least one backup copy isolated from primary credentials.
- Document dependency order: identity -> database -> object store -> queue -> API -> workers.

Degraded operation

If the coordinator is down, operators should have a clear rule: pause new publishing or use an approved manual fallback with later reconciliation. Do not create a shadow spreadsheet that permanently bypasses audit and approval controls. # SECTION 9 - Observability, Reliability, Security and Incident

Feynman primer — build the black box before the crash

Logs and metrics are only valuable if they explain failure after nobody remembers what happened. Every pod should report enough state to distinguish power loss, USB enumeration loss, network loss, app/tool failure and operator error. Reliability means failures are bounded, detected and recoverable; it does not mean failures never happen.

Response

The fleet contains devices, credentials and sensitive creator content. Security therefore has to be designed into networking, software and employee workflows.

55. Threat model

A threat model identifies assets, adversaries, attack paths and controls. The most realistic threats are often credential theft, malicious/curious insiders, lost devices, vulnerable phones, exposed admin panels, leaked media and configuration mistakes.

Assets

Creator accounts, social sessions, content, legal authorization records, subscriber/private data where applicable, analytics/revenue, device inventory, network credentials and administrative secrets.

Rank assets by impact and limit who can access them. A fleet technician does not need subscriber data; a growth analyst does not need MDM wipe rights.

- External credential phishing.
- Malware on admin workstation.
- Stolen/unreturned phone.
- Insider media exfiltration.
- Misconfigured public storage.
- Compromised VPN/proxy account.
- Unpatched mobile OS.

Controls

MFA, vaulting, MDM, encryption, segmentation, EDR on admin hosts, patching, least privilege, secure backups, audit logs and employee offboarding cover most high-probability paths better than exotic anti-fraud tricks.

56. Platform restriction and enforcement handling

When a platform restricts a post or account, treat it as a policy/operations incident, not an automatic trigger to create replacement identities.

Workflow

Preserve the notice, account/post ID, timestamp, asset, publishing path and recent changes. Determine whether it is content policy, spam/integrity, security login, copyright, age/consent or an unknown issue. Use the platform's appeal/remediation process where appropriate.

Pause similar queued tasks if the issue may repeat. A single restriction can reveal a broken campaign rule that would otherwise affect dozens of accounts.

- Capture evidence.
- Classify cause.
- Contain similar tasks.
- Appeal/remediate legitimately.
- Update campaign/policy rule.
- Document decision.

Prohibited response pattern

Do not make “ban evasion” a software feature. Automated creation of replacement accounts, identity/fingerprint spoofing, location falsification or rapid proxy/device churn can violate platform rules and create fraud/security risk. TikTok and X explicitly address circumvention/ban evasion in their authenticity rules. [S23][S27]

57. Incident response

Incident response should be predefined because the first minutes of a credential leak or overheating event are not the time to invent authority and evidence rules.

Figure 57-1. Incident-response sequence.

Seven stages

Detect -> contain -> preserve evidence -> assess scope -> remediate -> recover -> review. For a lost phone, containment may include MDM lock/wipe and secret revocation. For overheating, containment is physical power isolation and safe handling. For a content leak, containment includes access revocation and distribution/takedown actions consistent with law and platform processes.

Maintain an incident commander role so multiple employees do not issue conflicting changes.

- Incident ID and severity.
- Affected creator/account/device.
- Timeline.
- Actions/actors.
- Evidence.
- External notifications if required.
- Root cause and corrective action.

Testing

Run tabletop exercises for: lost device, compromised operator credential, database outage, storage exposure, creator revokes access, network failure during publishing, and rack thermal alarm.

58. Privacy and data minimization

A growth/fleet system should not become an uncontrolled surveillance database. Collect the minimum data needed to operate and secure the business.

Minimize

Device health does not require reading personal messages. Network health does not require storing full destination histories indefinitely. Analytics often need campaign/link IDs rather than raw IP addresses. Separate security logs from marketing data and apply different retention policies.

If employees operate creator accounts, document monitoring and privacy expectations in employment/contract policy and comply with applicable labor/privacy law.

- Purpose for each field.
- Retention period.
- Access role.

- Encryption requirement.
- Deletion/export process.

Data subject/creator exit

Be able to locate creator-specific records and assets for contractual deletion/return while preserving records the agency is legally required to retain. Avoid schemas that mingle multiple creators' sensitive content in untraceable folders.

59. Business continuity and creator exit

Continuity is not only infrastructure uptime; it includes a clean way for a creator to leave without the business or creator losing control of accounts.

Creator exit

Freeze new tasks, inventory pending posts, return/transfer authorized credentials or delegated roles according to contract/platform capabilities, revoke agency sessions, remove employee access, stop scheduled content, export agreed analytics/assets, and wipe creator material from reassigned devices.

Confirm recovery email/phone and MFA ownership before disabling agency access so the creator is not accidentally locked out.

- Stop queue.
- Reconcile scheduled/published content.
- Transfer/revoke access.
- Export/return agreed assets.
- Delete/wipe according to policy.
- Record completion.

Continuity

Document who can operate if the primary fleet admin is unavailable. Store network/MDM recovery procedures securely. At least two appropriately authorized people should be able to recover the system without sharing one administrator password. # SECTION 10 - Experiments, Pilot Build and Scaling Gates

Feynman primer — do not build a twenty-lane bridge before testing one span

Every expensive assumption becomes a gate. A gate has a test, measured outputs and a pass/fail rule. The sequence $1 \rightarrow 2 \rightarrow 5 \rightarrow 10 \rightarrow 20$ is not timidity; it is how you discover nonlinear limits before multiplying them by twenty. A failed test is valuable because it prevents a bad purchase.

10.1 Experiment discipline

The project has enough contradictory research that architecture-by-opinion would be expensive. Every experiment should have:

- **question** - one decision it resolves;
- **setup** - exact device/model/OS/host/hub/software versions;
- **procedure** - reproducible benign steps;
- **measurements** - logs, timestamps, power, CPU/RAM, packet/USB observations as relevant;

- **pass/fail** - defined before the test;
- **artifacts** - screenshots/logs/results stored in the repo;
- **decision unlocked** - which ADR/procurement choice may now be made.

10.2 Gate 0 - inventory before engineering

Record what already exists before buying alternatives:

- every phone model/OS/battery health/port state;
- chassis/hub make, upstream link, PSU rating;
- Windows host specs;
- available Mac mini candidate;
- Xiaowei version/configuration and exact successful steps;
- inbound Niazun SKU/order/doc/API package;
- cables/adapters/power supplies;
- network/AP/router model and available VLAN/policy-routing features.

The fastest way to waste money is to design from a vendor listing while the actual bench already disproves it.

10.3 TEST-MAC-CAPTURE-001 - does the proposed Mac USB display path exist programmatically?

Question: Can one or more tethered iPhones be addressed programmatically as screen/video sources without WDA, AirPlay/network mirroring or a phone-side control app?

Method: use owned test iPhones and public Apple-supported APIs/tools only. Enumerate capture sources, open streams if exposed, record resolution/fps/latency/CPU/RAM/USB behavior. Repeat at 1, 2, 5 devices before extrapolating.

Pass: a documented, repeatable programmatic stream for each connected phone with stable identity and no LAN dependency.

Fail: only GUI QuickTime capture; no separately addressable public stream; instability beyond one phone; hidden network dependency.

Decision: whether Mac USB display can replace network mirroring.

10.4 TEST-BOARD-001 - inbound control board capability

Validate only the purchased unit before ordering more:

- enumerate/control through existing hub;
- API authentication and device identification;
- click/swipe/text on a test screen;
- reconnect after cable/host restart;
- one script repeated 20 consecutive cycles;
- vendor process network connections captured for supply-chain review;
- CPU/RAM usage and crash behavior;
- whether the board requires/changes USB topology.

Do not connect production client accounts during this test.

10.5 TEST-IOS-INPUT-001 - characterize event type in an owned test app

Build a tiny internal UIKit test app that logs `UITouch.TouchType` and gesture event classes. Compare finger versus the control board on each relevant iOS version. This is a platform-engineering measurement, not a third-party detection test.

Pass: event behavior is documented and reproducible.

Decision: understand compatibility risk for apps/UI frameworks; do not infer enforcement behavior.

10.6 TEST-MEDIA-IOS-001 - real Photos ingestion

Send a known test JPEG/video through the board's claimed file/media route. Verify:

- asset visible in Photos;
- visible to a normal image picker;
- correct orientation/type/duration;
- duplicate behavior;
- deletion/reconciliation;
- batch behavior;
- behavior after restart.

A successful filesystem copy with no Photos asset is a fail.

10.7 TEST-COEXIST-001 - display and input simultaneously

If Mac USB capture works and the board works, connect them in the intended final topology. Confirm the phone stays visible to the host while accepting external input. This settles the "USB device versus accessory host" concern empirically for the actual board.

10.8 TEST-CAPACITY-001 - pod scaling curve

Run the **real workload**, not idle connected phones, at 1/5/10/20 devices. Measure:

- host CPU/GPU/RAM;
- USB errors/enumeration loss;
- network throughput/mDNS/discovery if used;
- screenshot/display latency;
- task success/latency;
- power draw/temperature;
- remote-operator responsiveness.

Stop adding devices when reliability or recovery becomes nonlinear. That point, not the hub's printed port count, defines pod size.

10.9 TEST-THERMAL-001 - soak

Run at expected brightness/charge/control workload for at least one business-day window, then longer. Record per-device/rack temperature and power. Inspect batteries/ports. Test fan failure if safe and monitored.

10.10 TEST-REMOTE-001 - unattended recovery

From a remote location, verify:

- host reboot;
- FileVault recovery method if enabled;
- remote-access service start;
- node agent start/reconciliation;
- device re-enumeration;
- one failed device quarantined without losing the pod;
- permissions failure detection.

10.11 Scale gates

Gate 1 - 1-2 device evidence bench

All architecture-blocking unknowns measured. No volume purchase.

Gate 2 - 5-device pilot pod

Inventory/control plane running; authorized test content; independent health; remote recovery; audit logs.

Gate 3 - 10-device stable pod

Seven-day operational soak; no unexplained enumeration loss; known failure/retry behavior; operator can run it without Dim physically present.

Gate 4 - 20-device production pod

Only if capacity data supports it. Power/thermal margin proven; remote operations usable; incidents bounded; vendor adapter stable.

Gate 5 - multi-pod

Scale by duplication. The central control plane adds pods; it does not increase one pod's mystery density.

10.12 Purchase rule

[INFERENCE] Never purchase 20 proprietary control boards because one vendor demo looked good. The minimum procurement evidence is: one board, exact hub, exact phone/OS, exact host, 20 unattended cycles, media/control path confirmed, supply-chain containment plan documented, and an adapter interface implemented.

Scale in stages. A ten-device pilot should prove enrollment, physical design, network stability, queueing, attribution and incident recovery before the agency buys fifty more phones.

60. Scaling from 5 to 100+ devices

Scaling changes the bottleneck. At five devices, manual labeling and one AP may be adequate. At fifty, power zones, Wi-Fi capacity, management automation and staffing dominate. At one hundred, redundancy, change control and inventory discipline become mandatory.

Stage 1: 5-10

Goal: validate workflow. Use complete standard phones, one or two device models, managed Wi-Fi, a small controller and basic MDM/Android Enterprise. Measure actual power, heat, posting labor and conversion.

Do not optimize rack density yet. Optimize observability.

Stage 2: 25-50

Add segmented VLANs, multiple planned AP cells as needed, rack zones, UPS-backed network/control plane, automated enrollment, formal content approvals, task queue, metrics and scheduled maintenance. Separate backend/database from heavy media processing if necessary.

Stage 3: 100+

Use multiple physical racks/zones, standardized spares, automated inventory reconciliation, redundant network edge, tested disaster recovery, role-separated staff and canary change rings. Consider multiple sites only after one site is operationally mature.

Dimension	5-10	25-50	100+
Enrollment	Manual assisted	MDM/Enterprise standardized	Automated procurement/enrollment
Power	Single measured zone	Multiple protected zones	Rack/site capacity plan
Wi-Fi	1 planned AP may suffice	Capacity-driven multi-AP	RF design + monitoring
Controller	Single host	Separated roles as needed	HA services
Staffing	1-2 cross-functional	Defined roles	Shift + incident ownership

61. Cost model and BOM categories

Total cost is CAPEX plus recurring OPEX. A cheap phone can be expensive if it requires twice the labor or fails frequently.

CAPEX

Phones, rack/enclosure, chargers/PD supplies, cables, PDU, UPS, router/firewall, managed switch, APs, controller/workstations, environmental sensors, spare devices and installation materials.

Price bands vary strongly by country and used-device market. Obtain current local quotes rather than relying on a static web price in this manual.

- Devices.
- Rack/trays.
- Power distribution.
- Network.
- Controller/storage.
- Spares.
- Safety/environmental monitoring.

OPEX

Internet/cellular plans, MDM licenses, cloud hosting, object storage/egress, monitoring, domain/link infrastructure, employee labor, repairs/batteries, platform/developer costs where applicable, legal/compliance and backup storage.

Allocate shared infrastructure by campaign or creator using a transparent model. Labor is often the largest cost in a device-first workflow.

Cost class	Driver	Track as
Phone hardware	Device count/lifecycle	Monthly amortization
Power/network	Rack/site	Shared infrastructure
Software/MDM	Managed devices/users	Recurring license
Cloud/storage	GB, requests, compute	Usage
Connectivity	WAN/SIM plans	Monthly recurring
Labor	Tasks/hours	Direct operating cost

62. Pilot build - 10 device reference

The reference pilot is deliberately conservative. Its goal is to prove the system and gather measurements, not maximize density.

Hardware categories

Ten organization-owned phones from one or two supported model families; a ventilated non-compressing rack/tray; reputable multi-port charging or zoned chargers; labeled data cables for the subset needing USB admin; business firewall/router; managed PoE switch; one appropriately placed business AP; controller workstation; small UPS for network/controller; temperature sensor; spare phone and cables.

Do not purchase dedicated proxy infrastructure for the pilot unless there is a legitimate documented networking requirement. Direct business internet with proper segmentation is simpler to debug.

Software

MDM/Android Enterprise, inventory database, credential vault, basic coordinator/task board, object storage, tracking-link service, metrics/logging and a weekly report. TikTok or other official APIs can be integrated only after platform application authorization requirements are understood.

Pilot exit criteria

Every device can be enrolled/recovered, every account is authorized, every task is attributable, network/thermal behavior is stable, backup restore works, creator exit can be executed, and the business can calculate cost per successful campaign output.

Engineering checklist

- Run 72-hour full-load thermal/network test.
- Simulate one AP/router/controller restart.
- Wipe and re-enroll one device.
- Revoke one operator and verify access is gone.

- Publish/verify through every supported path.
- Restore database from backup.

63. 25-50 device expansion

Expansion is justified only after pilot measurements show stable operations and positive business value.

Physical expansion

Divide the rack into power/service zones, add airflow based on measured heat, add AP capacity based on measured airtime/utilization, and maintain at least 5-10% spare device capacity. Use port/slot maps and environmental alerting.

If a single controller still performs ADB, video transcoding, database and dashboard functions, split those roles before they become a single point of performance and failure.

Process expansion

Introduce formal change management: canary devices, scheduled OS/app updates, weekly access review for high-privilege roles, incident severity levels and capacity dashboards. Add a second trained administrator before the system becomes dependent on one person.

Software expansion

Move task execution to asynchronous queues, implement idempotency, add metrics/alerting, automate inventory reconciliation and make the creator/account authorization model explicit in every API request.

64. 30/60/90 day implementation roadmap

The roadmap assumes the agency starts with no mature device-management control plane. Adjust based on existing infrastructure.

Days 0-30 - prove the foundation

Inventory creators/accounts; define policy boundary; select 5-10 devices; deploy network segmentation; establish MDM/Android Enterprise; create rack/power/thermal pilot; deploy credential vault; implement creator/account/device/asset database; create manual task queue and tracking links; record baseline costs.

Deliverables: physical pilot, authorization register, network diagram, enrollment SOP, incident contact list, first weekly funnel report.

Days 31-60 - automate administration

Build coordinator API and companion/dashboard; add task state machine, approvals, object storage, health telemetry, metrics/alerts, canary updates and backup restore tests. Integrate one supported publishing API such as TikTok Content Posting API if business requirements and app authorization permit.

Deliverables: auditable task flow, fleet dashboard, monitoring, restore evidence, controlled API publishing path.

Days 61-90 - scale and govern

Expand to 25-50 only if KPIs justify it. Add power zones/AP capacity, formal roles, weekly access reviews, incident exercises, creator exit test, cost allocation, performance experiment process and quarterly platform-policy review.

Deliverables: scaled rack, runbooks, staff training, policy register, cost/conversion dashboard and go/no-go criteria for further scale.

65. Acceptance test plan

The installation is not complete because the phones turn on. Acceptance tests prove the physical, network, management, security and business workflows.

Physical/electrical

Measure AC load at idle and representative peak, verify no supply exceeds its rating, confirm rack temperatures in normal and degraded fan conditions, inspect cable strain and confirm emergency disconnect. Run a controlled power failure and verify UPS behavior for network/control systems.

Network/fleet

Verify DHCP/DNS/VLAN policy, Wi-Fi stability, tunnel/proxy behavior if configured, management reachability, enrollment, remote lock/wipe or equivalent supported actions, OS/app inventory and recovery after reboot.

Business/security

Create an approved asset, assign a task, publish via each supported path, capture post ID, verify tracked link, ingest analytics, produce weekly report, revoke an operator, remove a creator, restore a backup and execute one incident tabletop.

Engineering checklist

- All test evidence attached to acceptance record.
- Known exceptions have owner and due date.
- No production scaling before critical failures are closed. # SECTION 11 - Claude / Engineering Execution Specification

Feynman primer — give the coding agent a construction contract, not a wish

An LLM or engineer should not decide hidden business policy while writing code. Define entities, invariants, adapter boundaries, allowed task types, evidence labels, tests and definition of done first. Then implementation becomes a series of checkable changes instead of a pile of scripts.

11.1 What Claude should do first

The coding agent should **not** begin with social-platform automation. It should begin with the neutral fleet substrate that is testable without any client account:

1. repository and configuration conventions;
2. device inventory database;
3. host/node agent heartbeat;
4. vendor adapter interface with a fake/simulator adapter;

5. task state machine;
6. observation/log model;
7. operator dashboard for inventory/health;
8. bench-result storage and Architecture Decision Records.

Only after the physical/control route is measured should the real adapter be implemented.

11.2 Repository layout

```
/device-farm
  /docs
    /adr
    /bench-results
    /runbooks
  /apps
    /control-api
    /operator-web
    /node-agent
  /packages
    /domain
    /db
    /telemetry
    /adapter-sdk
  /adapters
    /fake
    /niaozun          # only after bench/API review
    /wda              # optional policy branch
    /android-adb
  /infra
    /docker
    /monitoring
    /network-reference
  /tests
    /integration
    /hardware-contract
```

11.3 Domain invariants

Claude must encode these rules:

- a device belongs to exactly one active pod/node at a time;
- a task targets a device UUID, not a transient USB index;
- a client/platform asset cannot be used without a current authorization record;
- secrets are referenced, never embedded in task payloads/logs;
- vendor adapter calls are auditable;
- destructive actions require stronger authorization and confirmation;
- task retries are bounded and idempotency-aware;
- a quarantined device receives no production tasks;
- operator identity is recorded on every manual action;
- content assets carry client/creator rights/approval state.

11.4 Minimum data model

```
Client
Authorization
PlatformAsset
Operator
RoleGrant
Pod
HostNode
Device
DeviceObservation
NetworkAssignment
ContentAsset
ContentApproval
Task
TaskAttempt
VendorAdapterInstance
Incident
AuditEvent
SecretReference
BenchExperiment
ArchitectureDecision
```

Do not add “ban replacement,” “stealth score,” or similar evasion-oriented entities. Restrictions/incidents can be logged neutrally for support and reliability.

11.5 Adapter contract

Every device-control backend should implement a small capability-based interface, for example:

```
list_devices()
get_health(device)
capture_screen(device)
input_capabilities(device)
media_capabilities(device)
execute_authorized_action(device, action)
reconnect(device)
collect_diagnostics(device)
```

Actions are typed objects, not raw arbitrary scripts from the web UI. This reduces the blast radius of a compromised operator account and makes adapter replacement possible.

11.6 Task state machine

```
CREATED
-> VALIDATED
-> QUEUED
-> DISPATCHED
-> RUNNING
-> SUCCEEDED
    \\-> FAILED_RETRYABLE -> QUEUED (bounded)
    \\-> FAILED_FINAL
    \\-> HUMAN_REVIEW
```

Every transition has a timestamp, actor/process, reason and correlation ID.

11.7 First implementation backlog

1. Create repo, formatter/linter/test pipeline.
2. Add docs/adr and ADR template.
3. Define domain IDs and enums.
4. Create PostgreSQL schema/migrations.
5. Implement device/pod/host inventory API.
6. Implement node-agent registration/heartbeat.
7. Implement fake adapter with deterministic failure injection.
8. Implement task queue/state machine.
9. Implement audit-event append path.
10. Implement observation storage/retention.
11. Build operator inventory/health page.
12. Add screenshot object-storage abstraction.
13. Add RBAC/individual operator accounts.
14. Add SecretReference interface to chosen vault.
15. Add bench-experiment/result objects.
16. Add alerts for lost heartbeat/device enumeration.
17. Add pod capacity metrics.
18. Add media-asset/approval model.
19. Add client/authorization/platform-asset model.
20. Add device quarantine workflow.
21. Implement hardware-contract test suite against fake adapter.
22. After TEST-BOARD-001, implement Niao zun adapter if accepted.
23. After TEST-MAC-CAPTURE-001, implement capture adapter if public/stable path exists.
24. Add Android ADB adapter only for owned managed/test devices as needed.
25. Add remote-operator escalation/live-session link.
26. Add incident object/runbook links.
27. Add backup/export of inventory/ADRs/bench data.
28. Add 5-device pilot deployment manifest.
29. Run acceptance tests and write results into docs/bench-results.
30. Create ADR: "Pod size and control method for Pilot v1."

11.8 Definition of done for Pilot v1

- five devices continuously represented in inventory;
- stable device identity across host-agent restart;
- per-device heartbeat/health visible;
- task execution through fake + accepted real adapter;
- screenshot/diagnostic capture where supported;
- no plaintext production credentials in database/logs;
- authorized content object can be staged/tested on owned devices;
- individual operator authentication and audit;
- one device can fail/quarantine without blocking others;
- remote host restart/recovery tested;
- power/thermal soak recorded;

- all [OPEN] items that affect Pilot v1 either resolved or explicitly deferred in ADRs.

11.9 “Do not infer” list for Claude

Claude must stop and create a test/ADR rather than assume:

- that Mac USB screen capture scales because QuickTime shows one phone;
- that a vendor board’s file transfer writes Photos assets;
- that a 20-port hub supports 20 simultaneous full workloads;
- that resetting a phone clears backend anti-abuse state;
- that pointer input is indistinguishable from touch;
- that a platform actually consumes an available OS signal;
- that a proxy/VPN choice changes recommendation behavior;
- that a specific warmup duration is a documented platform rule;
- that a browser profile isolates platform ownership/billing relationships;
- that changing a password revokes delegated access;
- that an operator’s revenue/yield number is independently verified.

11.10 Feedback loop

Every bench result should update three things:

1. the experiment result file;
2. the relevant ADR;
3. the handbook’s verification ledger at the next revision.

The goal is a living engineering manual whose uncertainty decreases as the physical system is measured.

11.A - Prior Claude Handoff Deep Reference (updated evidence takes precedence)

4. Blocking experiments - run these before major coding or procurement

These tests are ordered by decision impact. Each test should create a markdown report under `/docs/tests/` with hardware IDs, OS/software versions, exact procedure, timestamps, raw evidence references, result, confidence and follow-up. Where a test uses a custom app, keep it limited to owned test software and neutral endpoints.

4.1 TEST-IOS-INPUT-001 - identify the actual iMouse touch type

Field	Specification
Question	Does the exact iMouse board/software path produce direct or indirectPointer touch events in an owned UIKit test app?
Devices	At least one iOS 16-era device if already owned and one iOS 17+ / current-supported device. Do not procure obsolete hardware solely for this test.
Inputs	Finger tap; standard pointing device; iMouse board click.
Test app	Minimal UIKit app logging UITouch.type, event timestamp, coordinates and app build SDK; include visible counter for evidence.

Field	Specification
Variants	Build once with UIApplicationSupportsIndirectInputEvents omitted, once YES, once NO.
Evidence	Device screen recording, app log, iOS version, app SDK version, iMouse version, board ID.
Pass condition	Actual event type for each combination is documented and repeatable.
Decision effect	Determines whether iMouse is acceptable as a human remote-control method and what observability caveat must be documented. It is not a basis for evasion.

4.2 TEST-NET-EGRESS-001 - Wi-Fi control plane versus cellular data plane

Field	Specification
Question	Can the phone maintain local mirroring/control on the control WLAN while normal Internet access leaves over cellular, and is that stable?
Topology	Dedicated AP/SSID and controller on the same private subnet. First run with normal Internet gateway; second run with WLAN Internet egress intentionally unavailable while local subnet remains reachable.
Measurement	Neutral HTTPS endpoint owned by XYZ logs source address; router records WAN counters; device records cellular-data delta; iMouse mirror stability is logged.
Scope	Test Safari/owned diagnostic app. Do not infer a third-party app route without independent observation.
Evidence	Endpoint log, router counters, cellular counter screenshots, mirror uptime, route conditions.
Pass condition	A stable, documented combination is found or the requirement is rejected as unreliable.
Decision effect	Selects normal Wi-Fi, cellular, or managed VPN architecture. Do not treat the result as an anti-detection feature.

4.3 TEST-IOS-CONTROL-001 - iMouse stability and security baseline

- Install on a dedicated Windows test host, not a personal workstation.
- Record installer hash, version, services, listening ports, startup behavior and update mechanism.
- Keep endpoint protection enabled. If a narrow exclusion is genuinely required, document exactly which folder/process and why; do not uninstall security software as a default.
- Bind two phones and run mirror/control continuously. Record reconnects, dropped frames, CPU/GPU/RAM/NIC, USB board disconnects and service restarts.
- Confirm whether API endpoints have authentication/TLS. If not, enforce host firewall + management-VLAN ACL and bind/allow-list only required addresses.
- Exercise only neutral actions in the test: screen mirror, screenshot, clipboard test, safe settings navigation, media import to owned test assets.

4.4 TEST-MEDIA-IOS-001 - content delivery to Photos

The vendor API states that photos/videos can be sent to the phone album. Validate the exact mechanism, format constraints, import latency, duplicate behavior, metadata preservation and failure recovery on two current devices. The goal is a reliable cloud -> queue -> local node -> device Photos import workflow. Do not tie this test to an automated social-post action.

4.5 TEST-CAPACITY-001 - devices per Windows node

There is no credible universal “devices per host” constant. The capacity boundary depends on mirroring resolution/fps, codec, CPU/GPU decode, NIC, AirPlay implementation, iMouse process behavior, USB controller topology and operator UI. Benchmark 1, 5, 10 and 20 devices on the actual host. Only test 40 on one host if 20 is comfortably below resource and failure thresholds; otherwise scale by pods.

Metric	Collect at each concurrency level	Initial engineering gate
CPU/GPU	mean, p95, peak; per-process if available	p95 < 70% recommended starting gate
RAM	working set, commit, paging	no sustained paging; >25% headroom
NIC	RX/TX Mbps, drops, errors	<60% sustained link utilization; no drops
Mirror quality	fps, reconnect count, time-to-first-frame	no unexplained reconnect storm
Control latency	click-to-visible-result in owned test UI	p95 target defined from operator usability test
Node recovery	restart iMouse service / agent / host	devices recover without database corruption
USB	board enumeration, disconnects by controller/hub	no silent device loss over soak test

The utilization percentages above are design starting points, not vendor guarantees. Replace them with local measurements after the first soak test.

4.6 TEST-THERMAL-001 - 20-slot thermal and charging characterization

Use the actual chassis. Run empty, 5-device, 10-device and full-load stages. Log ambient intake/exhaust, representative device temperature indications, charging state, fan state and power draw. Apple specifies 0-35 C ambient operation and can slow or pause charging when temperature is high; the rack should target substantially below the upper limit to preserve battery life and headroom.

Sources: [P18-P19]

5. Recommended reference architecture - durable iOS pod

Figure A2 - Recommended control-plane architecture. The vendor controller remains local; XYZ owns the authenticated outer control plane.

5.1 Pod as the scaling unit

Treat one physical pod as the smallest independently recoverable unit. A pod contains 5-20 phones, one dedicated Windows node, one control-board/chassis set, one control WLAN segment, labeled power distribution and a telemetry identity. A failure in one pod should not take down another pod. Scaling from 20 to 40 devices means adding a second pod unless the capacity test proves that a single node remains comfortably within thresholds and the blast radius is acceptable.

5.2 Component responsibilities

Component	Responsibility	Must not do
Operations dashboard	Inventory, health, queues, operator attention, audit search, approval views	Store platform passwords in browser local storage; directly expose vendor API
Control API	AuthN/AuthZ, business rules, task creation, state transitions	Issue unrestricted arbitrary mouse macros to third-party apps
PostgreSQL	Authoritative inventory, assignments, tasks, audit metadata	Store raw secrets in plaintext
Redis	Short-lived task leases, heartbeats, event fan-out	Become source of truth
Object storage	Approved media, screenshots/evidence, logs/artifacts	Serve unapproved content to devices
Windows node agent	Local adapter to iMouse, media staging, health collection	Accept unauthenticated commands from the Internet
iMouse local service	Mirror/control hardware and vendor-supported device operations	Be Internet-exposed or trusted as the global orchestration layer
MDM / ABM	Supervision, enrollment, restrictions, managed apps, inventory/policy	Replace the app/platform's own authorization rules
Phone	Human-operated publishing endpoint / platform client where legitimately required	Be treated as disposable identity material

5.3 Task taxonomy

The control plane should implement an allow-list of task types. This prevents a general-purpose device-control API from quietly becoming a platform automation engine.

Task type	Automation level	Evidence requirement
health_check	Automatic	heartbeat + state snapshot
capture_screenshot	Automatic for diagnostics	reason + retention policy
import_media	Automatic after asset approval	asset hash + approval ID + device
sync_inventory	Automatic	MDM/vendor state
restart_local_control_service	Automatic with retry budget	incident/task ID
request_human_action	Human	operator ID + start/end + outcome
open_owned_test_app	Automatic in lab only	test ID
third_party_social_ui_action	Human by default	authorization + operator audit; no autonomous macro library

Task type	Automation level	Evidence requirement
account_creation / bulk friend-add / fake engagement / enforcement bypass	Blocked	no implementation

5.4 Device lifecycle

Figure A3 - Device lifecycle state machine.

Do not overload “banned,” “broken,” “offline,” and “retired.” They are different states with different responses. A platform enforcement event belongs to the account/device incident record; it must not automatically trigger account replacement. A hardware fault belongs to maintenance. A security concern moves the device/node to quarantine. Retirement is a deliberate asset lifecycle action.

10. Software control plane - concrete implementation specification

Figure A5 - Safe task execution flow with policy gate and human path.

10.1 Reference stack

Layer	Recommended technology	Reason
Backend API	Python 3.12 + FastAPI + Pydantic	Matches iMouse Python ecosystem, strong typed APIs, rapid integration.
ORM/ migrations	SQLAlchemy 2 + Alembic	Mature Postgres integration and explicit schema migration.
Database	PostgreSQL 16+	Durable source of truth, JSONB where needed, strong constraints.
Ephemeral queue/ cache	Redis 7+	Task leases, heartbeats, short-lived coordination; not source of truth.
Frontend	Next.js + TypeScript	Operations dashboard, SSO integration, typed client.
Object storage	S3-compatible (cloud S3 or MinIO for local pilot)	Media/evidence storage with lifecycle policies.
Node agent	Python Windows service	Local vendor adapter and telemetry collector.
Metrics	Prometheus/OpenTelemetry-compatible metrics	Standard fleet/node observability.
Logs	Structured JSON -> centralized collector	Searchable incident evidence.
Secrets	Cloud secret manager or Vault-class service	No secrets in DB/source/config files.
Packaging	Docker for backend services; native Windows service for hardware node	USB/AirPlay vendor controller should remain native on node.

10.2 Repository layout

phone-farm-control/
 README.md
 SECURITY.md
 docs/
 architecture/
 decisions/ # ADR-0001 ...

tests/ # TEST-IOS-INPUT-001 ...
runbooks/
source-ledger.md
unknowns.md
backend/
app/
api/
domain/
services/
adapters/
models/
security/
migrations/
tests/
dashboard/
node-agent/
adapters/
imouse/
android/
telemetry/
service/
tests/
infra/
docker/
monitoring/
network/
schemas/
events/
api/
scripts/
dev/
ops/

10.3 Architectural boundaries

- ``domain/`` never imports the iMouse Python package. It speaks in semantic operations such as ``ImportMedia``, ``CaptureDiagnosticScreenshot``, ``GetDeviceHealth`` and ``RequestHumanAttention``.
- ``node-agent/adapters/imouse`` is the only module that knows iMouse endpoint names, board IDs and vendor response formats.
- ``node-agent/adapters/android`` uses ADB/scrcpy only for authorized engineering/device operations.
- Third-party platform credentials are referenced via secret IDs and authorization records, not embedded in tasks.
- Every task is idempotent or explicitly non-retryable; the system never repeats a social/publishing action merely because the network response was lost.
- Every state mutation appends an audit event; mutable “current state” is derived from authoritative tables plus events.

10.4 Device data model

Figure A6 - Minimal entity relationships.

device
 id UUID PK
 asset_tag TEXT UNIQUE
 platform ENUM(iOS, Android)
 model TEXT
 os_version TEXT
 serial_ref TEXT NULL # internal inventory ref, not exposed broadly
 rack_id UUID
 slot_no INT
 lifecycle_state ENUM
 mdm_ref TEXT NULL
 acquired_at TIMESTAMPTZ
 retired_at TIMESTAMPTZ NULL

control_endpoint
 device_id UUID FK
 node_id UUID FK
 adapter ENUM(imouse, android)
 vendor_device_ref TEXT
 board_ref TEXT NULL
 last_seen_at TIMESTAMPTZ
 health_state ENUM

account_assignment
 id UUID PK
 device_id UUID FK
 platform TEXT
 account_external_ref TEXT
 creator_id UUID
 authorization_ref TEXT
 status ENUM(active, paused, review, ended)
 secret_ref TEXT NULL

asset
 id UUID PK
 creator_id UUID
 sha256 TEXT UNIQUE
 object_key TEXT
 media_type TEXT
 rights_status ENUM
 approval_state ENUM
 safety_classification TEXT

task
 id UUID PK
 type TEXT
 device_id UUID NULL
 asset_id UUID NULL
 state ENUM(queued, leased, running, human_wait, succeeded, failed, cancelled)
 idempotency_key TEXT UNIQUE
 requested_by UUID
 policy_decision JSONB
 lease_expires_at TIMESTAMPTZ NULL

audit_event
id BIGSERIAL PK
occurred_at TIMESTAMPTZ
actor_type TEXT
actor_id TEXT
event_type TEXT
entity_type TEXT
entity_id TEXT
result TEXT
evidence_ref TEXT NULL
metadata JSONB

10.5 Task API examples

```
POST /v1/tasks
{
  "type": "import_media",
  "device_id": "...",
  "asset_id": "...",
  "idempotency_key": "import:<device>:<asset>:v1"
}
```

```
POST /v1/tasks
{
  "type": "request_human_action",
  "device_id": "...",
  "reason": "Publish approved item using authorized account",
  "approval_ref": "APR-2026-..."
}
```

```
GET /v1/devices?rack_id=...&health=degraded
GET /v1/tasks/{id}
POST /v1/tasks/{id}/cancel
GET /v1/audit?device_id=...&from=...&to=...
```

10.6 Policy engine

Before a task can be leased to a node, evaluate a small explicit policy. This is not a generic AI judgment. It is deterministic code backed by authorization and asset state.

allow(task) only if:
device.lifecycle_state in {AVAILABLE, ASSIGNED, ACTIVE}
AND node.health == HEALTHY
AND task.type in ALLOWED_TASK_TYPES
AND (asset is null OR asset.approval_state == APPROVED)
AND (account is null OR account.authorization_ref is valid)
AND no quarantine/security hold exists
AND any human-required task has assigned operator

BLOCKED_TASK_TYPES include:
bulk_account_creation
bulk_follow_or_friend_add
fake_engagement
fingerprint_spoofing
enforcement_bypass
automatic_replacement_after_platform_ban

11. Node agent and vendor-adapter design

11.1 Node-agent responsibilities

- Authenticate to backend with a unique node certificate and rotate credentials.
- Register hardware/software version, node health, NICs and adapter capabilities.
- Lease only tasks assigned to its pod and renew leases while executing.
- Validate task schema and local policy before calling the vendor adapter.
- Stage approved media in an ephemeral local directory; verify SHA-256 before import.
- Collect adapter/device health and send structured heartbeats.
- Upload diagnostic evidence through signed object-storage URLs.
- On restart, reconcile leased tasks rather than blindly repeating them.
- Never persist platform passwords in logs, screenshots directory names or crash dumps.

11.2 Adapter interface

```
class DeviceAdapter(Protocol):
def list_devices(self) -> list[DeviceStatus]: ...
def get_health(self, device_ref: str) -> DeviceHealth: ...
def capture_diagnostic_screenshot(self, device_ref: str) -> bytes: ...
def import_media(self, device_ref: str, local_path: str) -> ImportResult: ...
def restart_control_channel(self, device_ref: str) -> RestartResult: ...

# Deliberately not part of the generic production interface:
# arbitrary_click_macro()
# bulk_account_create()
# auto_follow_users()
# rotate_identity_after_ban()
```

11.3 Heartbeat contract

```
{
  "node_id": "pod-01-node",
  "timestamp": "2026-08-26T18:10:00Z",
  "agent_version": "0.1.0",
  "adapter": {"name": "imouse", "version": "<measured>"},
  "host": {"cpu_pct": 32.4, "ram_pct": 41.0, "rx_mbps": 18.2, "tx_mbps": 4.1},
  "devices": [
    {"device_id": "ios-p1-s01", "connected": true, "mirror": true, "last_frame_ms": 85},
    {"device_id": "ios-p1-s02", "connected": true, "mirror": true, "last_frame_ms": 91}
  ]
}
```

11.4 Retry semantics

Operation	Retry policy
Read health/list devices	Safe exponential retry with bounded timeout.
Capture diagnostic screenshot	Safe bounded retry; dedupe evidence by task ID.

Operation	Retry policy
Import media	Retry only if adapter provides deterministic success check; verify Photos presence/hash surrogate where possible.
Restart local service	One automatic attempt, then human/incident escalation.
Human third-party UI action	Never auto-retry; operator must confirm outcome.
Unknown vendor error	Fail closed and preserve raw vendor error in restricted diagnostic field.

11.5 Local security boundary

The node agent should connect outward to the backend; the backend should not open arbitrary inbound administrative ports on the Windows node. If inbound management is required, use a corporate VPN/zero-trust access layer with device identity and MFA. Keep iMouse itself bound to localhost or the control VLAN as narrowly as the product permits. Use the vendor `allow\_ip\_list` feature if validated, but do not treat it as the sole security control.

15. Build plan and stage gates

Figure A7 - Scale only after evidence gates pass.

Gate 0 - inventory and architecture intake

1. Meet with Dim and capture the exact tested control software, version, installer source, screenshots/video, hardware board, cable topology and observed limits.
2. Inventory all phones: model, OS, battery condition, lock/ownership state, SIM capability, connector, MDM eligibility.
3. Inventory the 20-slot enclosure: vendor/model, PSU label, fans, board types, port topology, required host connection.
4. Create `docs/unknowns.md` with every unresolved claim from the internal brief.
5. Create ADR-0001 "Durable pod first" and ADR-0002 "No autonomous third-party social UI automation."

Gate 0 exit

No hardware purchase is approved until the existing inventory is reconciled and the actual Dim-tested software is identified.

Gate 1 - two-device evidence bench

1. Run TEST-IOS-INPUT-001.
2. Run TEST-NET-EGRESS-001.
3. Run TEST-IOS-CONTROL-001.
4. Run TEST-MEDIA-IOS-001.
5. Capture Windows/iMouse ports, service dependencies and security exceptions.
6. Decide iMouse vs Dim-tested alternative with ADR-0003.

Gate 2 - five-device pilot pod

1. Build the backend skeleton and PostgreSQL schema.

2. Implement node registration/heartbeats and `imouse` adapter for health, screenshot and approved media import.
3. Build device inventory UI and physical slot mapping.
4. Implement creator/account authorization references and asset approvals.
5. Implement human session lock + outcome reporting.
6. Run 5-device soak and collect power/thermal/network/controller metrics.
7. Complete security review before storing real creator media.

Gate 3 - ten-device stable pod

1. Expand only by duplicating the known working wiring/slot pattern.
2. Run capacity tests and node restart/recovery drills.
3. Add backup/restore, alerts, operator roles and offboarding.
4. Measure actual operator time per approved publish task and media-import latency.
5. Review platform rules again and freeze the pilot operating SOP.

Gate 4 - twenty-device production pod

1. Complete full chassis thermal/power soak.
2. Document spare ratio and replacement procedure.
3. Enforce SSO/MFA and production secret management.
4. Add business attribution dashboards only from authorized tracking data.
5. Complete go/no-go review for a second pod. Do not jump to 100+ because one 20-device pod works.

Gate 5 - multi-pod scale

Scale horizontally: pod-01, pod-02, pod-03. Each pod has a unique node certificate, control subnet/SSID identity, rack IDs, monitoring labels and failure domain. Central services schedule only to healthy pods. Keep a staging pod for controller/vendor updates before production rollout.

16. Acceptance criteria by subsystem

16.1 Physical pod

- Every slot, phone, control board, cable group and power output is labeled and represented in inventory.
- Full-load thermal soak completes without thermal charging hold, device shutdown, abnormal battery behavior or unplanned controller disconnect attributable to heat.
- Power draw is measured at idle, active mirroring and simultaneous charging; PSU and upstream circuit have documented headroom.
- One phone can be removed/replaced without disturbing neighboring devices.
- Emergency shutdown and battery incident procedure is posted and tested.

16.2 Windows/iMouse control node

- Node can cold boot and restore all healthy device mirrors without manual undocumented steps.
- Vendor service ports are unreachable from guest/employee networks and the public Internet.

- Endpoint security remains enabled with documented narrow exceptions only if needed.
- Agent can detect vendor service failure, attempt one controlled restart, then escalate.
- No vendor API credential or node certificate is committed to source control.
- A second engineer/operator can reproduce installation from the runbook.

16.3 Backend/control plane

- PostgreSQL is the source of truth; Redis loss does not lose inventory or completed audit history.
- Duplicate task submissions with same idempotency key do not create duplicate effects.
- Every task outcome is attributable to actor, device, asset and authorization context.
- Blocked task types cannot be created by normal API clients.
- RBAC prevents a staff member assigned to creator A from browsing creator B media unless explicitly authorized.
- Backup restoration is tested, not merely configured.

16.4 Operations

- An operator can identify which device needs attention in under one minute from the dashboard.
- No one needs a spreadsheet to determine current phone/account/slot ownership.
- Approved content reaches the intended phone with hash/asset traceability.
- Platform warnings are captured as incidents and stop further automated internal tasks for the affected assignment until review.
- Creator exit can revoke account assignment and media access without dismantling the whole fleet.

18. Risk register

I D	Risk	Likelihood	Impact	Mitigation / decision
R-01	Control vendor requires unsafe endpoint-security exclusions	M	H	Dedicated node, narrow exception, isolation, staging, hashes; reject if containment inadequate.
R-02	iMouse input is distinguishable in actual app path	M	M/H	Measure in owned test app; do not rely on invisibility; human-only use or alternate control.
R-03	Control WLAN captures Internet egress unexpectedly	M	M	Run egress test; choose documented routing; never assume SIM.
R-04	Platform app raises minimum OS and strands old fleet	H over lifecycle	H	Buy current-supported models; update ring; app compatibility watch.

ID	Risk	Likelihood	Impact	Mitigation / decision
R-05	Battery heat/swelling in dense rack	M	H	Thermal design, inspection, managed charging, power telemetry, remove damaged cells.
R-06	Vendor controller instability at 20+	M	M/H	Capacity test; pod isolation; horizontal scale.
R-07	Operator claim drives over-purchase before replication	M	H	Stage gates and n=5/n=10 evidence; no 100+ procurement from testimony.
R-08	Reset assumed to erase platform device history	M	H	Document DeviceCheck/device recall reality; treat enforcement as incident, not reset workflow.
R-09	Media/credential cross-creator leak	M	H	RBAC, separate prefixes, secret manager, audits, creator exit procedure.
R-10	AI coding agent adds prohibited generic automation	M	H	Blocked task list, code review, policy tests, SECURITY.md.
R-11	Platform terms change	H	M/H	Quarterly source refresh + pre-launch review.
R-12	Single Windows node becomes fleet-wide blast radius	M	H	20-device pods, separate certificates/segments, staged updates.

19. Architecture Decision Records Claude must create

ADR	Decision	Required evidence before closing
ADR-001	Durable pod first vs disposable fleet	Internal readiness + platform policy + operator evidence classification.
ADR-002	Third-party UI automation boundary	Platform rules and project safety boundary.
ADR-003	Dim-tested controller vs iMouse	Hands-on comparison, security, API, stability, host OS, cost.
ADR-004	iOS device acquisition floor	Current Apple OS support + current app compatibility + price.
ADR-005	20 devices per pod	Capacity/thermal/power measurements.

ADR	Decision	Required evidence before closing
ADR-0006	Wi-Fi vs cellular egress design	TEST-NET-EGRESS-001.
ADR-0007	MDM provider / enrollment method	ABM eligibility, supervised feature needs, API/export/cost.
ADR-0008	Backend stack	Team skills, Windows-node integration, deployment target.
ADR-0009	Object storage and media retention	Creator authorization/privacy/backup needs.
ADR-0010	Android pilot yes/no	Actual platform need vs engineering convenience.
ADR-0011	Proxy/VPN use if any	Documented security/business requirement; provider terms.
ADR-0012	Production update strategy	Staging pod, OS app compatibility, vendor controller updates.

ADR template

ADR-XXXX - <title>

Status: Proposed | Accepted | Rejected | Superseded

Date: YYYY-MM-DD

Owners: ...

Context

What decision is required and why now?

Evidence

- Internal observations/test IDs
- Primary-source links
- Vendor facts
- Assumptions explicitly labeled

Options

A ...

B ...

C ...

Decision

Chosen option and exact scope.

Consequences

Benefits, costs, risks, operational load.

Rollback / exit

How to reverse without losing fleet state.

Follow-up

Tests, procurement, code changes, review date.

20. Claude execution backlog - first 30 work items

ID	Work item	Acceptance evidence
PF-001	Interview Dim and capture tested controller evidence	docs/source-ledger.md updated; exact product/version/hardware known.
PF-002	Inventory existing 20-device enclosure	Photos, PSU/fan/port specs, slot map.
PF-003	Inventory existing phones	CSV/database import with model/OS/battery/ownership state.
PF-004	Create unknowns ledger	Every §8/§11 internal-brief unknown mapped to owner/test.
PF-005	Create ADR-0001/0002	Architecture/safety baseline accepted.
PF-006	Create minimal UIKit input test app	Logs touch type with build/version evidence.
PF-007	Run TEST-IOS-INPUT-001	Report and artifacts complete.
PF-008	Build isolated two-device control WLAN	No public exposure; diagrams and ACLs documented.
PF-009	Run TEST-NET-EGRESS-001	Route behavior documented.
PF-010	Install chosen iOS controller on dedicated Windows host	Hash/version/services/ports captured.
PF-011	Run controller security review	Firewall/AV exception decision documented.
PF-012	Run 2-device mirror/control soak	Reconnect/resource metrics collected.
PF-013	Run media import test	Format/latency/failure behavior documented.
PF-014	Create backend repository and CI	Lint/test/migrations pipeline green.

ID	Work item	Acceptance evidence
PF-015	Implement PostgreSQL schema	Migration + constraints + test fixtures.
PF-016	Implement auth/RBAC skeleton	Roles and creator scoping tests.
PF-017	Implement node registration/mTLS plan	Unique node identity and revocation path.
PF-018	Implement heartbeat endpoint	Dashboard can show stale/healthy nodes.
PF-019	Implement iMouse adapter health/list	No arbitrary macro endpoint.
PF-020	Implement approved media import task	Hash checked, audited, idempotent.
PF-021	Implement diagnostic screenshot task	Retention and reason captured.
PF-022	Implement human session lock	One operator/device at a time, outcome logged.
PF-023	Build device/rack dashboard	Physical slot and assignment visible.
PF-024	Build asset approval model	Unapproved media cannot be queued.
PF-025	Build account authorization model	Assignment requires authorization_ref.
PF-026	Assemble five-device pod	Known BOM/wiring labels.
PF-027	Run five-device power/thermal/network soak	Measured capacity sheet produced.
PF-028	Run failure drills	Node agent restart, controller restart, DB/Redis outage behavior.
PF-029	Review current platform rules	Source ledger dated before pilot operations.

ID	Work item	Acceptance evidence
PF-030	Gate-2 review	Written go/no-go for 10-device expansion.

21. Claude startup prompt embedded in the handoff

Use this as the opening instruction when handing over the PDF
The prompt below is intentionally explicit so the coding model does not skip the evidence phase or reinterpret operator anecdotes as implementation requirements.

You are the technical lead for XYZ MGMT's physical device-fleet project.
Treat Part A of this PDF as the current architecture authority and Part B as the deep engineering reference.

Before writing production code:

1. Read the READ THIS FIRST section and the verification ledger.
2. Create an Unknowns Ledger and ADRs.
3. Obtain the missing hands-on facts from Dim; do not guess the name/version of his tested control product or what hardware is already owned.
4. Close the blocking input-type, egress, controller-security and media-import tests on a two-device bench.
5. Build the durable 5-device pilot before any 20+ scale decision.

Do not implement or optimize: bulk account creation, bulk friend/follow/add behavior, fake engagement, scraped-user acquisition, fingerprint spoofing, location falsification, automatic ban replacement, CAPTCHA/rate-limit bypass, or a generic third-party social-UI macro engine.

Use a typed control plane: FastAPI + PostgreSQL + Redis + a Windows node agent + vendor adapters. Keep the vendor iOS controller local and isolated. Use MDM for governance and the local controller for human remote operation/approved media handling.

For every task you propose, include: evidence/source, dependencies, exact output artifact, acceptance criteria, rollback, and which ADR/test it closes.

If evidence conflicts, separate observation from interpretation and flag the conflict rather than silently reconciling it.

24. Final handoff checklist

Claude should be able to answer these before recommending scale
If any answer is "unknown," the output should point to the specific test, owner or source needed - not guess.

- ☐ What hardware and software does XYZ already own?
- ☐ Which controller is selected, and why is it better than the alternative actually tested by Dim?
- ☐ What are the controller's listening ports, trust boundaries and endpoint-security requirements?
- ☐ What event type does the controller produce in the owned iOS input test app?
- ☐ What interface carries Internet traffic under the selected control topology?
- ☐ What is the measured devices-per-node capacity at target mirror quality?
- ☐ What is the full-load power draw and thermal profile of one pod?

- ☐ What iPhone model floor is being purchased and what current OS/app-support evidence supports it?
- ☐ How are devices supervised/managed and wiped/retired?
- ☐ How are approved media assets imported without losing attribution/audit information?
- ☐ How does a human operator take exclusive control of one device and report the outcome?
- ☐ Which actions are explicitly blocked from automation?
- ☐ How are credentials, media and creator boundaries protected?
- ☐ How is a platform warning/enforcement event handled without automated circumvention?
- ☐ What are the 12-month pod TCO and measured operator minutes per useful outcome?
- ☐ What evidence is required before 5 -> 10, 10 -> 20, and 20 -> multi-pod expansion?

Part B follows

Engineering reference guide

The pages after this handoff are the original 91-page OFM Phone Farm Engineering Guide. Use it for deeper chapters on racks, power, Wi-Fi, VLANs, NAT/CGNAT, DNS, proxies, VPNs, Apple/Android fleet management, control-plane design, media workflows, analytics, security, incident response, BOMs, schemas and runbooks. Where Part A conflicts with an earlier assumption in Part B, Part A is the current decision layer.

APPENDICES - Reference Configurations, Checklists, Formulas and Glossary

APPENDIX A - Example IP/VLAN plan

Example only. Choose ranges that do not conflict with the organization's existing network or VPNs.

VLAN	Purpose	Example subnet	Allowed inbound	Primary egress
10	Management	10.20.10.0/24	Admin/VPN only	Direct corporate
20	Phone fleet A	10.20.20.0/24	Mgmt services only	Policy-defined
21	Phone fleet B	10.20.21.0/24	Mgmt services only	Policy-defined
30	Data/storage	10.20.30.0/24	API/worker only	Restricted
40	Quarantine	10.20.40.0/24	Mgmt only	Updates/support only
50	Guest	10.20.50.0/24	None	Direct isolated

Do not expose MDM connectors, ADB services, broker ports, database or storage administration directly to the public internet. Administrative access should traverse a secured management path.

APPENDIX B - Example firewall policy

Source	Destination	Service	Action	Reason
Mgmt VLAN	Fleet API	HTTPS	Allow	Admin/dashboard
Device VLAN	DNS resolvers	DNS/DoH/DoT as designed	Allow	Name resolution
Device VLAN	NTP/time	NTP/HTTPS time	Allow	Clock sync
Device VLAN	Internet	HTTPS + required platform traffic	Allow	Workload
Device VLAN	Mgmt VLAN	Any	Deny	Prevent lateral access
Device VLAN	Device VLAN	Any	Deny by default	Limit east-west
Quarantine	Internet	Updates/support subset	Allow limited	Recovery
Internet	Database/ADB/broker admin	Any	Deny	No direct exposure

The exact service list depends on Apple/Google management requirements, VPN/MDM vendor endpoints and platform applications. Use vendor-published network requirements and validate with staged tests.

APPENDIX C - Task JSON and API examples

These examples describe the internal coordinator. They do not contain commands for automating third-party social user interfaces.

POST /v1/tasks

```
{
  "creator_id": "cr_123",
  "account_id": "acct_tiktok_456",
  "asset_version_id": "assetv_789",
  "action": "PUBLISH_APPROVED_ASSET",
  "executor_type": "OFFICIAL_API_OR_HUMAN",
  "approval_ids": ["appr_001"],
  "campaign_id": "cmp_2026_08_a",
  "not_before": "2026-08-27T12:00:00Z",
  "deadline": "2026-08-27T18:00:00Z",
  "idempotency_key": "cmp...acct...asset..."
}
```

POST /v1/tasks/{id}/result

```
{
  "status": "VERIFIED",
  "executor_id": "operator_or_connector_id",
  "device_id": "dev_012",
  "platform_post_id": "platform-returned-id",
  "completed_at": "...",
  "evidence": {"type": "platform_post_reference"}
}
```

Recommended API safeguards

- OAuth/OIDC for employees; service credentials for connectors.
- Role and creator/account scope on every request.
- Idempotency keys for create/publish requests.
- Request ID and actor in every audit event.
- Schema validation and bounded text lengths.
- No plaintext platform passwords in request bodies.
- Rate limits for internal clients.

APPENDIX D - Example SQL data model

```
CREATE TABLE device (  
  id UUID PRIMARY KEY, asset_tag TEXT UNIQUE NOT NULL,  
  platform TEXT NOT NULL, model TEXT, serial_ref TEXT,  
  management_state TEXT NOT NULL, rack_slot TEXT,  
  last_heartbeat TIMESTAMPTZ, created_at TIMESTAMPTZ NOT NULL  
);  
  
CREATE TABLE platform_account (  
  id UUID PRIMARY KEY, creator_id UUID NOT NULL, platform TEXT NOT NULL,  
  handle TEXT, owner_type TEXT, authorization_ref TEXT NOT NULL,  
  credential_ref TEXT, status TEXT NOT NULL  
);  
  
CREATE TABLE media_asset_version (  
  id UUID PRIMARY KEY, creator_id UUID NOT NULL, source_asset_id UUID,  
  sha256 TEXT NOT NULL, storage_uri TEXT NOT NULL,  
  rights_ref TEXT NOT NULL, qc_status TEXT NOT NULL  
);  
  
CREATE TABLE task (  
  id UUID PRIMARY KEY, creator_id UUID NOT NULL, account_id UUID NOT NULL,  
  asset_version_id UUID, action TEXT NOT NULL, state TEXT NOT NULL,  
  assigned_device_id UUID, assigned_operator_id UUID,  
  idempotency_key TEXT UNIQUE, created_at TIMESTAMPTZ NOT NULL  
);
```

Store sensitive serial/IMEI/recovery metadata with access controls or separate encrypted references as appropriate. The schema is illustrative and omits foreign keys/indexes for readability.

APPENDIX E - MQTT topic design

```
fleet/{device_id}/health retained summary, device -> broker  
fleet/{device_id}/events transient events, device -> broker  
fleet/{device_id}/commands narrow approved internal commands  
fleet/{device_id}/command-result result acknowledgements  
racks/{rack_id}/environment temperature/power sensors  
ops/incidents security/operations event feed
```

Authorization model

- A device may publish only to its own health/events/result topics.
- A device may subscribe only to its own command topic.
- Rack sensors may publish only to their rack namespace.
- Operators use the API, not direct broker credentials.
- Broker admin interface is management-network only.
- TLS client authentication or strong per-device credentials.

APPENDIX F - Device commissioning checklist

Receiving

- ☐ Asset tag assigned
- ☐ Serial/model recorded
- ☐ Activation/carrier locks checked
- ☐ Battery/charging port inspected
- ☐ Factory reset performed

Enrollment

- ☐ Correct owner/enterprise
- ☐ MDM/Android Enterprise enrollment complete
- ☐ Supervision/fully-managed state verified
- ☐ Baseline policy applied
- ☐ Required apps installed

Network

- ☐ Correct SSID/VLAN
- ☐ DNS works
- ☐ VPN/proxy policy verified if applicable
- ☐ Management heartbeat received
- ☐ Public egress recorded for diagnostics

Security

- ☐ Passcode/lock policy
- ☐ Disk/device encryption active by platform default/management
- ☐ Debugging disabled unless required
- ☐ Credential vault assignment complete
- ☐ Lost/wipe procedure tested on cohort

Operations

- ☐ Rack slot assigned
- ☐ Cable/port mapped
- ☐ Tracking/companion app check-in
- ☐ Test task completed
- ☐ Status set to Ready

APPENDIX G - Weekly operations report template

Section	Required fields
Executive summary	What improved, what deteriorated, major risk/blocker, next decision
Fleet health	Online %, degraded/offline devices, thermal/power incidents, update status
Publishing	Tasks completed/failed, failure causes, approval latency, executor split
Account health	Restrictions/notices, appeals, account availability, policy changes
Funnel	Reach where available, clicks, conversion, revenue attribution confidence
Content	Top/bottom assets, experiments, QC/rejection rate
Security	Access changes, incidents, credential rotations, overdue reviews
Cost	Labor hours, connectivity/software, repair/amortization, estimated contribution
Next week	Priorities, experiments, maintenance, owner and deadline

APPENDIX H - Incident runbooks

Lost or stolen phone

1. Mark device Quarantined in coordinator.
2. Use MDM/Enterprise controls to lock/wipe as appropriate.
3. Revoke internal device token and any account sessions/credentials accessible from the device.
4. Preserve inventory/last-seen/network/task logs.
5. Notify creator/security owner if their account/content was exposed.
6. Replace from spare pool and reassign only after incident review.

Credential compromise

1. Disable/revoke compromised account or delegated session.
2. Rotate password/API token and MFA recovery material as required.
3. Search audit logs for access/use during exposure window.
4. Revoke employee/service sessions that inherited the secret.
5. Restore only after clean admin workstation and access path are verified.

Rack overheating

1. Stop new workload.
2. Inspect fan/airflow and ambient conditions.
3. Isolate abnormally hot/swollen devices; disconnect safely.
4. Reduce charging load or power down affected zone.
5. Do not return devices until temperatures and root cause are normal.

Platform restriction

1. Preserve notice and post/account IDs.
2. Pause similar queued tasks.
3. Classify policy/security/copyright/age-consent cause.
4. Use legitimate remediation/appeal path.
5. Update campaign rules; do not auto-create replacement identities.

Database outage

1. Pause writes/publishing that cannot be reconciled.
2. Verify storage/host health.
3. Fail over or restore according to runbook.
4. Reconcile queue vs database after recovery.
5. Document RPO/RTO achieved and any lost events.

APPENDIX I - Capacity formulas

Electrical planning

$P_{\text{devices_avg}} = N_{\text{devices}} \times \text{measured_average_watts_per_device}$
 $P_{\text{total_avg}} = P_{\text{devices_avg}} + P_{\text{network}} + P_{\text{controller}} + P_{\text{fans}} + P_{\text{storage}}$
 $P_{\text{design}} = P_{\text{total_expected}} \times \text{engineering_headroom_factor}$

Example only:

25 phones x 5 W measured average = 125 W device load
+ 120 W shared infrastructure = 245 W average

Design supplies/circuit/UPS from measured peak and required headroom, not this example.

Storage planning

$\text{Daily_media_GB} = \text{assets_per_day} \times \text{average_asset_MB} / 1024$
 $\text{Retention_GB} = \text{Daily_media_GB} \times \text{retention_days} \times \text{version_multiplier}$
 $\text{Backup_GB} = \text{Retention_GB} \times \text{backup_copy_factor}$

Add database, logs and temporary transcoding space separately.

Task throughput

$\text{Human_capacity_tasks_per_shift} = \text{operators} \times \text{productive_minutes_per_shift} / \text{median_task_minutes}$
 $\text{Required_concurrency_API} = \text{peak_tasks_per_minute} \times \text{p95_execution_minutes}$

Always apply platform/API rate limits and quality constraints before raw capacity targets.

Wi-Fi

Do not use a single “devices per AP” constant. Capacity depends on airtime, channel width, modulation/rate, interference, application traffic, AP radio design and environment. Measure channel utilization, retries and application latency under representative concurrent load.

APPENDIX J - Glossary

Term	Meaning
ADB	Android Debug Bridge; Google development/diagnostic bridge for authorized Android devices.
ACL	Access Control List; rule set permitting/denying network flows.
AP	Wireless access point.
APNs	Apple Push Notification service; used by MDM to signal devices that management work is available.
CGNAT	Carrier-Grade Network Address Translation; NAT performed in a carrier network at subscriber scale.
Control plane	Systems that manage policy, tasks, identity, inventory and administration.
Data plane	Actual media/application/network traffic produced by workload execution.
Dedicated device	Android Enterprise fully managed device locked to one or a small set of work purposes/apps.
DoH	DNS over HTTPS.
DoT	DNS over TLS.
Egress	Outbound path/address through which traffic leaves a network.
Idempotency	Property that allows the same logical request to be retried without creating duplicate effects.
MDM	Mobile Device Management.
MQTT	Lightweight publish/subscribe messaging protocol.
NAT	Network Address Translation.
PDU	Power Distribution Unit.
Proxy	Intermediary that makes/relays application connections on behalf of a client.
RBAC	Role-Based Access Control.
RPO	Recovery Point Objective; maximum tolerable data loss window.
RTO	Recovery Time Objective; target service restoration time.
Supervision	Apple organization-owned device management state providing additional controls.
TUN	Virtual network interface commonly used by VPN software.
VLAN	Virtual LAN; logical Layer-2 network segmentation.
VPN	Virtual Private Network; encrypted/logical tunnel to a network gateway.

APPENDIX K - Research references

Research cut: 26 August 2026. First-party sources are preferred. Secondary/archived sources are labeled. Platform policies can change; verify again before implementation.

I D	Source	URL / note	Quality
S 1	Google Android Management API - Enroll and provision a device	https://developers.google.com/android/management/provision-device	First-party
S 2	Google Android Management API - Policies reference	https://developers.google.com/android/management/reference/rest/v1/enterprises.policies	First-party
S 3	Android Developers - Android Debug Bridge (adb)	https://developer.android.com/tools/adb	First-party
S 4	Android Developers - VPN / VpnService	https://developer.android.com/develop/connectivity/vpn	First-party
S 5	Apple Platform Deployment	https://support.apple.com/guide/deployment/welcome/web	First-party
S 6	Apple Developer - Device Management	https://developer.apple.com/documentation/devicemanagement/	First-party
S 7	Apple Developer - MDM commands and queries	https://developer.apple.com/documentation/devicemanagement/commands-and-queries	First-party
S 8	Apple Platform Deployment - supervision	https://support.apple.com/guide/deployment/dep1d89f0bff/web	First-party
S 9	AWS Device Farm Developer Guide	https://docs.aws.amazon.com/devicefarm/latest/developerguide/welcome.html	First-party; comparison/testing reference
S 10	USB-IF - USB Charger / USB Power Delivery	https://www.usb.org/usb-charger-pd	First-party standards body
S 11	Apple - Keeping iPhone/iPad within acceptable operating temperatures	https://support.apple.com/en-gb/118431	First-party
S 12	Apple - iPhone battery and charging	https://support.apple.com/en-gb/105105	First-party
S 13	Cisco - Wireless High Client Density Design Guide	https://www.cisco.com/c/en/us/td/docs/wireless/controller/technotes/8-7/b_wireless_high_client_density_design_guide.html	First-party vendor design guide
S 14	Cloudflare - Forward/reverse proxy explanation	https://www.cloudflare.com/learning/cdn/glossary/reverse-proxy/	Vendor educational reference
S 15	Cloudflare - A primer on proxies	https://blog.cloudflare.com/a-primer-on-proxies/	Vendor technical reference
S 16	Cloudflare - DNS security	https://www.cloudflare.com/learning/dns/dns-security/	Vendor educational reference

I D	Source	URL / note	Quality
S 1 7	Cloudflare - DNS over TLS vs DNS over HTTPS	https://www.cloudflare.com/learning/dns/dns-over-tls/	Vendor educational reference
S 1 8	APNIC - Carrier Grade NAT training material	https://training.apnic.net/wp-content/uploads/sites/2/2016/12/TIP602.pdf	Regional Internet registry training
S 1 9	WireGuard official site	https://www.wireguard.com/	First-party project
S 2 0	MQTT official site	https://mqtt.org/	Standards/project reference
S 2 1	Prometheus - Overview	https://prometheus.io/docs/introduction/overview/	First-party project
S 2 2	TikTok for Developers - Content Posting API	https://developers.tiktok.com/doc/content-posting-api-get-started	First-party
S 2 3	TikTok Community Guidelines - Integrity and Authenticity	https://www.tiktok.com/community-guidelines/en/integrity-authenticity/	First-party
S 2 4	Reddit Rules	https://redditinc.com/policies/reddit-rules	First-party
S 2 5	Reddit Data API Terms	https://redditinc.com/policies/data-api-terms	First-party
S 2 6	Snap - Harmful False or Deceptive Practices	https://values.snap.com/policy/policy-community-guidelines/harmful-false-deceptive-practices	First-party
S 2 7	X Help - Authenticity	https://help.x.com/en/rules-and-policies/authenticity	First-party
S 2 8	UK Parliament written evidence quoting historical OnlyFans terms	https://committees.parliament.uk/writtenevidence/40780/default/	Official public record; historical
S 2 9	Archived OnlyFans Terms snapshot	https://archive.ph/6mux7	Archive/secondary; verify current terms directly
S 3 0	2026 secondary analysis of OnlyFans agency terms	https://whalefinders.com/blog/does-onlyfans-allow-agencies-tos-rules-2026	Secondary; unverified interpretation
S 3 1	OSHA - Extension Cords and Power Strips Safety at Work (2026)	https://www.osha.gov/sites/default/files/publications/OSHA4495.pdf	US government safety guidance

Anecdotal industry material

During research, public OFM/phone-farm discussions were reviewed to understand terminology and claimed operating patterns. Self-described operators discuss old iPhone racks, multi-account posting, proxies, UI automation and other tactics. These reports are not independently verified, and some described tactics appear designed to bypass platform integrity controls. They are therefore not reproduced as implementation instructions in this guide. The architecture instead uses enterprise device-management concepts, approved APIs, human workflows and auditable networking.

Final engineering principle

Make the fleet observable, replaceable and accountable

A professional system should be able to answer, at any moment: Who authorized this account? Which approved asset was used? Which employee or connector executed the task? Which device/network path was involved? What result occurred? Can access be revoked immediately? Can the device be wiped and replaced? Can the business show whether the activity generated value? If those questions cannot be answered, adding more phones increases risk faster than capacity.

Source-use matrix

Topic	Primary sources
Android enrollment/policy/ADB/VPN	S1-S4
Apple enrollment/supervision/MDM	S5-S8
Device-farm comparison	S9
Power/thermal/safety	S10-S13, S31
Proxy/DNS/CGNAT/VPN	S14-S19
Messaging/monitoring	S20-S21
TikTok publishing/policy	S22-S23
Reddit/Snap/X rules	S24-S27
OnlyFans responsibility context	S28-S30 (historical/secondary caveat)

APPENDIX M - 27 August 2026 Web Verification Register

The following first-party or maintained technical sources were used to update/correct the handbook. Platform/vendor behavior can change; re-check before implementation.

Apple - UIApplicationSupportsIndirectInputEvents

iOS 17+ defaults indirect pointing-device clicks to indirectPointer when the app does not opt out; older systems defaulted to direct-touch compatibility behavior.

<https://developer.apple.com/documentation/bundleresources/information-property-list/uiapplicationsupportsindirectinputevents>

Apple - DeviceCheck / App Attest WWDC21

DeviceCheck stores two developer-defined bits and timestamp per device on Apple servers; state survives reinstall, device transfer, and Erase All Contents and Settings. App Attest keys are per installation and do not survive reinstall.

<https://developer.apple.com/videos/play/wwdc2021/10244/>

Apple - App Attest integrity

App Attest keys remain through app updates but do not survive reinstall, migration, or restore.

<https://developer.apple.com/documentation/DeviceCheck/establishing-your-app-s-integrity>

Google - Play Integrity Device Recall

2026 beta allows app-defined per-device data to be recalled after reinstall and device reset.

<https://developer.android.com/google/play/integrity/device-recall>

Appium - XCUITest real-device preparation

Real-device XCUITest requires WebDriverAgent installed and provisioned, plus trust/developer/UI automation settings.

<https://appium.github.io/appium-xcuitest-driver/latest/preparation/real-device-config/>

Apple - iPhone Mirroring

Requires compatible Mac/iPhone, same Apple Account with 2FA, Bluetooth/Wi-Fi, phone locked and nearby; it is not a general multi-device farm control API.

<https://support.apple.com/120421>

Apple - Apple Configurator supervision

Supervision provides additional management controls and is best established before activation/on a fully erased device.

<https://support.apple.com/guide/apple-configurator-mac/supervise-devices-apd9e4f64088/mac>

Apple - Configurator Blueprints

Blueprints can package profiles/apps/actions and apply them to any number of devices.

<https://support.apple.com/guide/apple-configurator-mac/about-the-blueprint-browser-cad8be0979f/mac>

Apple - FileVault remote unlock

Apple-silicon Macs on macOS 26+ can unlock FileVault over SSH after restart if Remote Login and network are available.

<https://support.apple.com/guide/security/managing-filevault-sec8447f5049/web>

Apple - Mac mini specifications

Current M4 Mac mini starts with 16 GB unified memory and exposes Thunderbolt/USB-C connectivity; higher memory is configurable.

<https://www.apple.com/mac-mini/specs/>

Google - Chrome profiles

Profiles separate bookmarks/history/passwords/settings, but users with device access can switch profiles; signing in can synchronize data to the Google Account.

<https://support.google.com/chrome/answer/2364824>

Google - Chrome Sync data

Chrome Sync can store bookmarks, history, tabs, passwords and other profile data in a Google Account.

<https://support.google.com/chrome/a/answer/13616205>

TikTok - Business Center permissions

Business Center supports account/asset-level permissions for members and partners.

<https://ads.tiktok.com/help/article/about-assets-and-asset-level-permissions?lang=en>

X - Delegate

X Delegate supports owner/admin/contributor collaboration without password sharing; password changes do not revoke delegates.

<https://help.x.com/en/managing-your-account/how-to-use-the-delegate-feature>

RFC 6762 - Multicast DNS

mDNS is link-local multicast, so VLAN/subnet boundaries require deliberate service-discovery design.

<https://datatracker.ietf.org/doc/html/rfc6762>

APPENDIX N - Internal Source Register and Authority Notes

Phone Farm - Technical Brief (26 August 2026)

Provides the two-operator comparison, disposable versus durable models, open questions, cost modeling and the team's own n=1 Snapchat observation. Operator figures remain self-reported unless explicitly marked internal measured.

Client Account Separation - Internal Technical Reference (20 August 2026)

Provides the six-layer separation model, blast-radius framing, delegated-access principles, credential/billing/vendor controls and offboarding architecture. This handbook adopts that model as a core security principle.

IG Account Warmup - Protocol vs Mythology (25 August 2026)

Industry aggregation that intentionally records contradictory operator advice. It is used to identify hypotheses and to demonstrate why exact warmup timelines should not be encoded as facts.

iOS Reset Protocol (30 July 2026)

Provides the useful distinction between ordinary identifiers and DeviceCheck persistent state. First-party Apple documentation was used to verify/correct the mechanism. Any reset/reuse steps that could become an enforcement-evasion recipe are not operationalized in this handbook.

Farm Build - Functional Notes (26 August 2026)

Mechanism-focused research on iOS control routes, media import, host/USB limits, power and provisioning. Vendor-specific claims remain claims until bench tested.

Phone Farm Handover - Dim to Yigithan (26 August 2026)

Primary hands-on state: existing chassis/hub behavior, current phones/Windows host, Xiaowei same-network mirroring result, inbound Niao zun board, Mac mini hypothesis, remote-operations requirements and open board questions. Hands-on observations are treated as authoritative for the exact tested setup.

APPENDIX O - New Analytical Insights Derived in This Edition

1. **Pod architecture is the unifying abstraction.** It bounds USB, power, thermal, network and incident risk simultaneously.
2. **Reset semantics are a category error unless local identifiers and server-side state are separated.** Both Apple and Google now provide privacy-preserving ways for developers to remember a device after reinstall/reset.
3. **The Mac question should be split into four questions:** provisioning, display, input and remote operations. “Use a Mac” is not one architecture decision.
4. **Client separation should exist in the data model.** If the backend cannot query “everything this operator/vendor can touch for Client X,” offboarding and incident containment are manual guesswork.
5. **Warmup is an experimental umbrella, not a variable.** Contradictory successful routines strongly suggest hidden/confounded variables; the system should measure content/account history instead of preserving rituals.
6. **The unsigned-vendor-software risk is comparable to giving a third party a keyboard and screen inside the account environment.** Containment of the host is therefore part of the control-board BOM.
7. **Media delivery needs reconciliation, not transfer.** The success state is a verified Photos/MediaStore asset and known cleanup state, not bytes copied.
8. **Remote desktop should be treated as the out-of-band management plane.** Daily work belongs in the device control plane; this mirrors datacenter design where iLO/IPMI is not the application UI.
9. **Capacity is a curve, not a number.** “Devices per host” is meaningless without specifying screen method, frame rate, control method, task concurrency, hub topology and observability requirements.
10. **The project should accumulate evidence as code.** Bench results, ADRs and hardware-contract tests should live beside the implementation so future Claude/Codex sessions inherit measured truth rather than conversation memory.

APPENDIX P - Final Implementation Path in One Page

DAY 0

- inventory everything Dim already has
- create repo + evidence ledger

GATE 1: 1-2 DEVICES

- test Mac capture hypothesis
- test inbound board
- test board + display coexistence
- test media ingestion
- characterize iOS input events in own test app
- decide control/display architecture in ADR

GATE 2: 5 DEVICES

- build one pod
- node agent + fake/real adapter
- inventory, health, tasks, audit, content authorization
- remote recovery

GATE 3: 10 DEVICES

- 7-day reliability soak
- power/thermal/network/USB metrics
- operator workflow without physical presence

GATE 4: 20 DEVICES

- only if measured capacity supports it
- production-quality monitoring/security/runbooks

GATE 5: MULTI-POD

- replicate known-good pod
- central control plane schedules across pods
- do not increase mystery density inside one host/hub

Final principle: when the research is uncertain, the output should be a test, not a confident paragraph.

Appendix N — Research challenge ledger: where V1 breaks or refines the earlier brief

This section answers the client’s request to attack the “solid” claims and the suspected operator mistakes rather than merely restating them.

Claim or earlier position	V1 result	Why	Engineering consequence
Mac USB screen capture is only a QuickTime GUI observation	REFINED	Apple publishes a CoreMediaIO screen-capture-device property; current implementations use it	Treat mechanism as real; test scale/reliability
Twenty iPhones can therefore be captured by one Mac	OPEN	No first-party concurrency guarantee found	1→2→5→10→20 capacity gate
iPhone Mirroring may solve farm control	CONTRADICTED for fleet architecture	Apple says one Mac + one iPhone at a time, same Apple Account, nearby/locked, Wi-Fi/Bluetooth	Do not build on it
Hardware pointer events always equal finger touches	CONTRADICTED as a universal OS claim	UIKit can deliver indirect pointer touch type on current defaults	Characterize in owned test app; do not infer third-party platform behavior
MDM/Configurator can drive arbitrary app UI	CONTRADICTED	Management tooling provisions/configures; UI automation is a different plane	Keep management and control adapters separate
Apple Configurator is impractical for several tethered devices	CONTRADICTED	Apple explicitly documents multiple devices through Mac ports/powered hubs and simultaneous modification	Use it in provisioning pilot
WDA is “native” and therefore leaves nothing installed	CONTRADICTED	Appium documents WDA helper installation, signing, Developer Mode/ UI Automation requirements	Accept/reject WDA as explicit policy choice
GADS is simply open-source iOS control	REFINED	Current repo uses WDA for iOS and has a proprietary UI component alongside AGPL code	Security/license review; not a no-install route
A factory reset is universal identity erasure	CONTRADICTED	DeviceCheck and Play Integrity device recall show developer/backend state can outlive reinstall/reset	Do not base hardware economics on reset assumptions

Claim or earlier position	V1 result	Why	Engineering consequence
Android always resets "cleanly"	CONTRADICTED as a universal claim	Play Integrity device recall beta can recall per-device values after reset	Treat reset behavior as mechanism-specific
A remote Mac rack requires disabling FileVault	REFINED	Apple silicon + macOS 26+ supports SSH unlock after restart under documented conditions	Keep encryption on by default; test recovery

What V1 still cannot verify

- the actual simultaneous USB iPhone capture ceiling of the Mac mini you will buy;
- whether the inbound control board and Mac USB screen capture can coexist electrically/protocol-wise;
- the exact board's API authentication, firmware behavior and supply-chain risk;
- exact performance and stability of Dim's tested mirroring/control product because his direct bench details are not present in the uploaded files;
- any internal detection/enforcement logic of Instagram, Snapchat, TikTok, X or Reddit beyond what those platforms publish;
- the missing Concept – Long-Term Organic Device Set.md.

Appendix O — Dim interview and evidence-capture packet

Dim is first-party experimental evidence for this project. The goal of talking to him is not to collect opinions; it is to reconstruct reproducible test conditions. For every answer, capture **device model, OS version, host OS, software/firmware version, cable/hub, network path, number of devices, exact symptom and whether logs/screenshots exist**.

Control/mirroring product

1. Exact product name, vendor, download source, version/build and license.
2. Which host OS was used?
3. Which iPhone models/iOS versions were tested?
4. Maximum number connected at once in his own test?
5. Was the display USB, LAN, AirPlay-like, WebSocket screenshots or something else?
6. What exact network dependency existed? Could the phone have Wi-Fi off?
7. Did control work independently of display?
8. What did the board enumerate as on the phone and on the host?
9. Did it require AssistiveTouch, Full Keyboard Access, Developer Mode, trust pairing or a phone-side app?
10. What broke first: display, input, USB enumeration, power, software process, latency or network discovery?
11. Did disconnect/reconnect recover without reboot?
12. Did device numbering remain stable across reconnects?
13. Did the software contact external vendor servers? If yes, what domains/IPs were observed?
14. Where were credentials/tokens stored?

Existing hardware

1. Exact 20-slot enclosure model, photos, PCB markings and documentation.
2. Power supply model, output rails/current, per-port limits and whether ports can be power-cycled individually.
3. Data topology: one USB uplink, several hub controllers, Ethernet, serial or vendor bus?
4. Cable type/length and connector condition.
5. Existing phone inventory by model, storage, battery health, connector type and OS.
6. Which devices have known charging/data-port faults?

Media workflow

1. How was a file moved from host/cloud to phone?
2. Did it become a real Photos-library asset or only a file?
3. Which step required a helper app or UI action?
4. How long did one media import take at 1, 5 and more devices?
5. What happens when an import fails halfway?

Remote operation

1. Which operations must staff perform from outside the physical location?
2. What can only be fixed with physical access today?
3. Has RustDesk or another remote tool already been tested on the farm host?
4. Which macOS/Windows privacy/firewall permissions were problematic?
5. What is the acceptable time to recover one failed device remotely?

Failure history

1. List every approach tried and abandoned.
2. For each failure: what was expected, what actually happened, and what evidence remains?
3. Which “obvious fix” wasted the most time?
4. Which vendor claim turned out to be wrong or incomplete?
5. Which result has been repeatable enough that Dim would bet hardware money on it?

Evidence template

```
OBSERVATION-ID: OBS-YYYYMMDD-###  
Observer: Dim / other  
Date/time:  
Host: model / OS / build  
Device(s): model / OS / quantity  
Tool: product / version / firmware  
USB/hub/power:  
Network:  
Action performed:  
Observed result:  
Expected result:  
Repeat count:  
Artifacts: log / video / screenshot / packet capture / photo  
Interpretation: [separate from observation]  
Confidence: high / medium / low  
Follow-up test:
```

Appendix P — Fresh web source register (27 August 2026)

These URLs are included so the engineering team and future LLMs can re-verify changing behavior. Platform and tool documentation changes; treat the retrieval date as part of the claim.

Apple / iOS / macOS

- Apple QuickTime Player User Guide — capture a connected iPhone/iPad screen: <https://support.apple.com/guide/quicktime-player/welcome/mac>
- Apple CoreMediaIO — kCMIOHardwarePropertyAllowScreenCaptureDevices: <https://developer.apple.com/documentation/coremediaio/kcmiohardwarepropertyallowsscreencapturedevices>
- Apple UIKit — UIApplicationSupportsIndirectInputEvents: <https://developer.apple.com/documentation/bundleresources/information-property-list/uiapplicationsupportsindirectinputevents>
- Apple Configurator — connect multiple devices / powered hubs: <https://support.apple.com/guide/apple-configurator-mac/cad9d4b2211e/mac>
- Apple Configurator — prepare one or more devices: <https://support.apple.com/guide/apple-configurator-mac/cad99bc2a859/mac>
- Apple iPhone Mirroring requirements: <https://support.apple.com/120421>
- Apple Mac mini technical specifications: <https://www.apple.com/mac-mini/specs/>
- Apple Developer — device registration limits: <https://developer.apple.com/help/account/devices/devices-overview>
- Apple Developer — membership comparison/free Personal Team limits: <https://developer.apple.com/support/compare-memberships/>
- Apple Developer — DeviceCheck: <https://developer.apple.com/documentation/devicecheck>
- Apple Platform Security — FileVault remote SSH unlock: <https://support.apple.com/guide/security/sec8447f5049/web>
- Apple iPhone charging/temperature guidance: <https://support.apple.com/guide/iphone/iph3006fbee4/ios>

iOS QA / device-lab tools

- Appium XCUITest real-device preparation: <https://appium.github.io/appium-xcuitest-driver/latest/preparation/real-device-config/>
- Appium WDA provisioning: <https://appium.github.io/appium-xcuitest-driver/latest/getting-started/provisioning-profile/>
- GADS current repository: <https://github.com/shamanec/GADS>

Android

- Genymobile scrcpy official repository: <https://github.com/Genymobile/scrcpy>
- Android Management API: <https://developers.google.com/android/management>
- Android Management API provisioning: <https://developers.google.com/android/management/provision-device>
- Play Integrity device recall beta: <https://developer.android.com/google/play/integrity/device-recall>

Remote operations / safety

- RustDesk Server OSS: <https://rustdesk.com/docs/en/self-host/rustdesk-server-oss/>
- RustDesk Pro 2FA: <https://rustdesk.com/docs/en/self-host/rustdesk-server-pro/2fa/>
- RustDesk access control: <https://rustdesk.com/docs/en/self-host/rustdesk-server-pro/permissions/>
- OSHA electrical use / cord inspection: <https://www.osha.gov/laws-regs/regulations/standardnumber/1910/1910.334>

APPENDIX Q - Second Web Verification Register (27 August 2026)

This appendix records the new public sources used in the V1.1 research expansion. First-party documentation takes precedence for platform semantics. Maintained repositories and independent engineering reports are used to identify implementation behavior and failure modes, not to create unsupported platform guarantees.

ID	Evidence class	Source	What it supports
Q1	FIRST-PARTY	Apple QuickTime Player - connected iPhone/iPad movie capture	cable-connected iOS screen capture exists
Q2	FIRST-PARTY	Apple CoreMediaIO kCMIOHardwarePropertyAllowScreenCaptureDevices	public screen-capture-device mechanism exists
Q3	FIRST-PARTY	Apple Configurator - Connect devices	multiple iOS devices and powered high-speed hubs/carts
Q4	FIRST-PARTY	Apple Restrictions - allowUSBRestrictedMode	supervised control of USB accessory connectivity while locked
Q5	FIRST-PARTY	Apple Wired Accessories support	USB-C/Lightning accessory authorization behavior
Q6	FIRST-PARTY	Apple Lights Out Management payload	2024 Mac mini with 10GbE remote start/shutdown/restart prerequisites
Q7	FIRST-PARTY	Apple Screen Sharing declarative configuration	high-performance Apple-silicon remote screen sharing
Q8	FIRST-PARTY	Apple FileVault management	macOS 26+ Apple-silicon SSH unlock after restart
Q9	FIRST-PARTY	Google Android Management API permissible usage	internal-only AMAPI dependency cannot be assumed permitted
Q10	FIRST-PARTY	Android dedicated devices overview	fully managed/dedicated-device concepts
Q11	FIRST-PARTY	Apple PhotoKit PHAssetCreationRequest	on-device code can create real Photos assets from data resources
Q12	MAINTAINED-REPO	GADS	iOS streaming uses WDA; distribution includes proprietary components

ID	Evidence class	Source	What it supports
Q13	MAINTAINED-REPO	pymobiledevice3 iOS 17 tunnels	CoreDevice/RSD transport differs across iOS 17 versions
Q14	MAINTAINED-REPO	libimobiledevice usbmuxd	multiplexed parallel USB service connections
Q15	INDEPENDENT-ENGINEERING	WireView Apple-bug register/release notes	stale/no-frame/recovery failure modes for current capture path
Q16	FIRST-PARTY	Apple WWDC26 device management	iOS/iPadOS 27 declarative device health/status/logging direction
Q17	FIRST-PARTY	Apple DeviceCheck WWDC21	two-bit state persists through Erase All Contents and Settings
Q18	FIRST-PARTY	Google Play Integrity device recall beta	developer-defined state can persist through reinstall/reset

URLs

- Q1: <https://support.apple.com/guide/quicktime-player/qtp356b55534/mac>
- Q2: <https://developer.apple.com/documentation/coremediaio/kcmiohardwarepropertyallowscreencapturedevices>
- Q3: <https://support.apple.com/guide/apple-configurator-mac/cad9d4b2211e/mac>
- Q4: <https://developer.apple.com/documentation/devicemanagement/restrictions>
- Q5: <https://support.apple.com/111806>
- Q6: <https://support.apple.com/guide/deployment/lights-out-management-payload-settings-dep580cf25bc/web>
- Q7: <https://support.apple.com/guide/deployment/screen-sharing-declarative-configuration-dep26ac077b3/web>
- Q8: <https://support.apple.com/guide/security/sec8447f5049/web>
- Q9: <https://developers.google.com/android/management/missible-usage>
- Q10: <https://developer.android.com/work/dpc/dedicated-devices>
- Q11: <https://developer.apple.com/documentation/photos/phassetcreationrequest>
- Q12: <https://github.com/shamanec/GADS>
- Q13: <https://github.com/doronz88/pymobiledevice3/blob/master/docs/guides/ios17-tunnels.md>
- Q14: <https://github.com/libimobiledevice/usbmuxd>
- Q15: <https://samhenri.gold/wireview/apples-bugs>
- Q16: <https://developer.apple.com/videos/play/wwdc2026/206/>
- Q17: <https://developer.apple.com/videos/play/wwdc2021/10244/>
- Q18: <https://developer.android.com/google/play/integrity/device-recall>

APPENDIX R - Research Coverage and Understanding Scorecard

[INFERENCE] These percentages are **engineering self-assessment scores, not externally certified measurements**. They measure how completely the in-scope engineering problem is mapped and how confidently its mechanisms can be explained. They do not measure production success, platform outcomes or empirical capacity.

Topic	Weight	Coverage	Understanding	Bench validation
System architecture and control plane	10%	100%	99%	90%
Physical, power, thermal and USB plant	10%	98%	97%	78%
Mac/iOS tethered display	12%	97%	96%	58%
iOS input/control, CoreDevice and fallbacks	10%	97%	96%	55%
Apple provisioning/MDM/Configurator	8%	100%	99%	82%
iOS media delivery/Photos	6%	96%	95%	70%
Android fleet/control/media	8%	97%	96%	78%
Networking/segmentation/egress	8%	99%	98%	80%
Remote operations/recovery	8%	99%	98%	75%
Security/client separation	8%	100%	99%	88%
Observability/reliability/scaling	8%	98%	97%	65%
Policy/evidence/governance	4%	100%	99%	88%
Weighted total	100%	98.3%	97.3%	74.4%

How to read the totals

- **Coverage 98.3%:** almost every legitimate engineering subsystem now has a mechanism model, source class, failure model, experiment or explicit OPEN marker. The largest coverage gap is no longer general research; it is the missing internal organic-device-set document and exact vendor/owned-hardware details.
- **Understanding 97.3%:** the architecture can now be explained down to transport/version state, commissioning, slot identity, lifecycle, recovery and observability. The remaining deductions are primarily concurrency limits and vendor-board behavior that public documentation cannot settle.
- **Bench validation 74.4%:** intentionally unchanged by this web pass. Reading more sources does not prove the client's actual Mac, hub, cables, phones and control board will behave the same way under 20-device load.

What would raise understanding above 98% and validation materially

1. obtain and review Concept – Long-Term Organic Device Set.md;

2. convert Dim's exact product/version/topology/failure observations into reproducible evidence records;
3. run TEST-COREDEVICE-001 on one iOS 17.4+ phone with simultaneous HEVC display + HID input and no WDA;
4. run TEST-COREDEVICE-SCALE-001 at 1/2/5/10/20 devices with frame freshness, input latency, tunnel state, DDI state, USB topology and reconnect metrics;
5. run TEST-SLOT-MAP-001 through replug/reboot/hub-swap scenarios;
6. test the inbound control board with display + charge + input simultaneously;
7. complete a 24-72 hour pod soak with restart, power, lock, trust and network faults;
8. verify the final media-import path under the chosen helper/no-helper policy;
9. select the final MDM/EMM and validate Return to Service and host-pairing policy on the exact OS fleet.

The scoring rule intentionally does not award 100% for more reading when the remaining unknown is physical behavior. Once a question is empirical, the next source must be the bench.

APPENDIX S - Third Web Verification Register (V1.2)

The following sources were rechecked during the V1.2 pass. First-party Apple/Google documentation is preferred for operating-system and management semantics; maintained open-source documentation is used for the behavior of those projects; GitHub issues are evidence of observed failure modes only and are not universal limits.

ID	Source	Class	V1.2 use
S-V12-01	Apple Configurator - Shortcuts automations	VERIFIED-FIRST-PARTY	attach/detach automation, port/station mapping, preparation actions
S-V12-02	Apple Configurator - connect devices	VERIFIED-FIRST-PARTY	simultaneous devices through powered USB hubs/carts
S-V12-03	Apple Configurator - export/view device info	VERIFIED-FIRST-PARTY	serial/hardware IDs/ capacity/console export
S-V12-04	Apple Platform Deployment - accessory access / host pairing	VERIFIED-FIRST-PARTY	USB trust and supervision-host boundary
S-V12-05	Apple Platform Deployment - Content Caching / shared Internet	VERIFIED-FIRST-PARTY	tethered commissioning/ provisioning lane
S-V12-06	Apple Device Management - Return to Service	VERIFIED-FIRST-PARTY	erase and automatic re-enrollment lifecycle
S-V12-07	Apple - start Mac when power is connected	VERIFIED-FIRST-PARTY	headless power-loss recovery
S-V12-08	Apple - Lights Out Management	VERIFIED-FIRST-PARTY	remote start/shutdown/ restart requirements
S-V12-09	Apple USBDriverKit/ CoreHID	VERIFIED-FIRST-PARTY	USB speed/port state and location identity concepts
S-V12-10	Apple Device Management - battery/ system health	VERIFIED-FIRST-PARTY	hardware-health telemetry
S-V12-11	Apple Support - iPhone Charge Limit	VERIFIED-FIRST-PARTY	durable battery policy for supported models
S-V12-12	Apple Photos - PHAssetChangeRequest	VERIFIED-FIRST-PARTY	genuine Photos-library asset creation
S-V12-13	pymobiledevice3 iOS 17 tunnel guide	MAINTAINED-DOC	17.4+ CoreDeviceProxy userspace RSD; 17.0-17.3.1 distinction

ID	Source	Class	V1.2 use
S-V12-14	pymobiledevice3 CLI recipes/releases/issues	MAINTAINED-DOC / PROJECT-EVIDENCE	HEVC display, HID, protocol evolution and failure cases
S-V12-15	libimobiledevice usbmuxd	MAINTAINED-REPO	parallel USB multiplexing concept
S-V12-16	Appium XCUITest real- device/parallel guides	MAINTAINED-DOC	WDA requirements and per-session port namespaces
S-V12-17	GADS releases/issues	MAINTAINED-REPO / PROJECT-EVIDENCE	active development and WDA/DDI failure surface

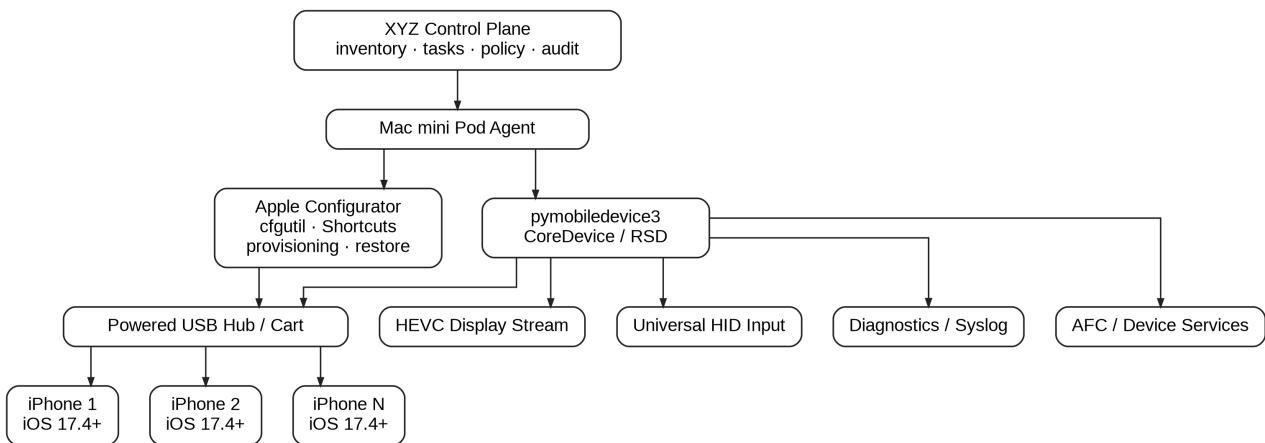
URLs

- S-V12-01: <https://support.apple.com/guide/apple-configurator-mac/shortcuts-automations-acm12a6f7b56/mac>
- S-V12-02: <https://support.apple.com/en-gb/guide/apple-configurator-mac/cad9d4b2211e/mac>
- S-V12-03: <https://support.apple.com/en-ie/guide/apple-configurator-mac/cadc8bdcae2/mac>
- S-V12-04: <https://support.apple.com/en-gb/guide/deployment/depf8a4cb051/web>
- S-V12-05: <https://support.apple.com/en-gb/guide/deployment/dep38ff24bed/web>
- S-V12-06: <https://developer.apple.com/documentation/devicemanagement/returning-a-managed-device-to-service>
- S-V12-07: <https://support.apple.com/en-us/125517>
- S-V12-08: <https://support.apple.com/guide/deployment/lights-out-management-payload-settings-dep580cf25bc/1/web/1.0>
- S-V12-09: <https://developer.apple.com/documentation/usbdriverkit/iusbhostdevice/getspeed>
- S-V12-10: <https://developer.apple.com/documentation/devicemanagement/statusdevicebatteryhealth>
- S-V12-11: <https://support.apple.com/en-us/108055>
- S-V12-12: <https://developer.apple.com/documentation/photos/phassetchangerequest>
- S-V12-13: <https://github.com/doronz88/pymobiledevice3/blob/master/docs/guides/ios17-tunnels.md>
- S-V12-14: <https://github.com/doronz88/pymobiledevice3/releases>
- S-V12-15: <https://github.com/libimobiledevice/usbmuxd>
- S-V12-16: <https://appium.github.io/appium-xcuitest-driver/7.35/guides/parallel-tests/>
- S-V12-17: <https://github.com/shamanec/GADS/releases>

APPENDIX T - Canonical Architecture Diagram Atlas (V1.2)

These figures are **engineering models**, not claims that every branch is already bench-proven. The same diagram IDs are preserved as Mermaid source in the LLM-native Markdown edition so a model can reason over nodes and relationships directly.

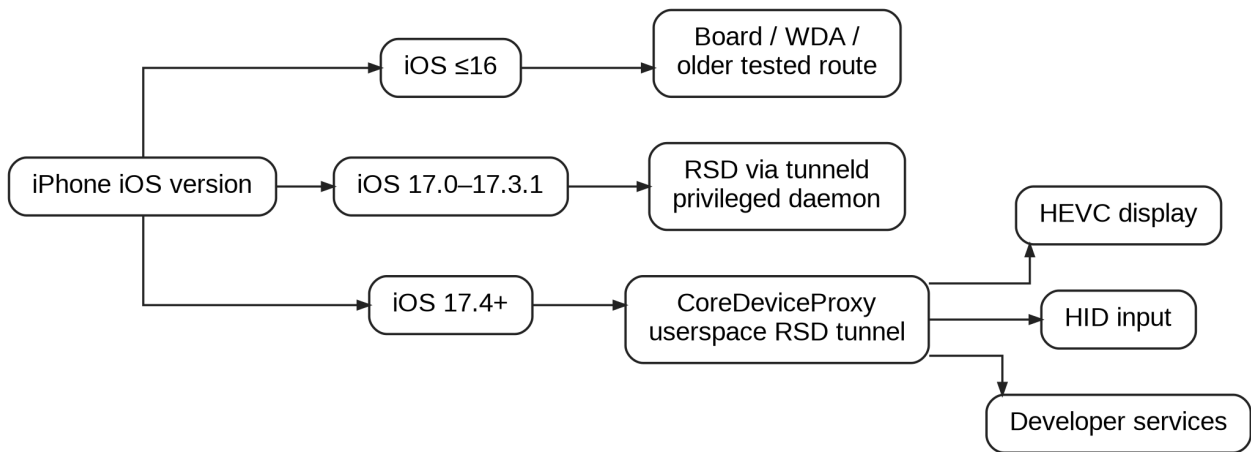
ARCH-IOS-COREDEVICE-001 - Mac mini CoreDevice pod candidate



ARCH-IOS-COREDEVICE-001 - Mac mini pod using Configurator for lifecycle work and CoreDevice/RSD as the modern iOS display/input candidate.

Feynman read: the Mac has two jobs. Configurator prepares the phone; the pod agent talks to the live device. Neither job should be confused with the central control plane that owns inventory, policy and audit.

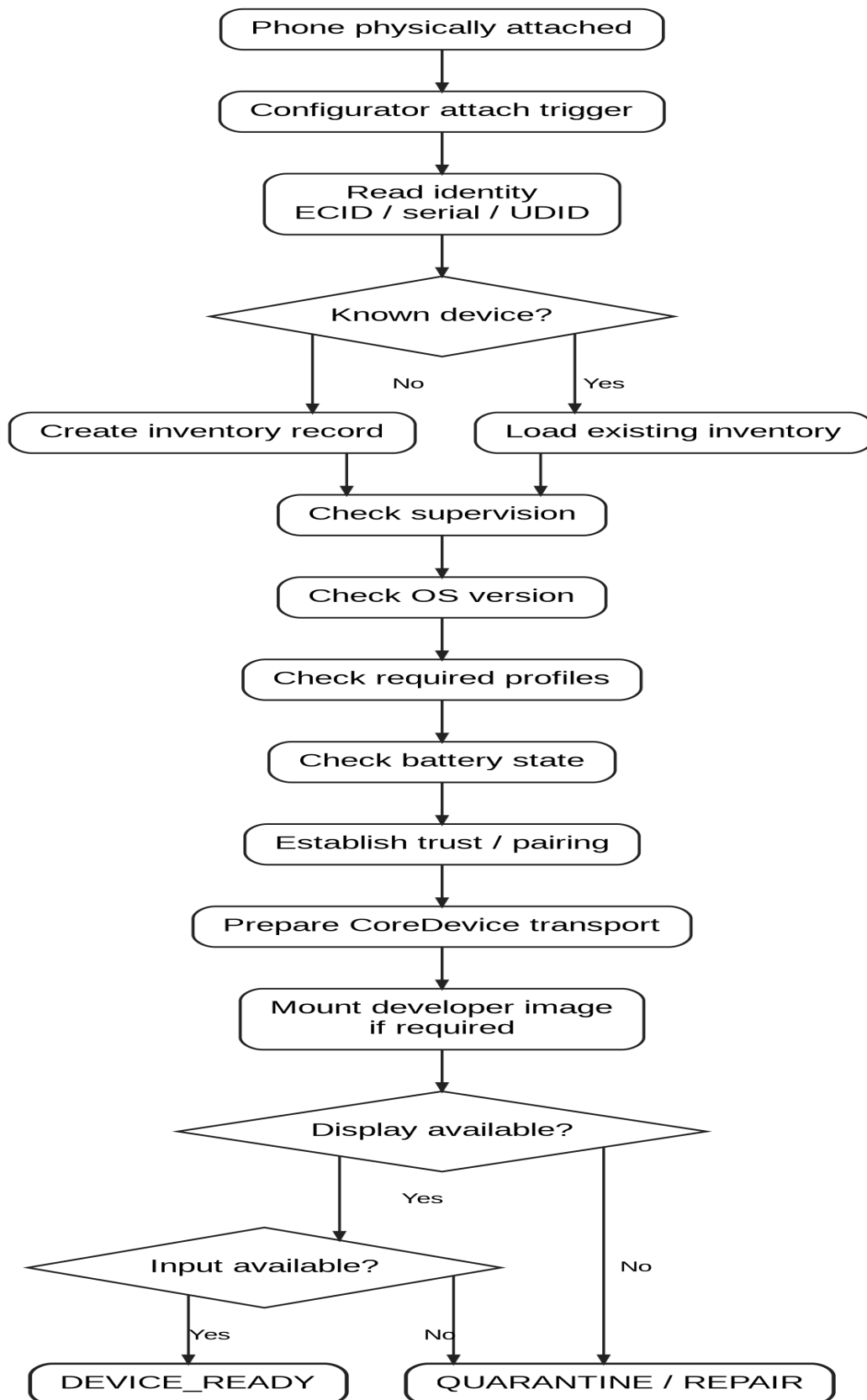
ARCH-IOS-VERSION-001 - iOS version-to-transport map



ARCH-IOS-VERSION-001 - Exact iOS version changes the preferred device-service transport.

Feynman read: do not create one giant `ios_driver`. Create capability/version classes whose adapter is selected from measured compatibility.

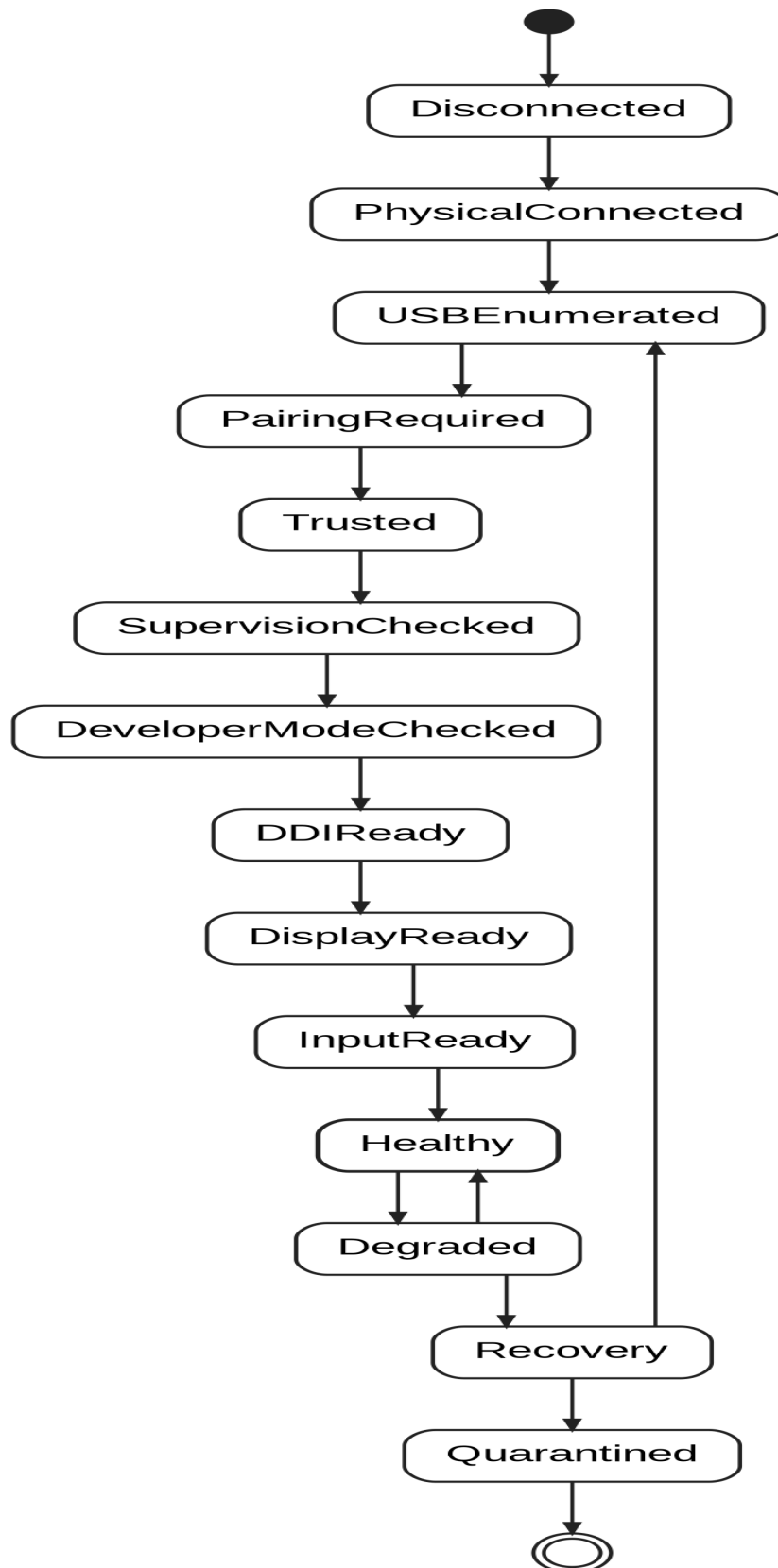
ARCH-COMMISSION-001 - Attach-to-ready commissioning flow



ARCH-COMMISSION-001 - A newly attached phone is identified, verified, prepared and either admitted or quarantined.

Feynman read: plugging in a phone is not the same as the phone being ready. Readiness is a chain of checks.

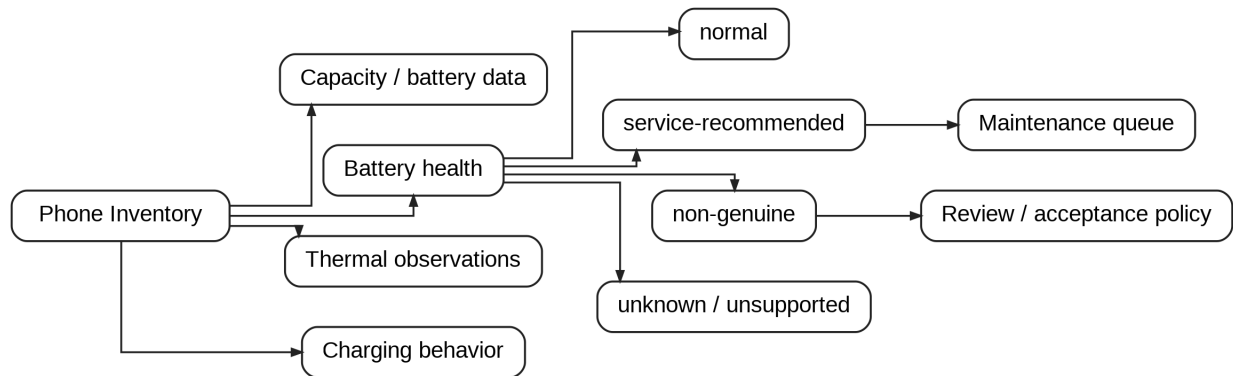
ARCH-DEVICE-STATE-001 - Runtime device lifecycle



ARCH-DEVICE-STATE-001 - Runtime health state machine for a tethered iPhone pod.

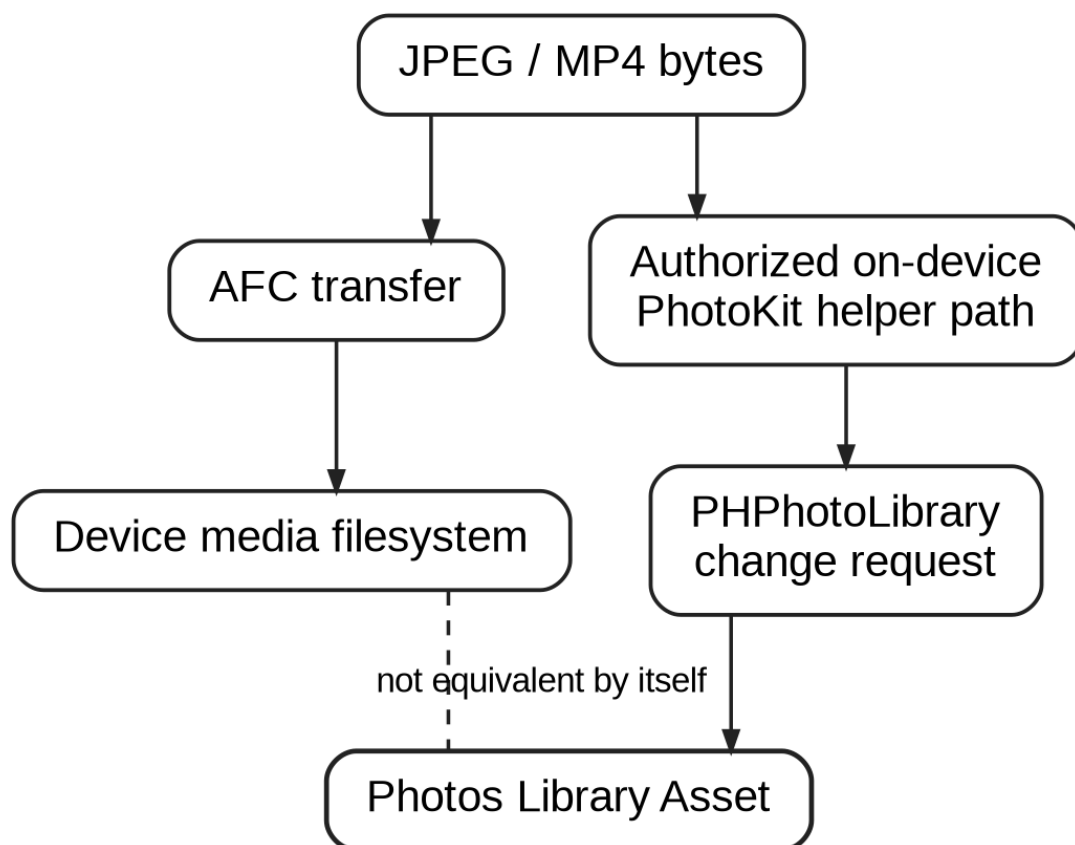
Feynman read: a device can fail at several doors: physical USB, pairing, developer services, display or input. The state tells the recovery system which door failed.

ARCH-BATTERY-001 - Battery and thermal maintenance model



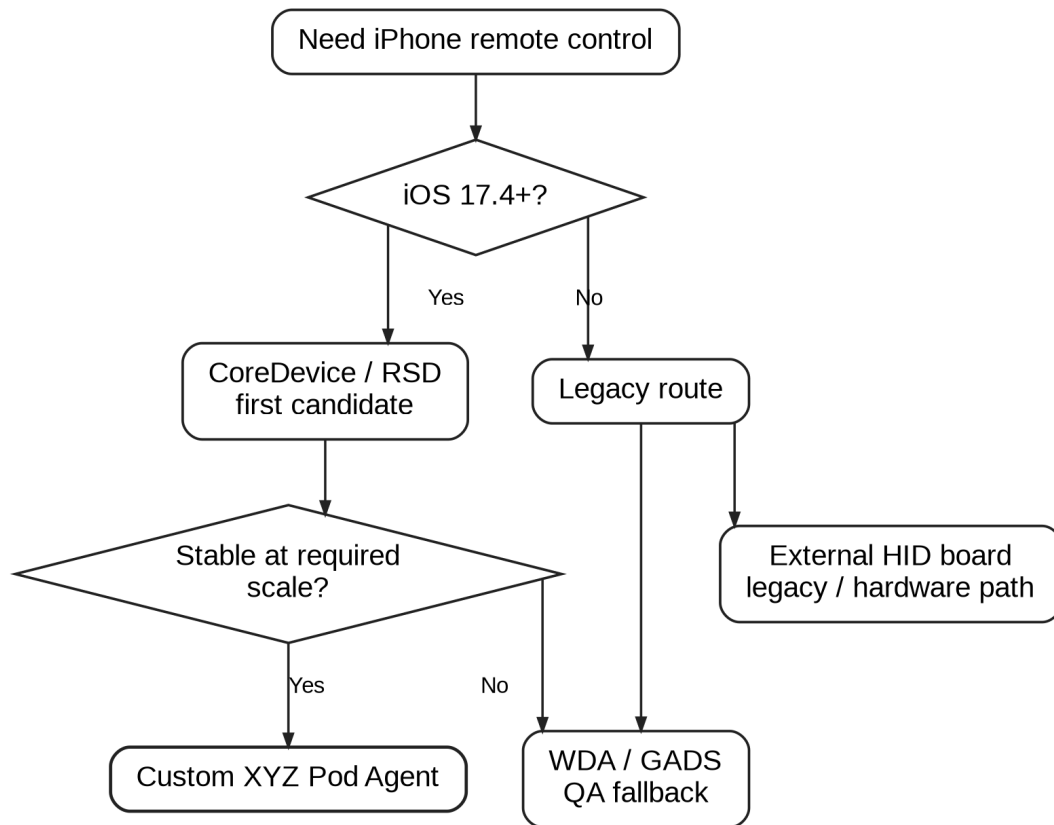
ARCH-BATTERY-001 - Battery health, charging behavior and thermal observations feed maintenance policy.

ARCH-PHOTOS-001 - Bytes versus a Photos-library asset



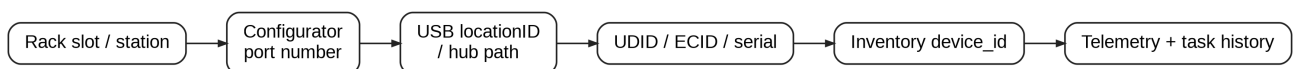
ARCH-PHOTOS-001 - AFC byte transport is not itself Photos-library ingestion; PhotoKit is the first-party asset-creation path when a helper is allowed.

ARCH-IOS-CHOICE-001 - iOS control-path decision tree



ARCH-IOS-CHOICE-001 - Modern CoreDevice is tested first; WDA/GADS and external hardware remain fallbacks.

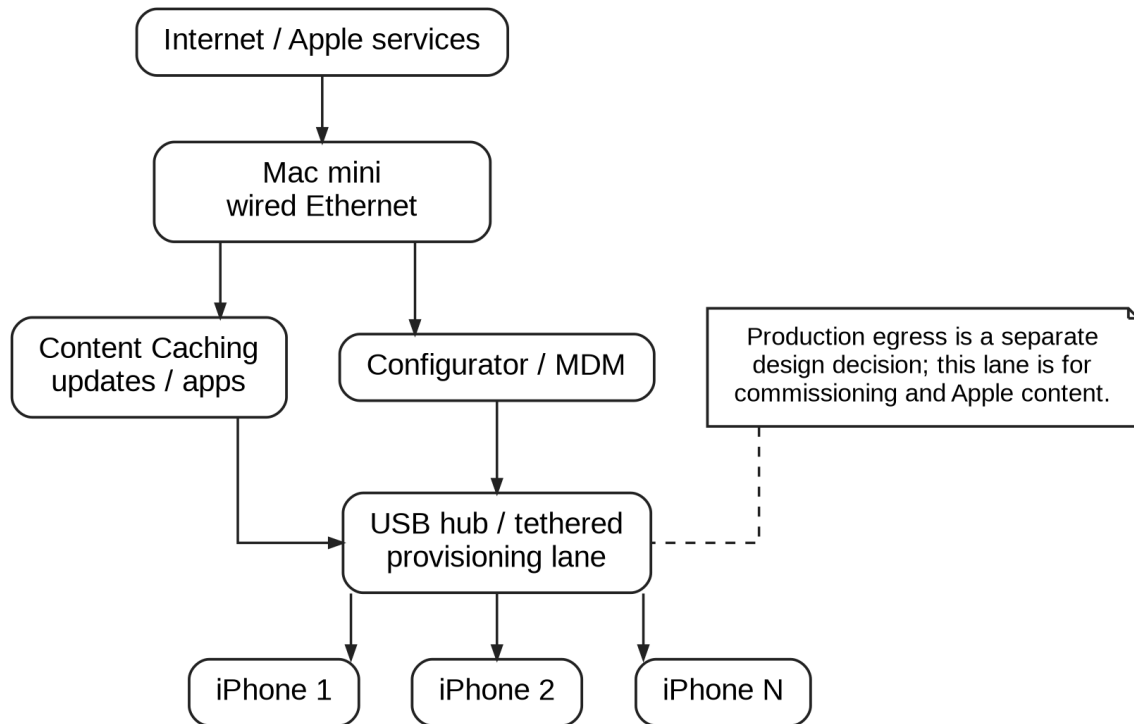
ARCH-SLOT-ID-001 - Physical slot to logical history



ARCH-SLOT-ID-001 - The identity chain binds physical rack position to the inventory record and history.

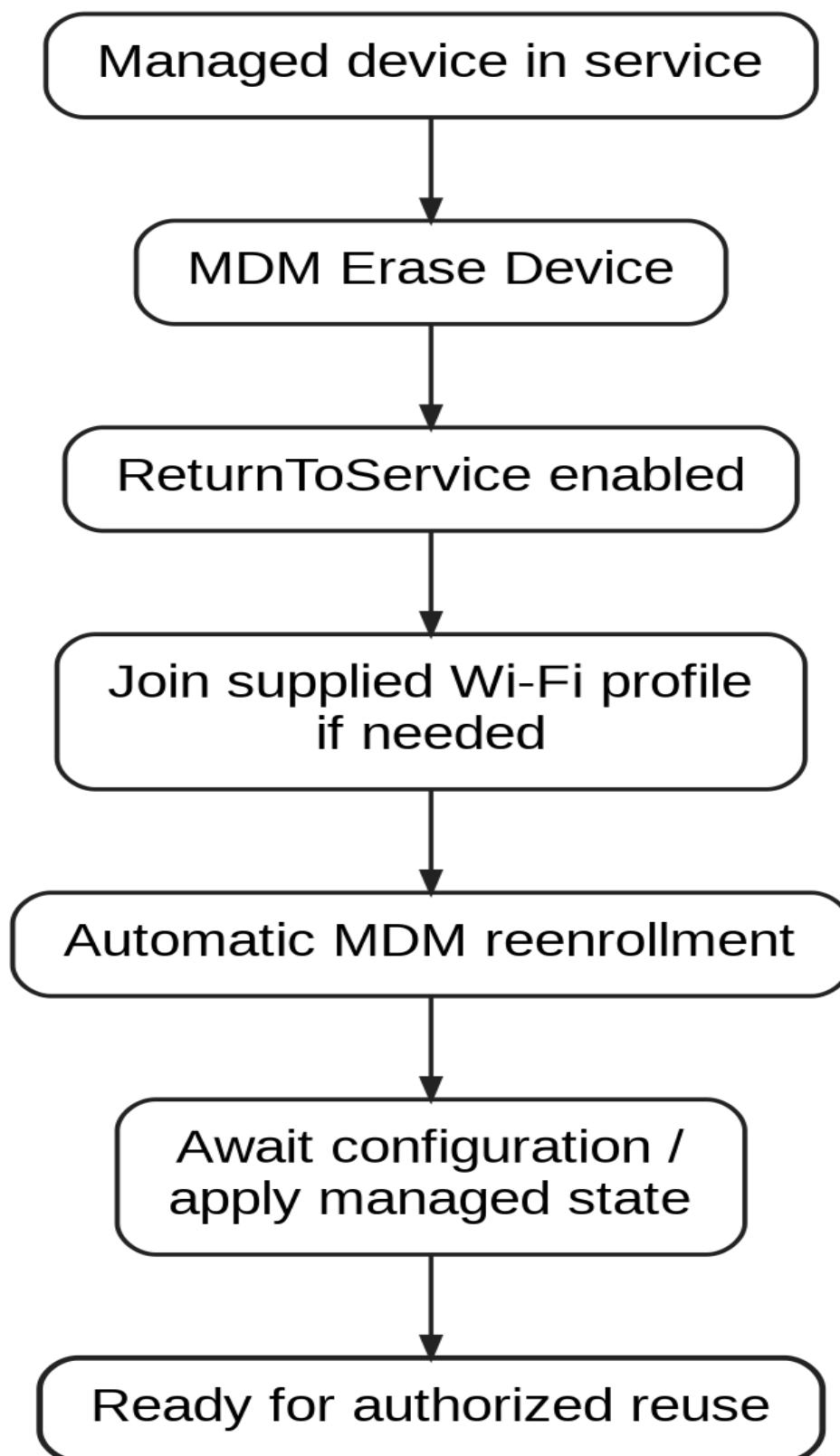
Feynman read: a cable swap must not silently swap identities. The system should detect that slot 7 now contains a different ECID/UDID.

ARCH-PROVISION-001 - Tethered provisioning lane



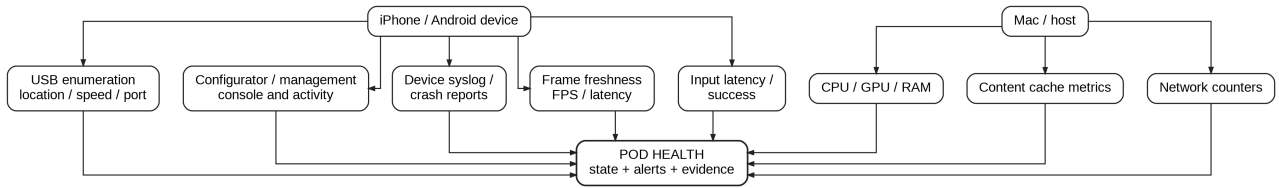
ARCH-PROVISION-001 - Apple content caching and Configurator form a commissioning lane separate from production egress.

ARCH-RTS-001 - Return to Service lifecycle



ARCH-RTS-001 - Authorized device reset and re-enrollment flow using Apple Return to Service.

ARCH-OBS-001 - Pod observability stack



ARCH-OBS-001 - USB, management, device, video, input, host and network signals converge into one pod-health model.

Feynman read: “online” is not one boolean. A healthy pod is a set of fresh, mutually consistent signals.

Phone Farm Handbook V1.3 - Research Delta

Research cut: 27 August 2026

Scope: authorized physical-device fleet engineering, Mac/iOS pod design, lifecycle, reliability, observability and scaling.

Evidence rule: first-party Apple documentation for Apple behavior; maintained project documentation for pymobiledevice3; bench testing remains required for capacity and client-specific hardware.

What V1.3 changes: The system is now modeled as a pod supervisor controlling one isolated worker process per modern iPhone, with explicit pairing, RSD, Developer Disk Image, display, input, power and recovery states. Configurator is kept in a commissioning plane. Runtime device control belongs to the pod software. The 20-device target is treated as a stack of simultaneous acceptance gates, not as a screen-count target.

1. Modern iOS transport: two doors, not one

The current pymobiledevice3 Python API separates classic Lockdown/usbmux services from iOS 17+ developer services. Classic services include AFC, application installation, syslog, diagnostics, backup and profiles. Developer/DVT services on iOS 17+ require Remote Service Discovery (RSD).

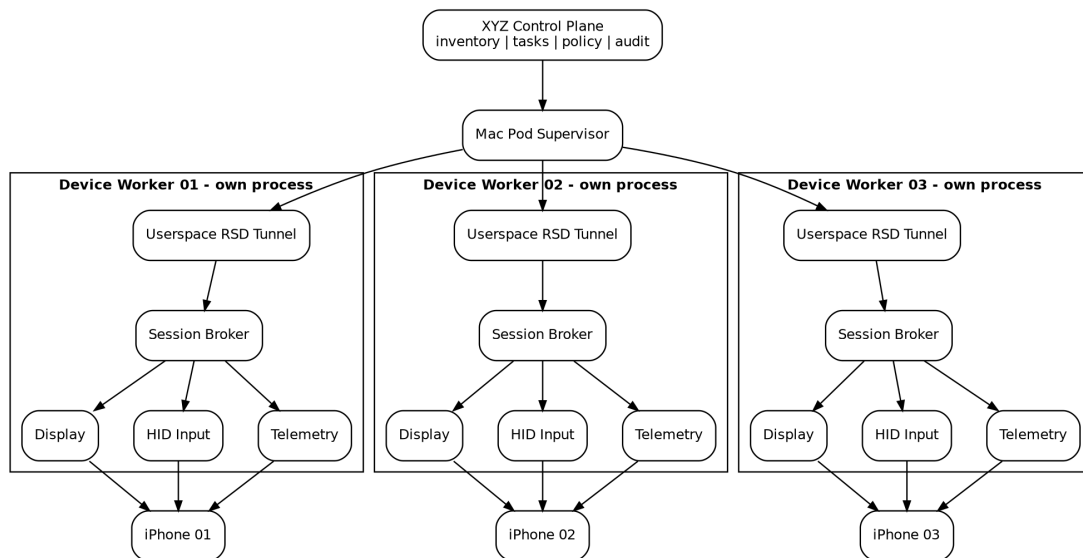
Feynman explanation: imagine the phone has two service doors. The first is the long-established USB management door. The second is the developer-tool door that Apple moved behind CoreDevice/RemoteXPC in iOS 17. A reliable device worker chooses the correct door for each job instead of forcing every operation through the same transport.

For iOS 17.4+, the maintained pymobiledevice3 documentation says its preferred userspace RSD tunnel can be created in-process over USB without root/admin on macOS, Windows and Linux. iOS 17.0-17.3.1 is different: those releases predate the CoreDeviceProxy service used by the cleaner USB path, so the project normally routes them through tunneId.

This means the engineering rule is now explicit: **do not create one generic “iOS 17 driver.” Version classes matter.**

2. One device worker process per iPhone

The current Python API documentation describes the userspace RSD tunnel as in-process and states a one-tunnel-per-process caveat. That creates a natural software boundary.



ARCH-WORKER-PROCESS-001

Feynman explanation: instead of building one giant process with twenty phones tangled inside it, give each phone its own small worker. If phone 14 wedges its transport, restart worker 14. Phones 1-13 and 15-20 should not care.

The Mac pod supervisor should own:

- worker creation, restart and resource limits;
- inventory-to-worker mapping;
- aggregate health;
- leases from the central control plane;
- pod-level USB, CPU, memory and thermal capacity;
- quarantining devices that repeatedly fail.

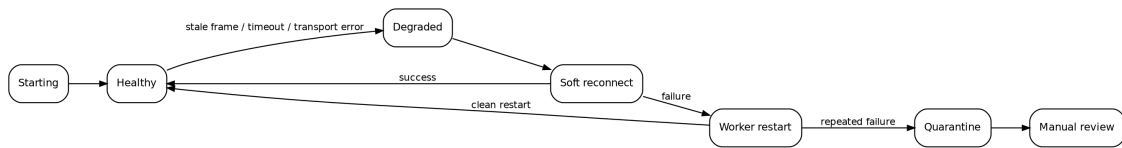
The per-device worker should own:

- the device's RSD tunnel when required;
- Lockdown/usbmux service connections;
- a session broker for display/input/diagnostics;
- fresh-frame monitoring;
- input state;
- device telemetry;
- local recovery and bounded reconnect attempts.

This is an **inference from the maintained project's documented tunnel model**, not a claim that Apple requires one process per phone.

3. Recovery should escalate, not retry forever

A production rack should assume that a cable, device service, developer image, process or transport will occasionally fail. The system therefore needs a bounded escalation path.



ARCH-WORKER-RECOVERY-001

A practical order is:

1. detect degradation from stale frames, timeouts, transport errors or health mismatches;
2. try one bounded soft reconnect;
3. if that fails, terminate and recreate the device worker;
4. if repeated clean worker restarts fail, quarantine the phone;
5. keep the other workers running throughout.

The goal is **fault containment**. A single misbehaving phone must not slowly poison the pod through leaked sockets, stale sessions, duplicated task leases or incorrect identity mapping.

4. Developer Mode and DDI are real readiness states

Current `pymobiledevice3` documentation states that many developer services require Developer Mode and a mounted Developer Disk Image. On iOS 17+, the implementation uses Apple's Personalized Developer Disk Image model and may obtain personalization material through Apple's ticketing path.

So "USB connected" is not equal to "control ready." A useful state chain is:

physical connection -> USB enumeration -> pairing/trust -> Developer Mode -> RSD -> DDI ready -> display ready -> input ready -> healthy.

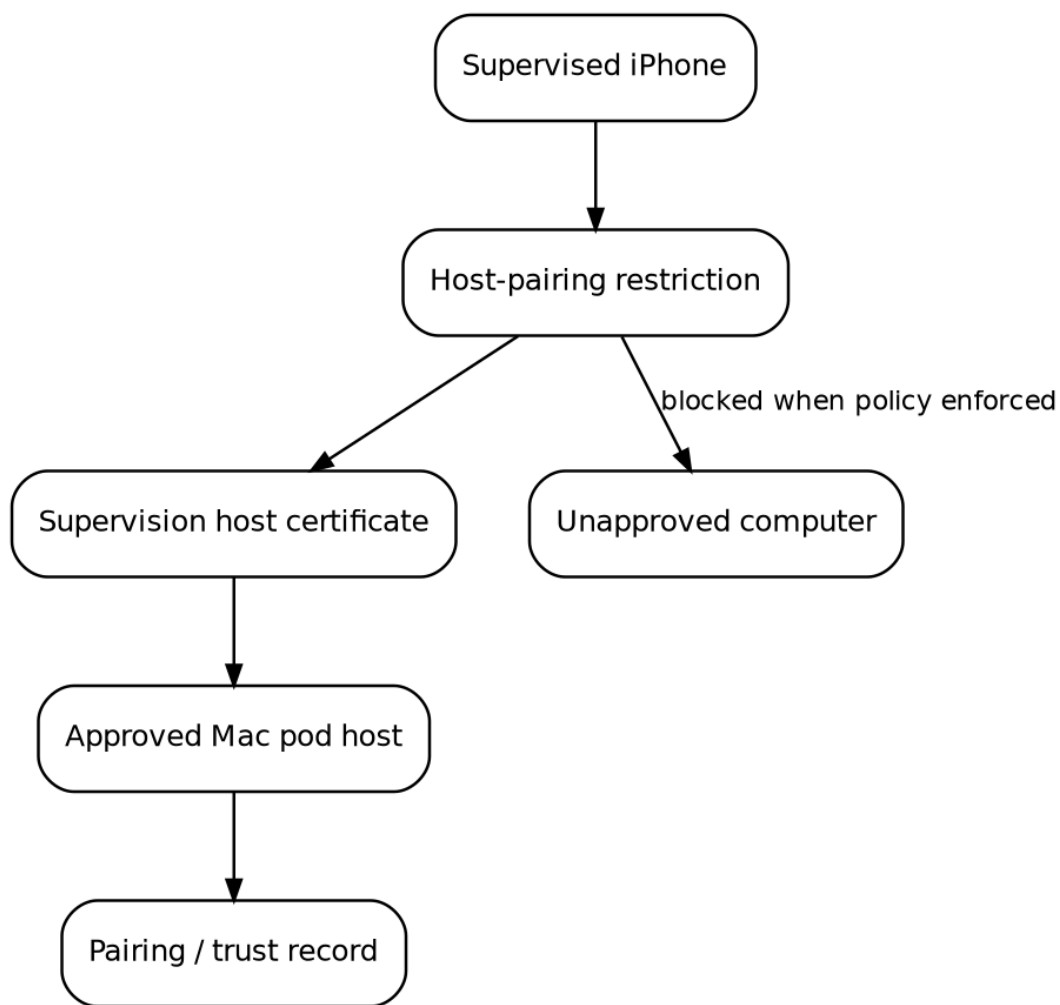
A cold reboot test should therefore record how the phone returns to each state, which transitions require human interaction, and how long every transition takes.

A second detail matters for scaling: the current project documentation notes that sustained host-to-device transfer such as DDI mounting and large pushes can be slower over the userspace tunnel. So the easiest tunnel is not automatically the best bulk-transfer path. Bulk work must be measured separately from interactive display/input.

5. USB pairing is a security relationship

Apple documents that supervised devices can restrict pairing with non-Apple-Configurator hosts. By distributing the correct supervision identities, an organization can make only approved host computers with the correct supervision certificate eligible to pair with the supervised iPhone/iPad.

Apple also documents that unused pairing trust records are removed after 30 days.



ARCH-PAIRING-BOUNDARY-001

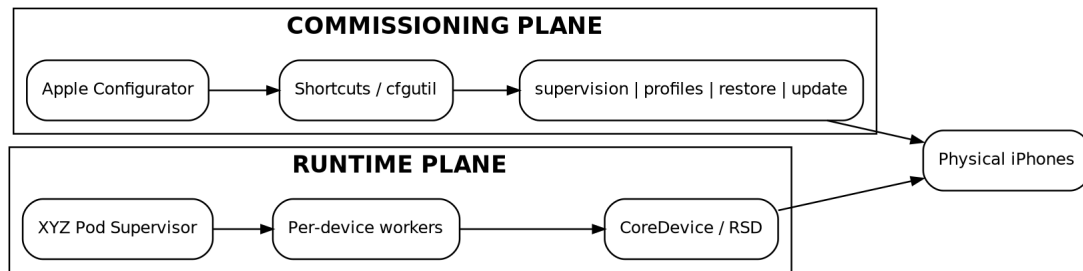
Feynman explanation: the popup saying “Trust This Computer?” creates a relationship. In a rack, that relationship should belong to the approved Mac host, not to whichever laptop happens to be plugged in.

If the fleet also relaxes USB Restricted Mode for unattended locked-device operation, physical security becomes more important. Availability and security must be designed together.

6. Configurator belongs in the commissioning plane

Apple Configurator can automate attach/detach actions with Shortcuts and can gather supported port numbers. But Apple’s documentation explicitly notes that other Configurator functions become unavailable while attach/detach shortcuts are active.

That makes the clean architecture:



ARCH-COMMISSION-RUNTIME-002

Commissioning plane: prepare, supervise, profile, restore, update, identify and register devices.

Runtime plane: pod supervisor, per-device workers, CoreDevice/RSD, Lockdown services, display, input and telemetry.

This separation avoids turning Configurator into a permanent runtime dependency for functions it was not designed to own continuously.

7. Tethered caching is a real commissioning network

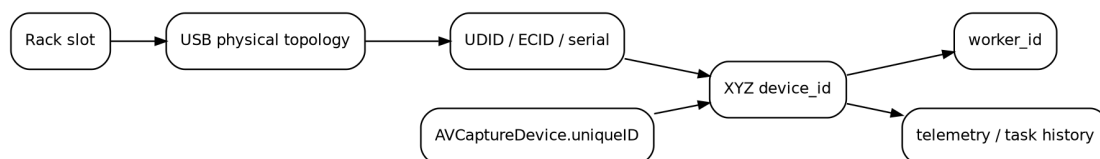
Apple documents that USB-connected iOS devices can receive the Mac's internet connection and cached content. The devices can have Wi-Fi and cellular disabled and still receive internet through the Mac. Managed tethered iOS devices automatically check in with device management. Apple also documents this as useful with carts or USB hubs and many physically connected devices.

Feynman explanation: during provisioning, the cable can act like a temporary network cable for the phone. That means restore, activation, enrollment, operating-system updates and managed-app downloads can be designed around the Mac's Ethernet connection rather than around a crowded Wi-Fi network.

This is a **commissioning/provisioning lane**. It should not be confused with or represented as a method for hiding or altering production application egress.

8. Stable identity: never trust “iPhone” or capture index 7

Apple's AVFoundation documentation says `AVCaptureDevice.uniqueID` persists on a Mac across device disconnections, application restarts and Mac reboots.



ARCH-CAPTURE-ID-001

The inventory should therefore bind several independent identities:

- physical rack slot;
- USB topology/branch where observable;
- UDID/ECID/serial;
- persistent AVFoundation capture uniqueID when using that capture path;
- internal device_id;
- current worker process ID;
- historical telemetry/task records.

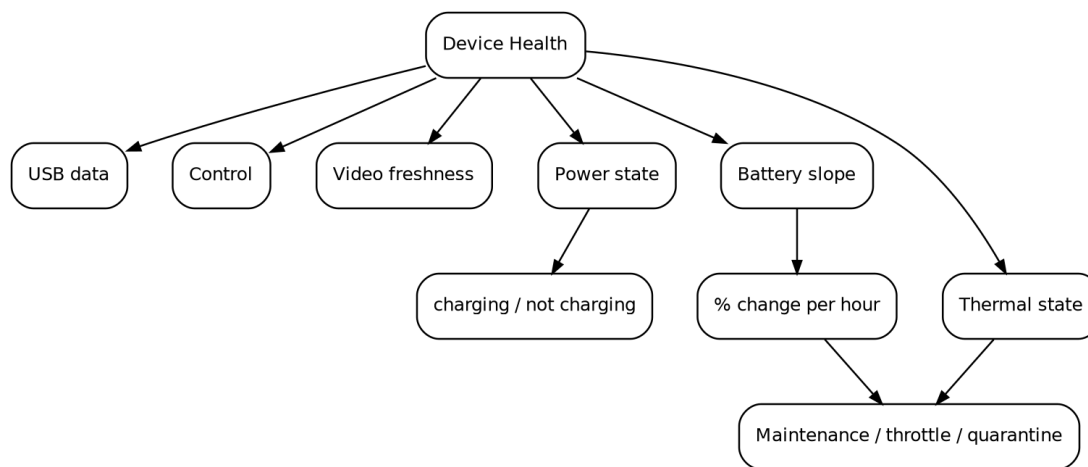
Feynman explanation: list order is temporary; identity is durable. If two phones change list position after reconnect, the control plane must still know exactly which physical phone is which.

9. Charging is a separate health channel

Apple states that when an iPhone is charged from a computer or a USB hub with several attached devices, the maximum power can fall depending on combined device demand and system conditions.

So this situation is possible:

USB data works + video works + input works + battery is slowly falling.



ARCH-POWER-HEALTH-002

A rack health model should therefore track **battery slope**: percentage change per hour over a representative window. A phone at 85% and losing 3% per hour is unhealthy even though the current percentage looks comfortable.

Power telemetry should be interpreted together with battery-health status, host/device thermals, reconnect count and workload.

10. Thermal pressure is a capacity limit

Apple's thermal-state documentation describes `serious` as a state in which the system reduces performance and recommends reducing I/O, CPU/GPU use and frame rate. `critical` means performance is significantly affected and the device needs to cool down.

For a phone rack, this means thermal pressure is not merely “the phones feel warm.” It is a capacity signal. A pod that only meets latency targets when cold has not passed its soak test.

The acceptance test should therefore include:

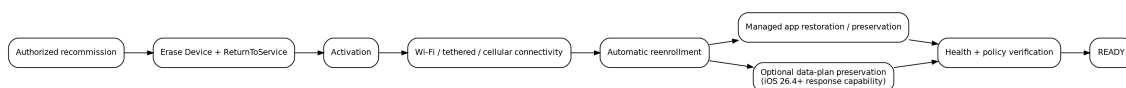
- cold start metrics;
- metrics after steady-state charging;
- metrics under representative simultaneous video/control load;
- metrics after 24 hours;
- ambient temperature;
- host thermal state;
- device surface/rack observations where safely measurable.

11. Return to Service is now a serious fleet-lifecycle tool

Apple's current device-management documentation describes Return to Service as an automated erase and reenrollment mechanism. For supported iOS devices using Automated Device Enrollment, Return to Service with app preservation can preserve managed app executable/system state while erasing local user data, using a bootstrap token.

Apple also documents newer version-gated capabilities:

- iOS 27+: `ShouldRetryEnrollment` can retry enrollment after an initial failure;
- iOS 27: deployment updates describe software-update enforcement during Return to Service;
- iOS 26.4+: the `ReturnToService` response supports a `PreserveDataPlan` choice for an existing eSIM data plan.

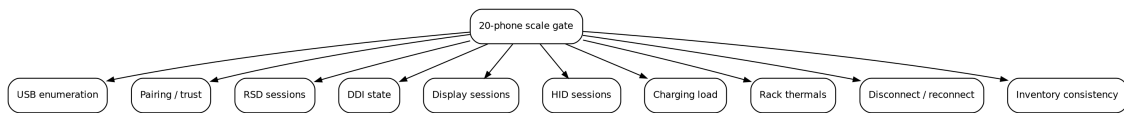


ARCH-RTS-ESIM-002

Feynman explanation: “erase the phone” is not one switch. User data, managed apps, management enrollment and cellular plan are separate lifecycle layers. The durable fleet should decide each layer deliberately.

These mechanisms are for owned-device recommissioning and do not imply that third-party platform enforcement state is erased.

12. The 20-device test is a stack of simultaneous gates



ARCH-SCALE-GATE-002

A pod does **not** pass because twenty phone windows happen to be visible. All of these must remain healthy together:

- USB enumeration;
- pairing/trust;
- RSD sessions;
- DDI readiness;
- display freshness;
- input sessions;
- charging capacity;
- rack thermals;
- disconnect/reconnect behavior;
- stable inventory identity.

The recommended sequence remains **1 -> 2 -> 5 -> 10 -> 20**.

At each stage record:

- setup time and failure distribution;
- CPU/RAM and process count;
- fresh-frame rate and stale-frame duration;
- input latency and failures;
- per-device battery slope;
- USB reconnect events;
- DDI/RSD recoveries;
- host and rack thermal behavior;
- worker restarts and memory after restart;
- whether one-device faults disturb neighboring devices.

Then perform a 24-hour soak, followed by 72 hours before calling the 20-device pod durable.

13. Updated experiments

TEST-WORKER-ISO-001 - Worker isolation

Kill or crash one worker intentionally. The neighboring phone sessions must remain healthy. Restarted worker must recover the same inventory identity without creating duplicate leases.

TEST-PAIRING-001 - Approved-host pairing

Verify that the intended Mac can pair/reconnect under supervision policy and an unapproved host cannot pair when the restriction is enabled. Document recovery after trust expiry or device erase.

TEST-DDI-001 - DDI cold-recovery lifecycle

Reboot the phone and record every step back to DDI-ready, display-ready and input-ready. A production path needs a bounded deterministic sequence.

TEST-CACHE-001 - Tethered commissioning

With Wi-Fi and cellular disabled, verify that the USB-connected managed phone can reach required Apple/management services through the Mac, perform intended provisioning and remain correctly inventoried.

TEST-RTS-001 - Return to Service

On one owned managed pilot phone, validate erase -> activation -> reenrollment -> configuration -> health verification. Test app/data-plan preservation only on OS/MDM combinations that document those features.

TEST-POWER-SLOPE-001 - Sustained charging

Under representative screen/control workload, record battery percentage over hours. A device that slowly discharges despite data connectivity fails the power design.

14. Host resilience

Apple now documents automatic startup when power is restored for 2024-or-later Mac mini models running macOS 26.5 or later. This should be enabled on an unattended pod after validating that the pod services themselves also restart cleanly.

The recovery chain becomes:

utility power returns -> Mac starts -> supervisor starts -> USB inventory reconciles -> workers start -> devices return through trust/RSD/DDI/display/input gates -> pod reports healthy.

The Mac returning to power is only the first step. The acceptance test must exercise the whole chain.

15. What is still OPEN

No public source reviewed in this pass establishes the following client-specific facts:

1. one M4 Mac mini can sustain 20 simultaneous target iPhone display streams at the required freshness/latency;
2. 20 simultaneous userspace RSD/Developer-service stacks are stable on the client's exact OS mix;
3. the selected hub can maintain both data and sufficient charging under the real cable/device load;
4. the incoming external control board can coexist with the selected Mac display path and charging topology;
5. a 24-72 hour pod soak remains stable after repeated unplug/replug, reboot, lock, trust and worker-restart faults.

These are bench questions. Additional web reading should refine test design, but it cannot convert them into facts.

16. V1.3 coverage and understanding

These percentages are **engineering self-assessments, not independently measured statistics.**

Measurement	V1.2 handbook	V1.3 after this pass
Research coverage	98.3%	99.2%
Conceptual understanding	97.3%	98.9%
Actual bench validation	74.4%	74.4%

Coverage rose because the remaining legitimate architecture is now mapped down to process boundaries, pairing identity, DDI state, tethered commissioning, persistent capture identity, managed recommissioning variants, power slope and explicit 20-device acceptance gates.

Understanding remains below 100% because simultaneous multi-device behavior is empirical. The architecture is increasingly clear; the capacity numbers are not.

Bench validation did not rise because this pass added documentation and design evidence, not a physical test.

17. V1.3 source register

1. pymobiledevice3 Python API - <https://github.com/doronz88/pymobiledevice3/blob/master/docs/guides/python-api.md>
2. pymobiledevice3 iOS 17+ tunnels - <https://github.com/doronz88/pymobiledevice3/blob/master/docs/guides/ios17-tunnels.md>
3. pymobiledevice3 protocol layers - https://github.com/doronz88/pymobiledevice3/blob/master/misc/understanding_idevice_protocol_layers.md
4. pymobiledevice3 image mounter - https://github.com/doronz88/pymobiledevice3/blob/master/pymobiledevice3/services/mobile_image_mounter.py
5. pymobiledevice3 transport notes - <https://github.com/doronz88/pymobiledevice3/blob/master/misc/claude-plugin/skills/pymobiledevice3-device-operator/references/transport-and-safety.md>
6. Apple accessory access / host pairing - <https://support.apple.com/en-gb/guide/deployment/depf8a4cb051/1/web/1.0>
7. Apple device-management restrictions - <https://developer.apple.com/documentation/devicemanagement/restrictions>
8. Apple Configurator Shortcuts automations - <https://support.apple.com/guide/apple-configurator-mac/acm12a6f7b56/mac>
9. Apple content caching on Mac - <https://support.apple.com/en-ca/guide/mac-help/mchl3b6c3720/mac>
10. Apple shared internet for iPhone/iPad fleets - <https://support.apple.com/guide/deployment/set-up-a-shared-internet-connection-dep38ff24bed/1/web/1.0>
11. Apple Return to Service - <https://developer.apple.com/documentation/devicemanagement/returning-a-managed-device-to-service>
12. Apple ReturnToService command schema - <https://developer.apple.com/documentation/devicemanagement/erasedevicecommand/command-data.dictionary/returntoservice-data.dictionary>
13. Apple WWDC26 device-management updates - <https://support.apple.com/guide/deployment/device-management-updates-depd638aa061/web>
14. Apple ReturnToServiceResponse - <https://developer.apple.com/documentation/devicemanagement/returntoserviceresponse>
15. Apple AVCaptureDevice.uniqueID - <https://developer.apple.com/documentation/avfoundation/avcapturedevice/uniqueid>
16. Apple iPhone charging speeds - <https://support.apple.com/tr-tr/120619>
17. Apple thermal state - <https://developer.apple.com/documentation/foundation/processinfo/thermalstate-swift.enum/serious>
18. Apple Mac mini automatic power-on - <https://support.apple.com/en-us/125517>
19. Apple Mac mini 2024 specifications - <https://support.apple.com/tr-tr/121555>

XYZ MGMT Phone Farm Handbook V1.4

Research Delta

XYZ MGMT Phone Farm Engineering

Handbook V1.4 - Research Delta

Human / Feynman edition - 27 August 2026

This appendix extends the V1.3 handbook. It is written for authorized organization-owned device-fleet engineering. It does not operationalize platform-evasion, fake-engagement, identity-spoofing or prohibited automation.

The diagrams in this appendix have matching canonical Mermaid source in the LLM .md edition.

V1.4 RESEARCH DELTA - 27 AUGUST 2026

Precedence: V1.4 is newer than the V1.3 delta. Where this section narrows a V1.3 recommendation, V1.4 wins. All concurrency and capacity claims remain OPEN until they are reproduced on the client's own Mac, hub, cables and phones.

V.1 What this pass changed

This pass focused on five gaps that still mattered after V1.3:

1. how to combine second-by-second pod telemetry with Apple-managed inventory truth;
2. how to survive CoreDevice service changes without hard-coding fragile OS assumptions;
3. how the Mac pod software itself should start, restart, log and recover after reboots;
4. how RustDesk should fit into the architecture without becoming the device-control authorization system;
5. how current pymobiledevice3 tunnel/DDI behavior changes the recovery and version-pinning rules.

The architectural theme is **separation of concerns**. Runtime observation, managed-device truth, operator remote desktop, host process supervision and device-control transport are different planes. Treating them as one layer makes failures ambiguous.

V.2 New evidence-backed claims

PF-V14-OBS-001 - Declarative status is event-driven, but also has a periodic full safety report

- **status:** VERIFIED-FIRST-PARTY
- **claim:** Apple's Declarative Device Management status protocol sends incremental status reports when subscribed items change. Apple also documents a periodic full status report, typically about once per day, so a management server can recover if it missed an incremental update.
- **feynman_explanation:** Imagine a warehouse clerk who usually tells you only what changed: "Phone 12 updated to iOS 27." Once a day, the clerk sends the entire inventory sheet again. That full sheet is not fast telemetry, but it is valuable for repairing drift in the database.
- **architecture_use:** MDM/DDM is a **slow authoritative reconciliation plane**, not the primary source for frame freshness, RSD health, HID latency or sub-minute device state.
- **source:** S-V14-01, S-V14-02.

PF-V14-OBS-002 - Managed-device status can provide durable inventory and health fields

- **status:** VERIFIED-FIRST-PARTY
- **claim:** Apple's current status-item model includes serial number, UDID, hardware model identifier, OS version, OS build version, battery health and managed-app status. Apple also documents a device system-health status item, currently beta, that can report component health where supported.

- **feynman_explanation:** The pod worker knows what is happening **right now**. Device management knows what the organization says the device **is**: its serial, model, software build and managed state. Those two views should meet in the database.
- **do_not_infer:** Do not assume every status item exists on every OS/model/enrollment type. Subscribe only to supported items and treat beta status as optional evidence.
- **source:** S-V14-03 through S-V14-09.

PF-V14-CAP-001 - Runtime feature probing is safer than OS-version-only branching

- **status:** MAINTAINED-REPO + REPRODUCIBLE-ISSUE
- **claim:** Current pymobiledevice3 evidence shows that the set of CoreDevice/RSD services can differ across devices and OS releases. A 2026 issue documented a CoreDevice pasteboard service absent from both iOS 17.5.1 and iOS 26.2 even though other CoreDevice/DDI services were healthy. Current release notes also mention newer iOS versions where a previously expected CoreDevice service is no longer exposed.
- **feynman_explanation:** Asking only “Which iOS version is this?” is like assuming every car of the same model year has the same optional equipment. A stronger worker asks the phone which capabilities are actually present **right now**, records the answer, and enables only the adapters that passed the probe.
- **architecture_use:** Build a capability registry keyed by device ID + iOS build + host tool version. Version rules choose the likely transport; runtime probes decide which features are enabled.
- **source:** S-V14-10, S-V14-11.

PF-V14-CAP-002 - Capability loss is a supported degraded state, not necessarily a fatal pod failure

- **status:** INFERENCE
- **claim:** Because individual services can disappear while the RSD path remains healthy, a missing optional service should disable only the dependent feature where possible.
- **feynman_explanation:** If the radio in a car stops working, the car should not report “engine destroyed.” Likewise, missing clipboard or one diagnostic service should not automatically kill display, input and inventory if those remain healthy.
- **do_not_infer:** A missing **required** service such as the selected display or HID path can still make that device unready for the intended workload.

PF-V14-HOST-001 - The pod supervisor should be a macOS-managed persistent service

- **status:** VERIFIED-FIRST-PARTY + INFERENCE
- **claim:** Apple recommends Service Management / SMDaemon for persistent launch agents and launch daemons on modern macOS. launchd is the system service manager and can keep or relaunch jobs according to their configuration.
- **feynman_explanation:** Do not make the farm depend on somebody opening Terminal and running `python supervisor.py`. The operating system already has a foreman whose job is to start background services after boot and supervise them. Let that foreman start the pod supervisor.

- **architecture_use:** The Mac should recover to a running supervisor after reboot without an operator launching the application manually. Per-phone workers are still controlled by the pod supervisor so the application can enforce bounded retries and device-specific quarantine.
- **source:** S-V14-12, S-V14-13, S-V14-14.

PF-V14-LOG-001 - Unified Logging and signposts fit the pod observability model

- **status:** VERIFIED-FIRST-PARTY
- **claim:** Apple's unified logging system supports structured logs and signposts, and the OSLog framework can read historical log entries. Signposts can record time-bounded operations for performance analysis.
- **feynman_explanation:** A log says "worker 08 started a tunnel." A signpost lets you measure "how long did tunnel startup take?" At 20 phones, timing distributions matter more than isolated anecdotes.
- **architecture_use:** Every lifecycle operation should carry a device ID, worker ID, operation ID and result. Sensitive account content should not be written to infrastructure logs.
- **source:** S-V14-15, S-V14-16.

PF-V14-REMOTE-001 - RustDesk belongs to the operator-access plane, not the device-control trust plane

- **status:** VENDOR-DOC + INFERENCE
- **claim:** RustDesk's self-hosted OSS backend uses hbbs for rendezvous/signaling and hbbr when relay traffic is needed. RustDesk Server Pro adds centralized features including OIDC, LDAP, 2FA, device management and access control. On macOS the client requires Screen Recording and Accessibility permissions for remote operation, with Input Monitoring needed in some cases.
- **feynman_explanation:** RustDesk is the window through which a remote employee sees the Mac. It should not decide which employee is allowed to lease Phone 07 or which client owns an account. Those decisions belong to the XYZ control plane and audit system.
- **architecture_use:** Remote desktop can be self-hosted and hardened, but all consequential device actions still pass through the control-plane authorization model where practical.
- **source:** S-V14-17 through S-V14-20.

PF-V14-REMOTE-002 - macOS privacy permissions are a deployment dependency for unattended remote desktop

- **status:** VENDOR-DOC
- **claim:** RustDesk documents that macOS remote control requires Screen Recording and Accessibility permissions, and may require Input Monitoring. Its 2026 MDM deployment notes state that installing the auto-start service does not itself grant these privacy permissions; they must be granted separately, manually or through an appropriate managed privacy profile.
- **feynman_explanation:** "The RustDesk service is running" does not imply "the team can see and control the Mac." The privacy permissions are a second gate.
- **source:** S-V14-19, S-V14-20.

PF-V14-REMOTE-003 - RustDesk/macOS compatibility belongs in the acceptance test

- **status:** REPRODUCIBLE - COMMUNITY - ISSUE
- **claim:** A June 2026 RustDesk discussion reports a macOS 26 Tahoe scenario in which a manually created LaunchAgent/LaunchDaemon path failed Screen Recording capture because of TCC responsible-process behavior even though the application itself had permission.
- **feynman_explanation:** macOS privacy permission can depend on **how** a process is launched, not merely whether the binary appears in the permission list. Therefore unattended remote access needs its own reboot/login-window test on the actual macOS/RustDesk version.
- **do_not_infer:** This is not proof that RustDesk's supported service-install path is universally broken. It is evidence that custom launch arrangements must be tested rather than improvised.
- **source:** S-V14-21.

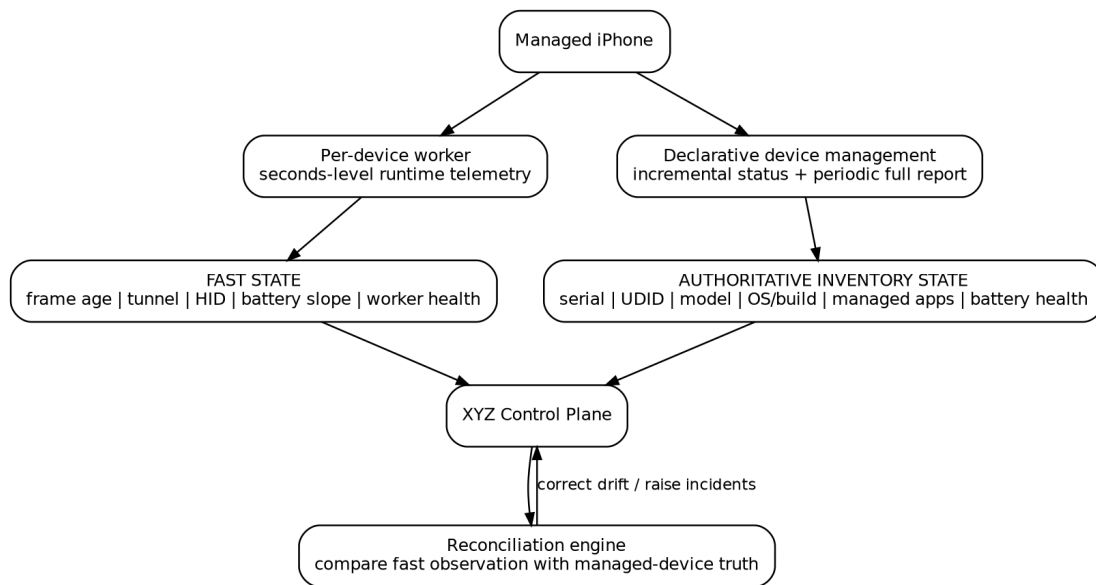
PF-V14-TRANS-001 - Userspace tunnel teardown and reconnect remain active engineering failure modes

- **status:** MAINTAINED-REPO / 2026 ISSUES
- **claim:** 2026 pymobiledevice3 issues documented userspace tunnel cleanup hanging during teardown and a reconnect failure after unplug/replug on Linux. Current releases also include fixes around userspace-tunnel retry and service errors.
- **feynman_explanation:** A tunnel is not a magical permanent wire. It is software state that can get stuck during shutdown or disappear when the phone reconnects. The worker must assume that transport teardown itself can fail.
- **architecture_use:** Keep retries bounded. If one reconnect attempt fails, restart the worker process so its event loop, tunnel and sockets are rebuilt together. Escalate repeated failure to quarantine.
- **source:** S-V14-22, S-V14-23, S-V14-24.

PF-V14-DDI-001 - DDI readiness is partly a supply-chain/version-pinning problem

- **status:** MAINTAINED-REPO + INFERENCE
- **claim:** The current personalized Developer Disk Image implementation uses a local image/build manifest/trust cache and may request a fresh personalization ticket from Apple's TSS server. The implementation also warns when the downloaded DDI build does not match the version expected by the installed tooling.
- **feynman_explanation:** DDI is not just a checkbox on the phone. It is also a host-side artifact with versions. If the host silently updates its device library or the expected image changes, a previously working boot-to-ready path can break.
- **architecture_use:** Pin the pod-agent dependency version, record the DDI build used for each readiness event, pre-cache required artifacts where licensing permits, and run a compatibility canary before fleet-wide host updates.
- **source:** S-V14-25.

V.3 Canonical architecture diagram - two-speed truth model



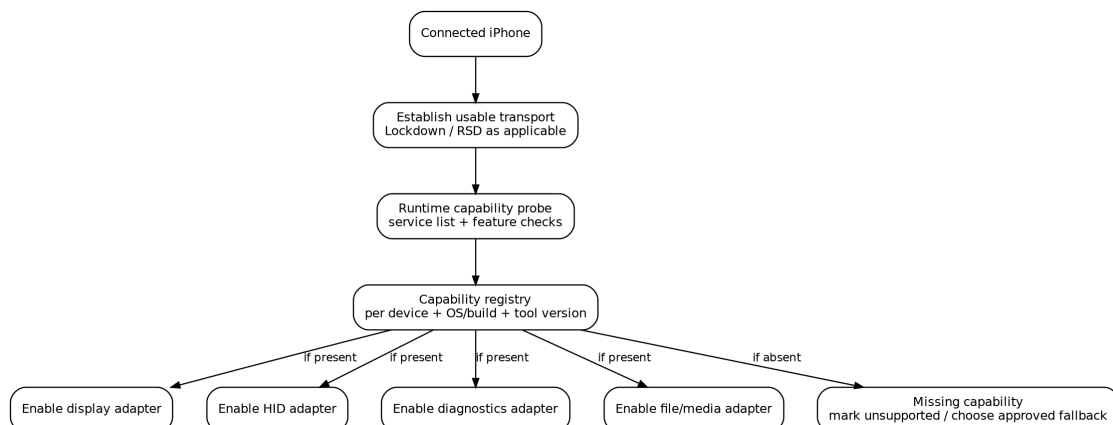
ARCH-TELEMETRY-TIERS-001 - Two-speed truth model

Diagram ID: ARCH-TELEMETRY-TIERS-001

Feynman explanation

The fast path answers “**Can I use this phone right now?**” The slow managed path answers “**What device is this and what managed state should it have?**” Do not ask MDM for 30-fps video health, and do not ask a transient worker connection to be the organization’s permanent inventory authority.

V.4 Canonical architecture diagram - capability probing



ARCH-CAPABILITY-PROBE-001 - Runtime feature discovery

Diagram ID: ARCH-CAPABILITY-PROBE-001

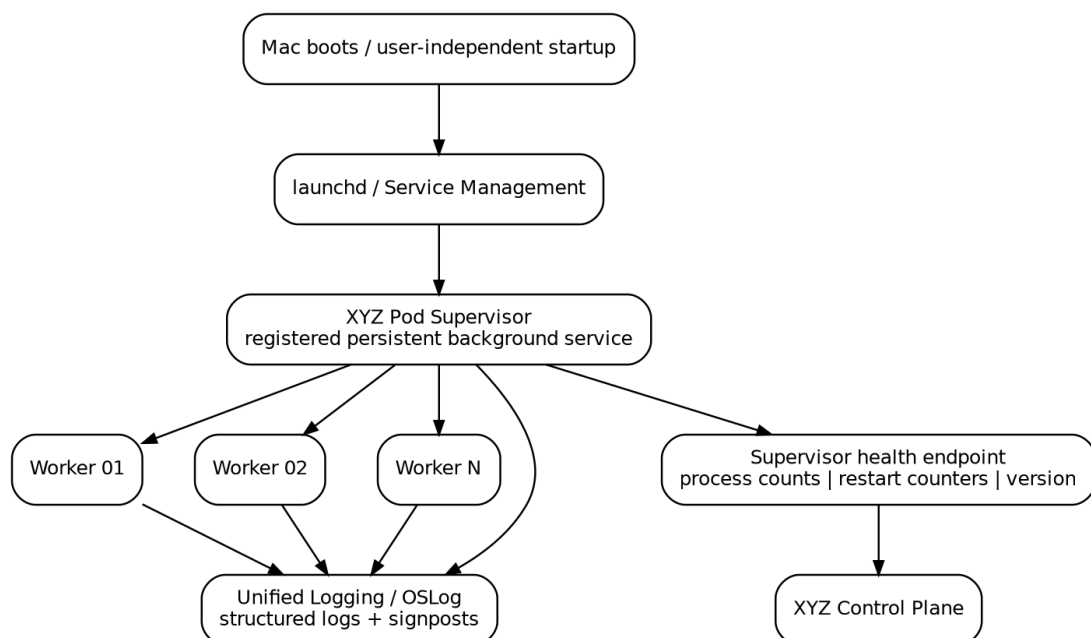
Capability record example

A production registry should record **observed capabilities**, not only expected ones:

```
observed_device_capabilities:
  device_id: phone-007
  udid: "<inventory value>"
  ios_version: "26.2"
  ios_build: "<build>"
  pod_agent_version: "1.4.x"
  pymobiledevice3_version: "10.2.1"
  transport:
    lockdown: true
    userspace_rsd: true
  services:
    display_hevc: true
    hid: true
    device_info: false
    pasteboard: false
  observed_at: "2026-08-27T..."
```

A version matrix remains useful for planning, but the runtime probe is the final truth for adapter activation.

V.5 Canonical architecture diagram - host service supervision



ARCH-HOST-SUPERVISION-001 - Mac pod process supervision

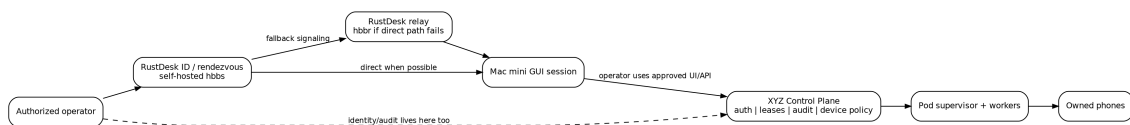
Diagram ID: ARCH-HOST-SUPERVISION-001

Host boot acceptance test

The Mac pod is not considered remotely recoverable until all of the following are demonstrated after an actual cold reboot:

1. macOS boots after power restoration under the selected configuration;
2. the pod supervisor starts without a human launching Terminal or an IDE;
3. inventory storage is reachable;
4. workers respawn for connected devices;
5. each worker re-establishes trust/transport/DDI/capability state as required;
6. the control plane marks devices healthy only after their readiness gates pass;
7. RustDesk or the selected operator-access path is usable after its required session/privacy gates;
8. the system produces an audit record for the recovery event.

V.6 Canonical architecture diagram - remote operator plane



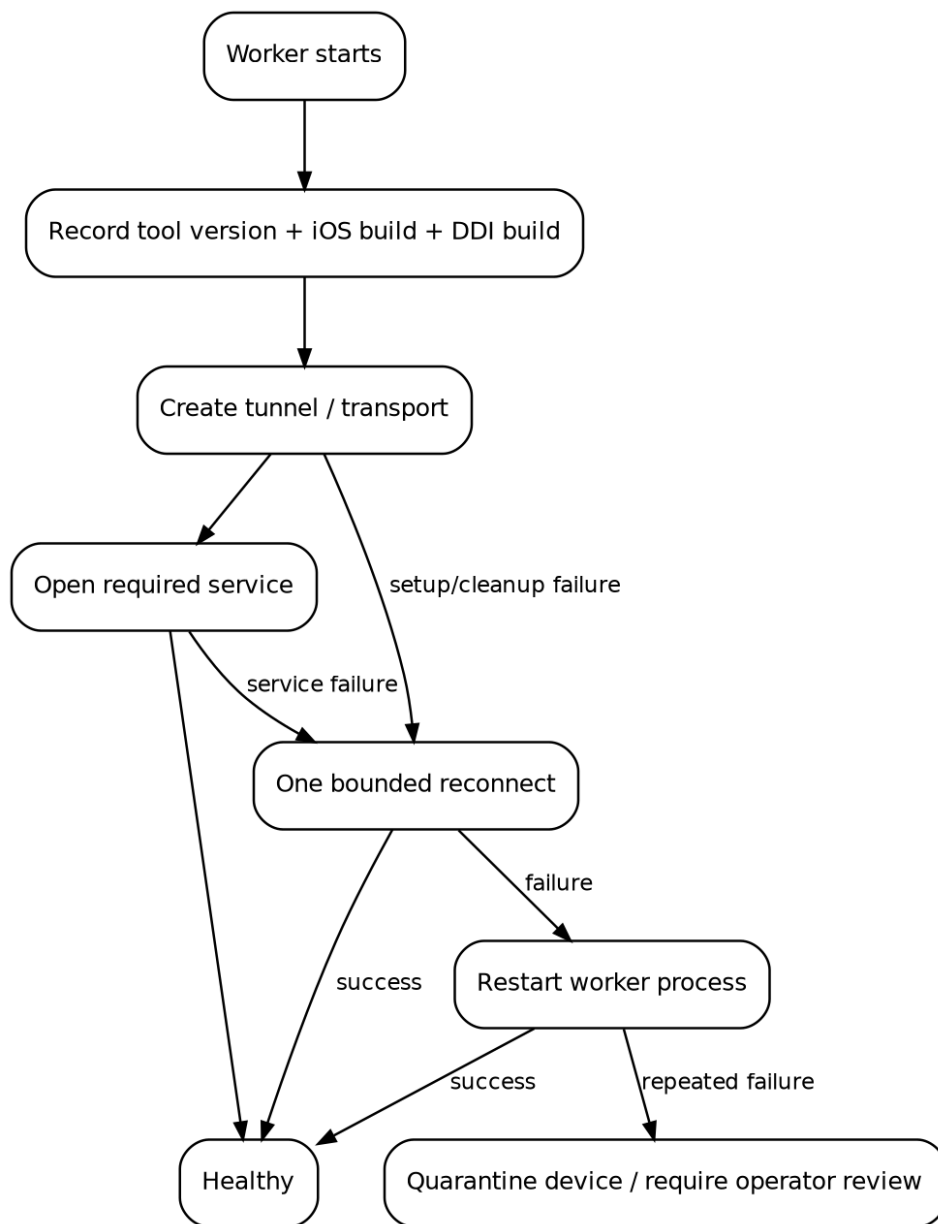
ARCH-REMOTE-OPS-001 - Remote operator plane

Diagram ID: ARCH-REMOTE-OPS-001

Feynman explanation

Remote desktop gives a person **presence on the Mac**. It does not replace device inventory, role-based client separation, task leases or action auditing. If the remote-desktop vendor changes, the farm's business authorization model should remain intact.

V.7 Canonical architecture diagram - bounded transport recovery



ARCH-TRANSPORT-RECOVERY-002 - Bounded transport recovery

Diagram ID: ARCH-TRANSPORT-RECOVERY-002

V.8 Database model update - separate observations from assertions

V1.4 recommends storing three categories of state rather than overwriting one `device.status` field:

Category	Example	Typical source	Lifetime
inventory_fact	serial, UDID, model	ADE/MDM/DDM + commissioning	long-lived
observed_runtime_state	tunnel alive, frame age, battery slope	pod worker	seconds/minutes
desired_state	required OS ring, required profiles, pod assignment	control plane / MDM policy	until changed

The reconciliation engine compares them. Example:

- desired OS: 27.0
- managed-device reported OS: 26.5
- worker reported device reachable: true

Result: **reachable but noncompliant**, not simply “healthy” or “offline.”

V.9 Version/capability canary rule

Before changing any of these on a production pod:

- macOS major/minor version;
- pymobiledevice3 version;
- Python runtime;
- DDI artifact/build;
- RustDesk client/service version;
- Apple device OS version used by a meaningful fraction of the fleet;

promote the new combination through a canary sequence:

```
1 device -> 2 devices -> representative 5-device pod -> fleet rollout
```

The canary must exercise:

- cold boot;
- unplug/replug;
- worker restart;
- display + HID together;
- DDI readiness;
- capability probe results;
- RustDesk reconnect after reboot;
- managed-device status reconciliation;
- at least one soak window.

V.10 New acceptance tests

TEST-DDM-RECON-001 - managed truth reconciliation

Goal: prove that incremental DDM changes and a later full status report converge the control-plane inventory to the device's reported managed state.

Pass: missed/duplicated updates do not create permanent drift; a full report replaces stale state correctly.

Fail: the control plane treats incremental arrays as complete snapshots or permanently diverges after a missed report.

TEST-CAP-PROBE-001 - runtime capability discovery

Goal: prove the worker can classify supported/unsupported services without crashing when an expected service is absent.

Pass: missing optional services produce a degraded capability set; required-service absence prevents READY with a precise reason.

Fail: OS-version assumptions cause repeated startup crashes or false READY state.

TEST-HOST-BOOT-002 - unattended host recovery

Goal: prove the Mac returns from full power loss to usable pod state without local keyboard/mouse intervention.

Pass: supervisor, workers, telemetry and remote-operator path return within the documented recovery objective.

Fail: any critical layer requires launching Terminal/IDE manually or leaves the pod apparently healthy while workers are absent.

TEST-RD-MAC-001 - RustDesk macOS privacy/launch path

Goal: validate the **supported** RustDesk service/install method on the exact macOS version.

Pass: after reboot, the authorized remote operator receives video and permitted input, including the intended pre-login/login-window behavior if required.

Fail: connected-with-black-screen, missing Accessibility, or launch-context/TCC problems prevent operation.

TEST-TRANSPORT-TEARDOWN-001 - repeated worker lifecycle

Goal: expose tunnel/event-loop cleanup leaks before 20-device deployment.

Method: repeatedly create transport -> open services -> close -> restart worker across many cycles while monitoring process count, open sockets, memory and reconnect time.

Pass: bounded resource use and stable recovery distribution.

Fail: cleanup hangs, orphan tasks, increasing memory/socket count or increasing recovery time.

V.11 V1.4 evidence register

ID	Source	Class	Used for
S-V14-01	Apple StatusReport documentation	VERIFIED-FIRST-PARTY	incremental status + periodic full safety report
S-V14-02	Apple declarative management scale/data-model article	VERIFIED-FIRST-PARTY	status categories and server reconciliation model
S-V14-03	Apple StatusDeviceSerialNumber	VERIFIED-FIRST-PARTY	serial inventory
S-V14-04	Apple StatusDeviceUDID	VERIFIED-FIRST-PARTY	UDID inventory availability
S-V14-05	Apple StatusDeviceModelIdentifier	VERIFIED-FIRST-PARTY	hardware model identifier
S-V14-06	Apple OS version/build status items	VERIFIED-FIRST-PARTY	software inventory
S-V14-07	Apple StatusDeviceBatteryHealth	VERIFIED-FIRST-PARTY	managed battery health
S-V14-08	Apple StatusDeviceSystemHealth	VERIFIED-FIRST-PARTY / BETA	component health where supported
S-V14-09	Apple managed-app status documentation	VERIFIED-FIRST-PARTY	installed/managed app state
S-V14-10	pymobiledevice3 issue #1744	REPRODUCIBLE-ISSUE	service absence despite healthy RSD/DDI path
S-V14-11	pymobiledevice3 releases	MAINTAINED-REPO	service/version behavior changes and tunnel retry fixes
S-V14-12	Apple Service Management / SMapService	VERIFIED-FIRST-PARTY	launch agents/daemons
S-V14-13	Apple persistent background-process guidance	VERIFIED-FIRST-PARTY	preferred persistent service pattern
S-V14-14	Apple launchd daemon/agent guide	VERIFIED-FIRST-PARTY / ARCHIVED-GUIDE	KeepAlive/on-demand service lifecycle background
S-V14-15	Apple Logging documentation	VERIFIED-FIRST-PARTY	unified logging and signposts
S-V14-16	Apple OSLog framework	VERIFIED-FIRST-PARTY	historical log reading
S-V14-17	RustDesk Server OSS documentation	VENDOR-DOC	hbbs/hbbr roles

ID	Source	Class	Used for
S-V14-18	RustDesk self-host documentation	VENDOR-DOC	direct vs relay architecture; Pro control features
S-V14-19	RustDesk macOS client documentation	VENDOR-DOC	Accessibility/Screen Recording/Input Monitoring
S-V14-20	RustDesk macOS auto-start/MDM notes	VENDOR/MAINTAINED-WIKI	service setup does not grant privacy permissions
S-V14-21	RustDesk discussion #15364	COMMUNITY-ISSUE	macOS 26 TCC launch-context risk to test
S-V14-22	pymobiledevice3 issue #1756	REPRODUCIBLE-ISSUE	userspace tunnel cleanup hang
S-V14-23	pymobiledevice3 issue #1742	REPRODUCIBLE-ISSUE	reconnect behavior after unplug/replug
S-V14-24	pymobiledevice3 releases v10.x	MAINTAINED-REPO	active tunnel retry/error fixes
S-V14-25	pymobiledevice3 mobile_image_mounter.py	MAINTAINED-REPO	personalized DDI/TSS/version behavior

URLs

- S-V14-01: <https://developer.apple.com/documentation/devicemanagement/statusreport>
- S-V14-02: <https://developer.apple.com/documentation/devicemanagement/leveraging-the-declarative-management-data-model-to-scale-devices>
- S-V14-03: <https://developer.apple.com/documentation/devicemanagement/statusdeviceserialnumber>
- S-V14-04: <https://developer.apple.com/documentation/devicemanagement/statusdeviceudid>
- S-V14-05: <https://developer.apple.com/documentation/devicemanagement/statusdevicemodelidentifier>
- S-V14-06: <https://developer.apple.com/documentation/devicemanagement/statusdeviceoperatingsystemversion>
- S-V14-07: <https://developer.apple.com/documentation/devicemanagement/statusdevicebatteryhealth>
- S-V14-08: <https://developer.apple.com/documentation/devicemanagement/statusdevicesystemhealth>
- S-V14-09: <https://developer.apple.com/documentation/devicemanagement/processing-status-for-managed-apps>
- S-V14-10: <https://github.com/doronz88/pymobiledevice3/issues/1744>
- S-V14-11: <https://github.com/doronz88/pymobiledevice3/releases>
- S-V14-12: <https://developer.apple.com/documentation/servicemanagement/smappservice>
- S-V14-13: <https://developer.apple.com/documentation/appkit/managing-ongoing-background-processes-in-your-mac>
- S-V14-14: <https://developer.apple.com/library/archive/documentation/MacOSX/Conceptual/BPSystemStartup/Chapters/CreatingLaunchdJobs.html>
- S-V14-15: <https://developer.apple.com/documentation/os/logging/>

- S-V14-16: <https://developer.apple.com/documentation/oslog>
- S-V14-17: <https://rustdesk.com/docs/en/self-host/rustdesk-server-oss/>
- S-V14-18: <https://rustdesk.com/docs/en/self-host/>
- S-V14-19: <https://rustdesk.com/docs/en/client/mac/>
- S-V14-20: <https://github.com/rustdesk/rustdesk/wiki/macOS-Auto%E2%80%90Start-Service-Setup-%28for-Remote%E2%80%90MDM-Deployment%29>
- S-V14-21: <https://github.com/rustdesk/rustdesk/discussions/15364>
- S-V14-22: <https://github.com/doronz88/pymobiledevice3/issues/1756>
- S-V14-23: <https://github.com/doronz88/pymobiledevice3/issues/1742>
- S-V14-24: <https://github.com/doronz88/pymobiledevice3/releases>
- S-V14-25: https://github.com/doronz88/pymobiledevice3/blob/master/pymobiledevice3/services/mobile_image_mounter.py

V1.4 CURRENT COVERAGE / UNDERSTANDING SCORE

These are INFERENCE engineering self-assessments, not independently measured scientific percentages.

Measurement	V1.3	V1.4 current
Research coverage	99.2%	99.45%
Conceptual understanding	98.9%	99.25%
Actual bench validation	74.4%	74.4%

Why the research score increased only slightly

V1.4 did not discover a completely new phone-control mechanism. It closed **operational architecture** gaps: managed truth reconciliation, runtime feature discovery, Mac service supervision, host logging and the remote-operator plane. These are important, but the major physical-device questions were already mapped.

Why understanding remains below 100%

The remaining uncertainty is dominated by real hardware behavior:

- 5/10/20-device CoreDevice display + HID concurrency;
- aggregate USB hub data/power behavior and exact port mapping;
- long-duration userspace RSD/DDI stability on the chosen macOS/tool versions;
- the incoming hardware control-board coexistence path;
- exact media ingestion behavior under the client's final no-helper/helper-app policy;
- RustDesk/macOS unattended behavior on the exact production image;
- Dim's undocumented hands-on observations being converted into reproducible evidence.

Why bench validation remains unchanged

No new local Mac/phone experiment occurred in this web-research run. Documentation can improve the model and tests; it cannot prove the client's exact 20-device pod.

V1.5 RESEARCH LEDGER - FINALIZED WEB PASS - 27 AUGUST 2026

STATUS: DELIVERABLE RESEARCH CUT. Web/documentary research is substantially saturated for the current architecture. The **actual XYZ/Dim bench validation score remains below 80%** because no new local Mac/phone measurements were produced in this research run. This document is therefore a research-complete engineering handbook with explicitly OPEN bench gates, not a claim that the 20-device pod has been validated.

New findings from deep-research loop

V15-CLAIM-01 - Multi-device CoreMediaIO capture is real, but simultaneous session start can race

Evidence: Quern's 2026 physical-device implementation uses a long-running macOS CoreMediaIO preview process with independent windows per physical iPhone. Its developer notes report a framework-level race when multiple AVCaptureSessions are started simultaneously and say that staggering starts by about one second avoided the race.

Classification: EXTERNAL-MEASURED / MAINTAINED-OPEN-SOURCE

Engineering consequence: session startup must be serialized/staggered. start N sessions concurrently is not an acceptable production assumption.



Figure: ARCH-CMIO-MULTIDEVICE-002 - rendered from the canonical Mermaid topology preserved in the LLM Markdown.

Canonical ID: ARCH-CMIO-MULTIDEVICE-002

V15-CLAIM-02 - Released software independently confirms multi-device USB iOS capture, but not a 20-device ceiling

WireView's current Mac App Store description explicitly supports multiple connected devices and uses the same built-in USB iOS capture technology as QuickTime. Its current reliability notes describe real failure states: device listed but no frames, capture starting before video format settles, trust-state problems, and the need for explicit stale-frame detection.

Classification: EXTERNAL-PRODUCT / EXTERNAL-MEASURED

Engineering consequence: AVCaptureSession.isRunning or device enumeration alone is insufficient health evidence. A healthy capture requires increasing frame timestamps/byte counts within a bounded freshness window.

V15-CLAIM-03 - USB device count is constrained by endpoint/controller topology, not just physical port count

Cambrionix's current hub documentation gives a concrete baseline of 15 iPhone 13/15 devices on one SuperSync15 USB 2 topology. Its PDSync-C4 manual gives a baseline of 16 iPhone 15 devices through a standard USB host controller with four four-port hubs and warns that USB endpoint memory and hub depth are limiting resources. Cambrionix's modular-station material additionally describes architectures up to 96 devices from one host/BOSS using multiple managed hubs; this latter number is vendor architecture evidence, not validation of 96 simultaneous screen/control sessions.

Classification: VENDOR-MEASURED / VENDOR-CLAIM (scope-dependent)

Engineering consequence: the 20-device target must be split across host-bus branches and measured as an endpoint budget. A single giant downstream tree is not the default assumption.

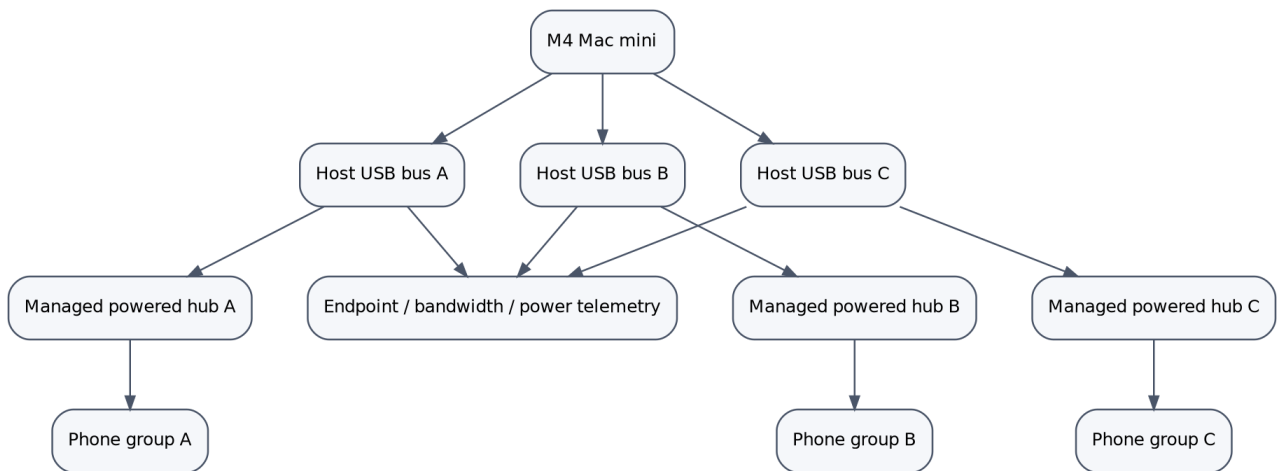


Figure: ARCH-HUB-ENDPOINT-001 - rendered from the canonical Mermaid topology preserved in the LLM Markdown.

Canonical ID: ARCH-HUB-ENDPOINT-001

V15-CLAIM-04 - M4 Mac mini ports are not one homogeneous USB resource

Apple documents two front USB-C ports and three rear Thunderbolt ports on the M4 Mac mini. Third-party hardware mapping by Twocanoes reports that the two front ports share a USB 3.1 bus while each of the three rear ports maps to a separate USB 3.1 bus; a 2026 ChargerLAB teardown independently found separate Thunderbolt/USB controllers and a Genesys hub for the front ports.

Classification: APPLE-FIRST-PARTY for port inventory; INDEPENDENT-HARDWARE-MEASUREMENT for bus topology.

Do not overclaim: Apple has not published an official '20 iPhone farm' bus-allocation specification.

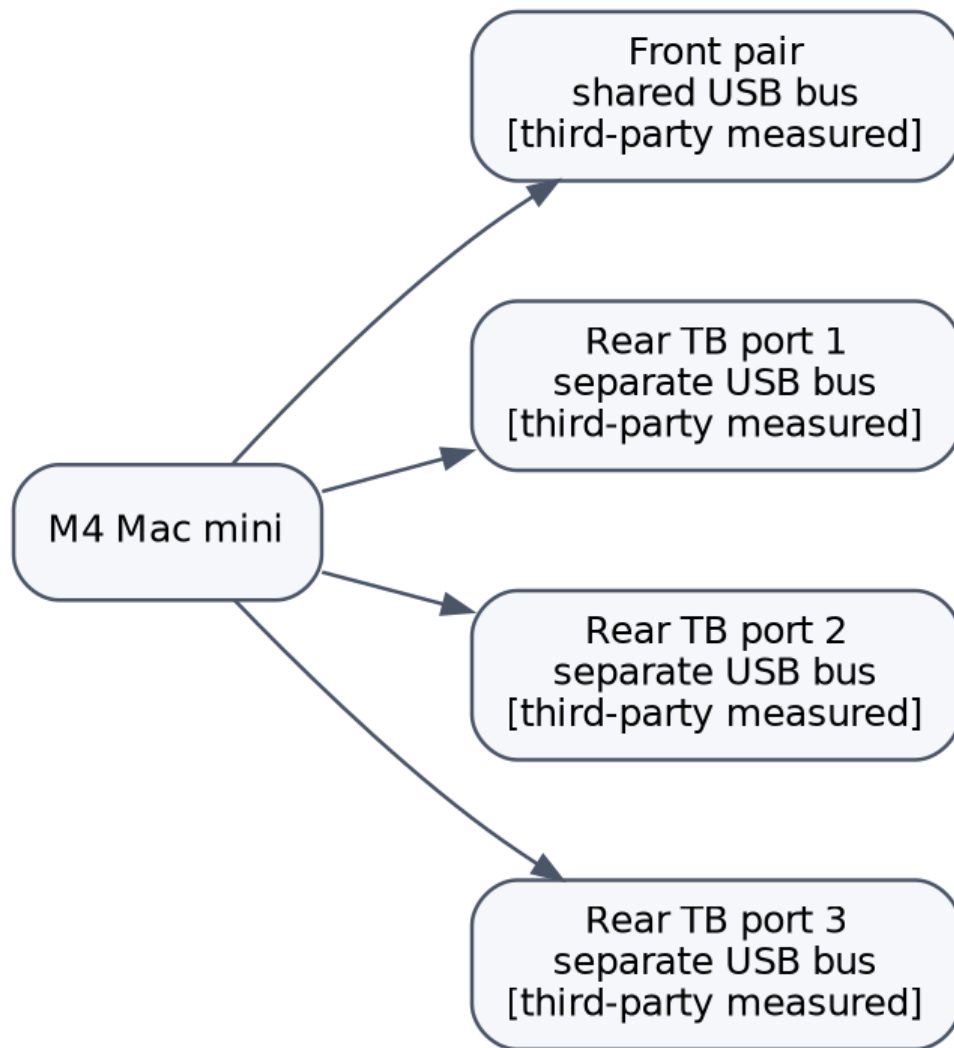


Figure: ARCH-M4-USB-BUSES-001 - rendered from the canonical Mermaid topology preserved in the LLM Markdown.

Canonical ID: ARCH-M4-USB-BUSES-001

V15-CLAIM-05 - The iMouse board path is more separable than earlier notes implied, but Mac USB display coexistence remains OPEN

Some3C's own current iMouse documentation says:

- one dedicated hardware board corresponds to one iPhone;
- the board is connected to the computer over a separate USB-C 'computer' port;
- the phone uses the vendor's included Lightning/USB-C OTG cable;
- iMouse XP is Windows-only;
- screen display is normally network mirroring/AirPlay and requires phone and PC on the same network;
- one screen-mirroring service can connect up to ten phones;
- the kernel runs as a Windows service even when the console exits.

The public documentation does **not** document a transparent USB data pass-through from the phone through the iMouse board to a Mac CoreMediaIO host.

Classification: VENDOR-DOCUMENTED + OPEN for pass-through coexistence.

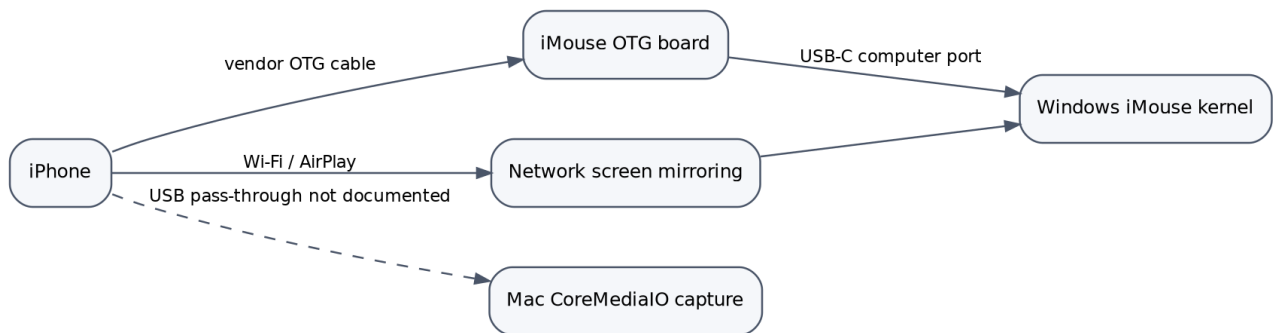


Figure: ARCH-IMOUSE-SPLIT-001 - rendered from the canonical Mermaid topology preserved in the LLM Markdown.

Canonical ID: ARCH-IMOUSE-SPLIT-001

Critical Gate: bench-test whether the exact incoming board permits simultaneous direct Mac USB capture. Do not infer this from separate PC and phone connectors.

V15-CLAIM-06 - The iMouse 'no app' claim does not mean 'no phone-side software/configuration'

The vendor's basic HID control path does not require an installed native app, but the documented media/shortcut functions require adding an iMouse Shortcut, changing Shortcuts Advanced permissions, configuring an IP address, running the shortcut once, and granting requested permissions. The vendor's shortcut API includes adding photos/videos to the phone album.

Classification: VENDOR-DOCUMENTED.

Engineering consequence: media ingestion via the iMouse ecosystem is a separate phone-side automation dependency and must be security-reviewed and acceptance-tested. It must not be described as equivalent to a zero-phone-side-component architecture.

V15-CLAIM-07 - iMouse host security posture is a production risk that requires isolation

Current iMouse XP setup documentation recommends adding the download directory to Windows Security exclusions and, in older setup instructions, trusting/exiting security products and even recommends uninstalling antivirus if it interferes. The iMouse kernel runs persistently as a Windows service and the console requires a vendor account.

Classification: VENDOR-DOCUMENTED SECURITY SIGNAL.

Engineering consequence: if iMouse is tested, use a dedicated isolated Windows bench/pod host, non-production credentials, outbound/network allowlists where feasible, immutable recovery image, and packet/process observation before any authorized client account is exposed to it.

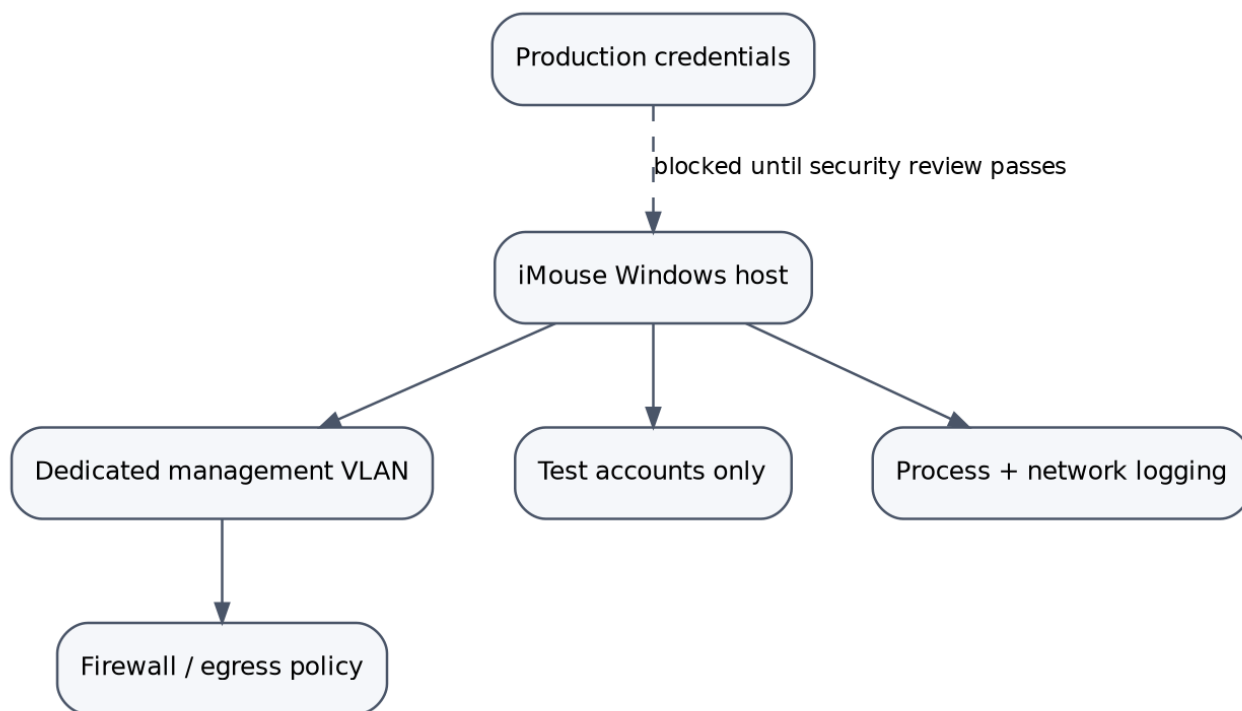


Figure: ARCH-IMOUSE-SECURITY-001 - rendered from the canonical Mermaid topology preserved in the LLM Markdown.

Canonical ID: ARCH-IMOUSE-SECURITY-001

V15-CLAIM-08 - macOS camera permission is a first-run gate for the native capture helper

Apple requires explicit camera/video-capture permission for each macOS app that uses AVFoundation capture devices, and the OS remembers the user's decision. Current macOS Privacy Preferences Policy Control documentation allows MDM to manage many TCC services, but for Camera it documents denial rather than silent pre-approval; the payload itself requires user approval. Apple's WWDC26 management update describes a newer consolidated per-app privacy-default mechanism, including Camera, for the 27-generation OSes; that is a future deployment option and should not be assumed for the macOS 26 pilot.

Classification: VERIFIED-FIRST-PARTY.

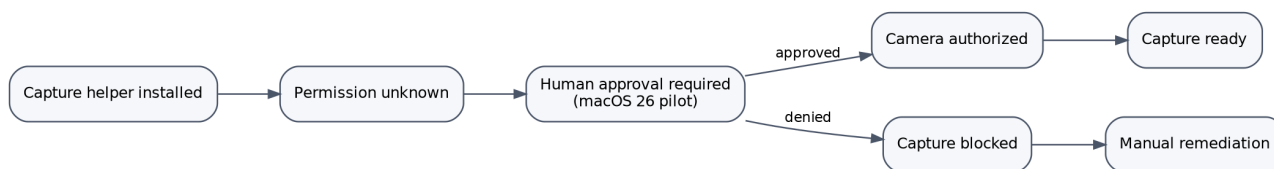


Figure: ARCH-MAC-TCC-CAPTURE-001 - rendered from the canonical Mermaid topology preserved in the LLM Markdown.

Canonical ID: ARCH-MAC-TCC-CAPTURE-001

V15-CLAIM-09 - Current Appium/XCUI Test architecture proves multi-real-device parallelism is supported, but with explicit per-device namespaces

Current Appium XCUI Test documentation says parallel tests on real devices are supported. Each session requires a unique device UDID, unique wdaLocalPort, unique mjpegServerPort when using MJPEG, and a unique derived-data path. Its current RemoteXPC tunnel manager is multi-device-aware and creates independent tunnels per device.

Classification: MAINTAINED-PROJECT-DOCUMENTATION.

Engineering consequence: WDA remains a credible fallback reference architecture for concurrency. The port/worker isolation pattern strongly supports the XYZ per-device-worker design even if XYZ ultimately uses CoreDevice HID instead of WDA.

V15-CLAIM-10 - Physical USB screen capture should use VideoDataOutput-style health rather than MovieFileOutput as the core observation channel

A 2026 Apple Feedback report reproduces a gamma/color-space bug when AVCaptureMovieFileOutput is attached to a USB-connected iOS capture session; AVCaptureVideoDataOutput was not affected in that report. AutoMobile's current physical-iOS capture work also uses a VideoDataOutput/BGRA helper path.

Classification: REPRODUCIBLE-ISSUE + MAINTAINED-OPEN-SOURCE.

Engineering consequence: the pod's live health/vision path should be based on frame delivery (AVCaptureVideoDataOutput or an equivalent low-level stream) rather than coupling health to movie-file recording.

New test gates

TEST-CMIO-START-STAGGER-001

1. Enumerate N capture devices.
2. Start one capture session.
3. Require first fresh frame before next start.
4. Repeat through 2/5/10/20.
5. Separately run a simultaneous-start stress test to characterize race rate.
6. Record time-to-first-frame, failures, stale-frame events, CPU, memory and recoveries.

TEST-USB-ENDPOINT-001

For each hub/bus branch record:

- physical upstream port;
- IOUSBHost location IDs;
- phone serial/UDID;
- negotiated speed;
- hub depth;
- enumerate success/failure;
- disconnect/reconnect behavior;
- charge-state and battery slope.

TEST-IMOUSE-PASSTHROUGH-001

With the exact incoming board and one sacrificial test phone:

- establish board control using vendor-supported topology;
- observe whether the phone simultaneously appears as a direct USB capture device on a Mac;
- if not, stop: board + native Mac capture are separate architectures;
- if yes, record USB tree, phone USB role, board firmware and exact cabling before promotion to project fact.

TEST-MAC-TCC-001

- clean/new macOS user or clean TCC state;
- launch signed capture helper;
- validate the required camera permission workflow;
- reboot;
- start helper through the intended launchd/service context;
- confirm capture remains authorized and yields fresh frames.

Research-source register - V1.5 working

ID	Source	Class	What it supports
S-V15-01	Apple Configurator - connect devices	VERIFIED-FIRST-PARTY	multiple simultaneous iOS devices through powered hubs/carts
S-V15-02	Cambrionix SuperSync15 current guide	VENDOR-MEASURED	baseline 15 iPhone 13/15 on one hub/host controller topology
S-V15-03	Cambrionix PDSync-C4 manual	VENDOR-MEASURED	16 iPhone 15 baseline; endpoint/hub-depth constraints
S-V15-04	Cambrionix ModIT datasheet	VENDOR-ARCHITECTURE	up to 96 provisioning devices via multiple managed hubs; not capture/control proof
S-V15-05	Quern 2026 development discussion	EXTERNAL-MEASURED	multi-device CoreMediaIO preview and start-race mitigation
S-V15-06	WireView App Store / Apple-bug ledger	EXTERNAL-PRODUCT	multi-device support and real stale/no-frame failure modes
S-V15-07	AutoMobile issue #2504	MAINTAINED-OPEN-SOURCE	physical iOS CoreMediaIO/AVFoundation VideoDataOutput implementation
S-V15-08	Apple Feedback report FB22281424	REPRODUCIBLE-ISSUE	MovieFileOutput gamma bug on USB iOS capture
S-V15-09	Apple AVFoundation authorization docs	VERIFIED-FIRST-PARTY	camera permission required and remembered
S-V15-10	Apple PPC payload docs	VERIFIED-FIRST-PARTY	current macOS TCC management limits; Camera deny behavior
S-V15-11	Apple WWDC26 app-management update	VERIFIED-FIRST-PARTY / FUTURE-OS	per-app privacy defaults including Camera on 27-generation OS

ID	Source	Class	What it supports
S-V15-12	Twocanoes M4 Mac mini port guide	INDEPENDENT-MEASUREMENT	front shared bus, rear separate USB buses
S-V15-13	ChargerLAB M4 Mac mini teardown	INDEPENDENT-HARDWARE	multiple USB/TB controllers and front hub architecture
S-V15-14	Some3C iPhone Farm Settings	VENDOR-DOC	iMouse board/PC connection, AirPlay mirroring, 10-phone mirroring service
S-V15-15	Some3C iMouse XP docs	VENDOR-DOC	Windows service/kernel, security exclusion recommendation
S-V15-16	Some3C API docs	VENDOR-DOC	WebSocket screen images, USB reset, Shortcuts/media functions
S-V15-17	Appium XCUITest parallel-tests docs	MAINTAINED-PROJECT	supported multi-real-device sessions and unique port namespaces
S-V15-18	Appium RemoteXPC tunnel docs	MAINTAINED-PROJECT	multi-device-aware independent tunnel registry

URLs

- S-V15-01: <https://support.apple.com/en-ca/guide/apple-configurator-mac/cad9d4b2211e/mac>
- S-V15-02: <https://www.cambrionix.com/blogs/blog/a-5-minute-guide-to-the-supersync15>
- S-V15-03: <https://downloads.cambrionix.com/documentation/en/PDS-C4-User-Manual.pdf>
- S-V15-04: https://www.cambrionix.com/wp-content/uploads/2021/02/ModIT_DataSheet.pdf
- S-V15-05: <https://github.com/quern-dev/quern/discussions/1>
- S-V15-06: <https://apps.apple.com/us/app/wireview/id6756973128?mt=12>
- S-V15-07: <https://github.com/kaeawc/auto-mobile/issues/2504>
- S-V15-08: <https://github.com/feedback-assistant/reports/issues/780>
- S-V15-09: <https://developer.apple.com/documentation/avfoundation/requesting-authorization-to-capture-and-save-media>
- S-V15-10: <https://support.apple.com/en-gb/guide/deployment/dep38df53c2a/web>
- S-V15-11: <https://support.apple.com/en-ie/guide/deployment/depd567c9ffa/1/web/1.0>
- S-V15-12: <https://twocanoes.com/knowledge-base/m4-mac-mini-port-guide/>
- S-V15-13: <https://www.chargerlab.com/teardown-of-apple-m4-mac-mini-a3238/>
- S-V15-14: <https://doc.some3c.com/iphone-farm-setup/iphone-farm-settings>
- S-V15-15: <https://doc.some3c.com/iphone-farm-setup/imouse-xp-new-version>
- S-V15-16: <https://doc.some3c.com/api-documentation>
- S-V15-17: <https://github.com/appium/appium-xcuitest-driver/blob/master/docs/guides/parallel-tests.md>
- S-V15-18: <https://github.com/appium/appium-xcuitest-driver/blob/master/docs/guides/remotexp-tunnels-real-devices.md>

V1.5 RESEARCH / UNDERSTANDING SCORE

These remain INFERENCE self-assessments, not measured scientific quantities.

Measurement	V1.4	V1.5 working
Research coverage	99.45%	99.80%
Conceptual understanding	99.25%	99.70%
Actual XYZ/Dim bench validation	74.4%	74.4%

Why actual bench validation did not move

This run added strong **external empirical evidence**: a released multi-device CoreMediaIO product, an open-source multi-device implementation with a documented start race, vendor USB endpoint/device-count measurements, current Appium parallel-device support, and current iMouse topology documentation. None of those measurements were produced on XYZ's Mac, hub, exact phone set, incoming control board, network, or production software image. They therefore cannot legitimately raise the project's actual bench-validation score.

Minimum evidence required to cross the 80% project threshold

The exact weighted score should be recalculated only after at least the following local measurements exist:

1. one-phone Mac capture/control run with exact versions and logs;
2. two-phone simultaneous run proving independent sessions and recovery;
3. five-phone continuous run with fresh-frame, input, power and reconnect telemetry;
4. at least one multi-hour soak window;
5. exact iMouse/control-board coexistence result (pass or fail is both useful evidence);
6. USB branch/location mapping on the chosen powered hub;
7. remote-reboot/recovery rehearsal on the intended Mac image.

If these pass with recorded artifacts, the 80% threshold can be recalculated from actual evidence rather than estimate inflation.

V1.5 FINAL COMPATIBILITY / ADVERSARIAL AUDIT - 27 AUGUST 2026

Precedence: This section supersedes narrower V1.5 working assumptions where it explicitly says so. Claims remain scoped to authorized, organization-owned device-fleet engineering. Capacity at 5/10/20 devices remains OPEN until reproduced on XYZ's intended Mac, hubs, cables, phones and production image.

V15-FINAL-01 - Correction: do not state a general iOS 18 floor for Device Hub physical-device input

Status: CORRECTION + VERIFIED-FIRST-PARTY

Correction: a previous research narrative stated that Xcode 27 Device Hub keyboard/pointer interaction on physical iPhones required iOS 18+. I cannot verify that as a general physical-device minimum from Apple's current Device Hub documentation. Apple states that Device Hub manages **physical and simulated devices** and that, when an app is run on a physical or simulated device, Device Hub opens a compact device screen that can be interacted with using Mac controls. Xcode 27 beta release notes contain pre-iOS-18 input fixes that are explicitly about **simulators**, not a universal physical-device floor.

Engineering consequence: treat Device Hub as a first-party **human interactive control** candidate for supported physical run destinations, but do not infer:

- a public automation/touch-injection API;
- 20-device simultaneous interactive capacity;
- an iOS-version floor beyond what the exact Xcode/device combination demonstrates in the bench.

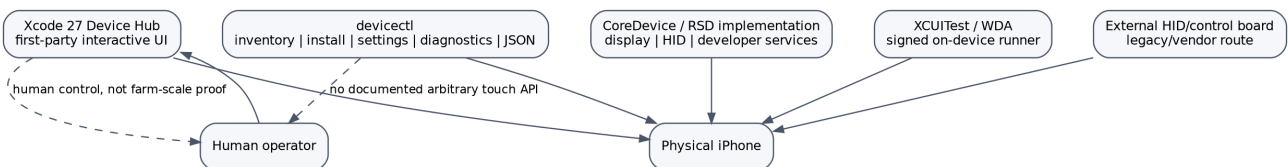


Figure: ARCH-IOS-CONTROL-LANES-002 - rendered from the canonical Mermaid topology preserved in the LLM Markdown.

Canonical ID: ARCH-IOS-CONTROL-LANES-002

V15-FINAL-02 - `devicectl` is a management/diagnostic automation surface, not a verified arbitrary touch surface

Status: VERIFIED-FIRST-PARTY

Xcode 27 release notes confirm structured `devicectl --json-output` to stdout. Apple's Device Hub material positions `devicectl` alongside device inventory, application management, settings and diagnostics. No current Apple source located in this research pass documents arbitrary scripted tap/swipe injection through `devicectl`.

Do not infer: `devicectl` replaces HID, Device Hub human control, CoreDevice HID, or WDA for arbitrary UI input.

V15-FINAL-03 - CoreDevice/RSD transport rules are now stable enough to encode as a version-aware prerequisite layer

Status: MAINTAINED - PROJECT - DOCUMENTATION

Current pymobiledevice3 documentation states:

- iOS 17+ developer services use CoreDevice/RemoteXPC/RSD flows;
- iOS 17.4+ over USB on macOS/Linux/Windows can use the default no-root, in-process userspace tunnel;
- iOS 17.0-17.3.1 is routed to privileged tunnel by default because the robust USB CoreDeviceProxy path is not available there;
- the userspace path supports host-initiated developer services plus device-initiated AV/HID paths including video display and HID gestures;
- the userspace tunnel address is process-local.

Appium's current XCUITest RemoteXPC implementation independently uses multi-device tunnel registries and unique per-device state, but its driver documentation currently applies its RemoteXPC tunnel mechanism to real iOS/tvOS 18+ devices. This is an **Appium implementation boundary**, not evidence that CoreDevice itself begins at iOS 18.

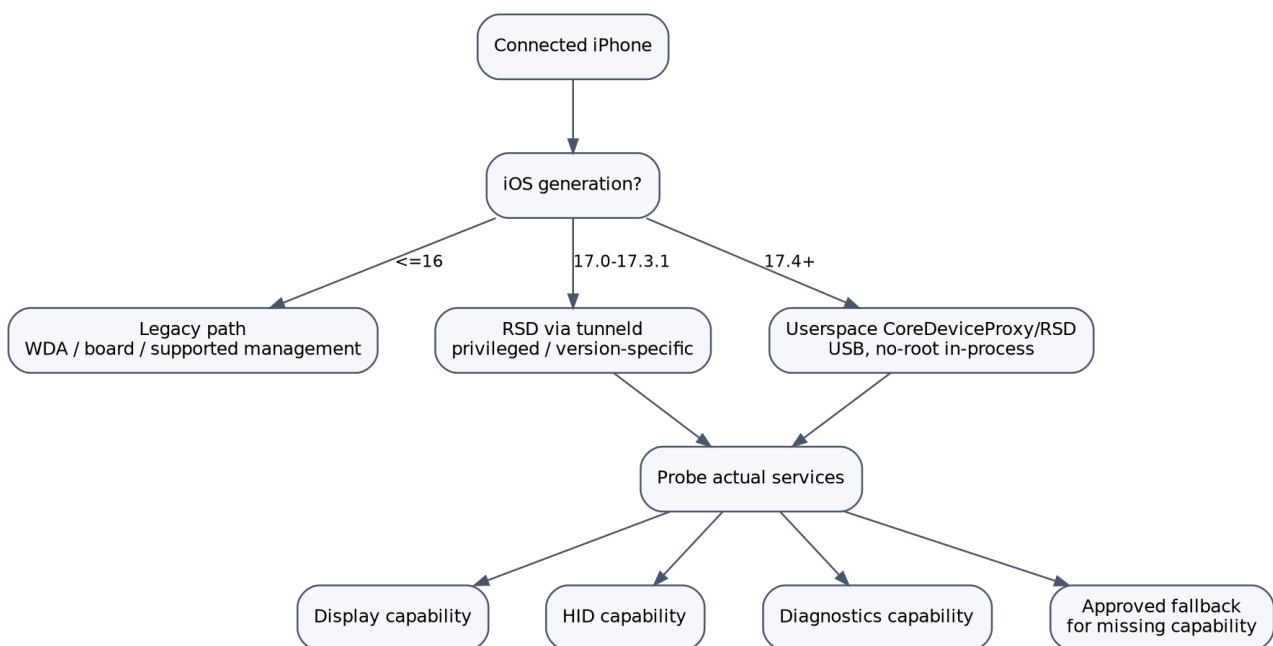


Figure: ARCH-IOS-RSD-VERSION-002 - rendered from the canonical Mermaid topology preserved in the LLM Markdown.

Canonical ID: ARCH-IOS-RSD-VERSION-002

V15-FINAL-04 - Capture identity and capture health must be separate records

Status: VERIFIED - FIRST - PARTY + EXTERNAL - MEASURED

Apple documents `AVCaptureDevice.uniqueID` as a persistent identifier on one Mac across device disconnects, app restarts and Mac reboots. This is the correct anchor for the **capture-device identity** side of the inventory map.

External 2026 CoreMediaIO implementations show that a device can remain enumerated while producing stale/no frames, and Quern reports a multi-session start race mitigated by staggered starts.

Engineering consequence: a production capture record needs both **identity** and **freshness**.

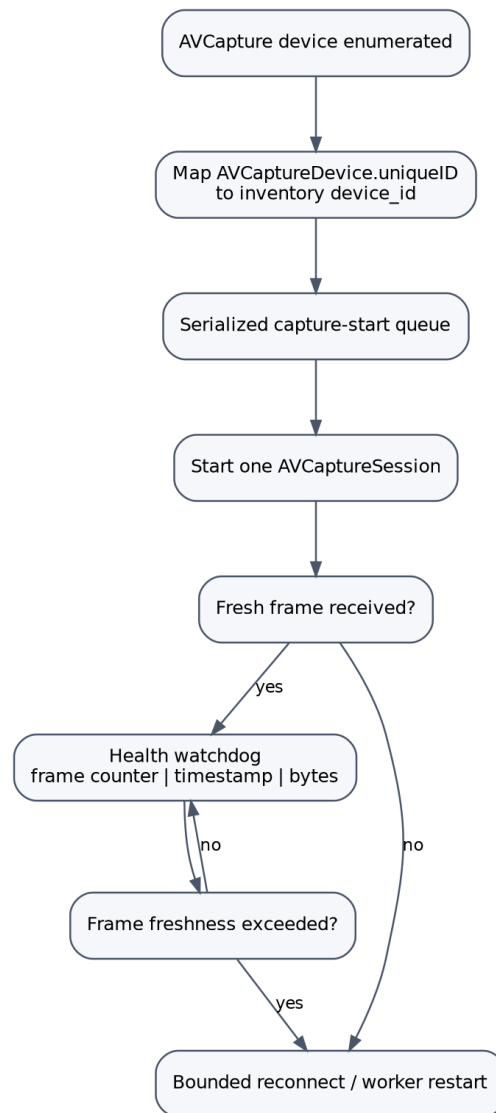


Figure: ARCH-CAPTURE-IDENTITY-HEALTH-003 - rendered from the canonical Mermaid topology preserved in the LLM Markdown.

Canonical ID: ARCH-CAPTURE-IDENTITY-HEALTH-003

V15-FINAL-05 - One powered hub is not a proof of a 20-phone topology

Status: VENDOR-MEASURED + OPEN FOR XYZ

Cambrionix's current SuperSync15 documentation gives a baseline of **15 iPhone 15 devices via USB 2** through one standard USB host-controller topology and explicitly warns that endpoint resources and hub depth matter. This is valuable external capacity evidence, but it is not a validation of 20 simultaneous CoreMediaIO display + CoreDevice/WDA input sessions on an M4 Mac mini.

Architecture rule: spread the pilot across identifiable host branches, measure endpoint enumeration, power/battery slope and frame/input health, and scale 1 -> 2 -> 5 -> 10 -> 20.

V15-FINAL-06 - The cleanest documented Photos ingestion route still requires phone-side code and user-granted permission

Status: VERIFIED-FIRST-PARTY

Apple PhotoKit provides `PHAccessLevel.addOnly`, which allows an app to add assets without general read access to the user's Photos library. Apple also exposes file-URL asset creation APIs for image and video assets. Therefore a small organization-owned helper app can implement a narrow, auditable **true Photos-library ingestion** path.

Do not infer: raw USB/AFC file copy alone creates a Photos-library asset.

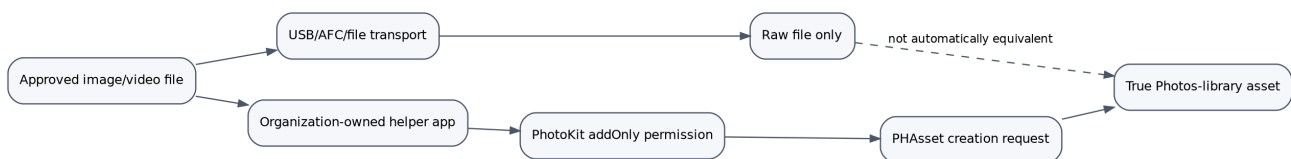


Figure: ARCH-MEDIA-ADDONLY-002 - rendered from the canonical Mermaid topology preserved in the LLM Markdown.

Canonical ID: ARCH-MEDIA-ADDONLY-002

V15-FINAL-07 - Supervision can make locked-USB behavior explicit; Return to Service can preserve eSIM data-plan state on supported OS versions

Status: VERIFIED-FIRST-PARTY

Apple's restrictions schema states that on supervised iOS devices, setting `allowUSBRestrictedMode` to false allows USB accessories to connect while the device is locked; Lockdown Mode overrides that policy. Apple also documents `PreserveDataPlan` for Return to Service on iOS 26.4+ / iPadOS 26.4+, allowing an eSIM data plan to be preserved where supported.

Engineering consequence: wired-accessory policy, physical security, lifecycle reset and cellular-plan lifecycle should be separate inventory/policy fields.

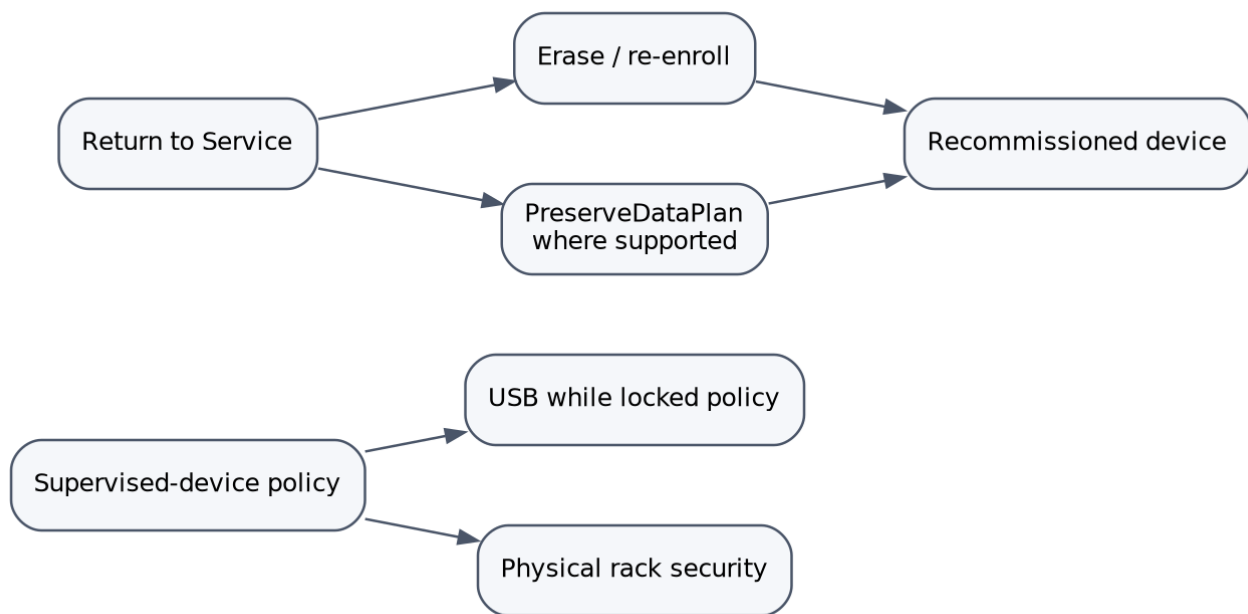


Figure: ARCH-USB-MDM-LIFECYCLE-002 - rendered from the canonical Mermaid topology preserved in the LLM Markdown.

Canonical ID: ARCH-USB-MDM-LIFECYCLE-002

V15-FINAL-08 - Remote desktop remains an operator-access plane with explicit macOS privacy gates

Status: VENDOR-DOCUMENTED + INFERENCE

RustDesk documents self-hosted hbbs rendezvous/signaling and hbbr relay services. Its macOS client requires Accessibility and Screen Recording, and may require Input Monitoring. This confirms the existing rule: RustDesk can provide remote presence on the Mac, but XYZ's control plane must remain the authority for device leases, client separation, task policy and audit.



Figure: ARCH-REMOTE-OPS-003 - rendered from the canonical Mermaid topology preserved in the LLM Markdown.

Canonical ID: ARCH-REMOTE-OPS-003

V15-FINAL-09 - Final adversarial architecture decision tree

Status: INFERENCE

The research now supports a layered decision rather than one universal iPhone-control stack:

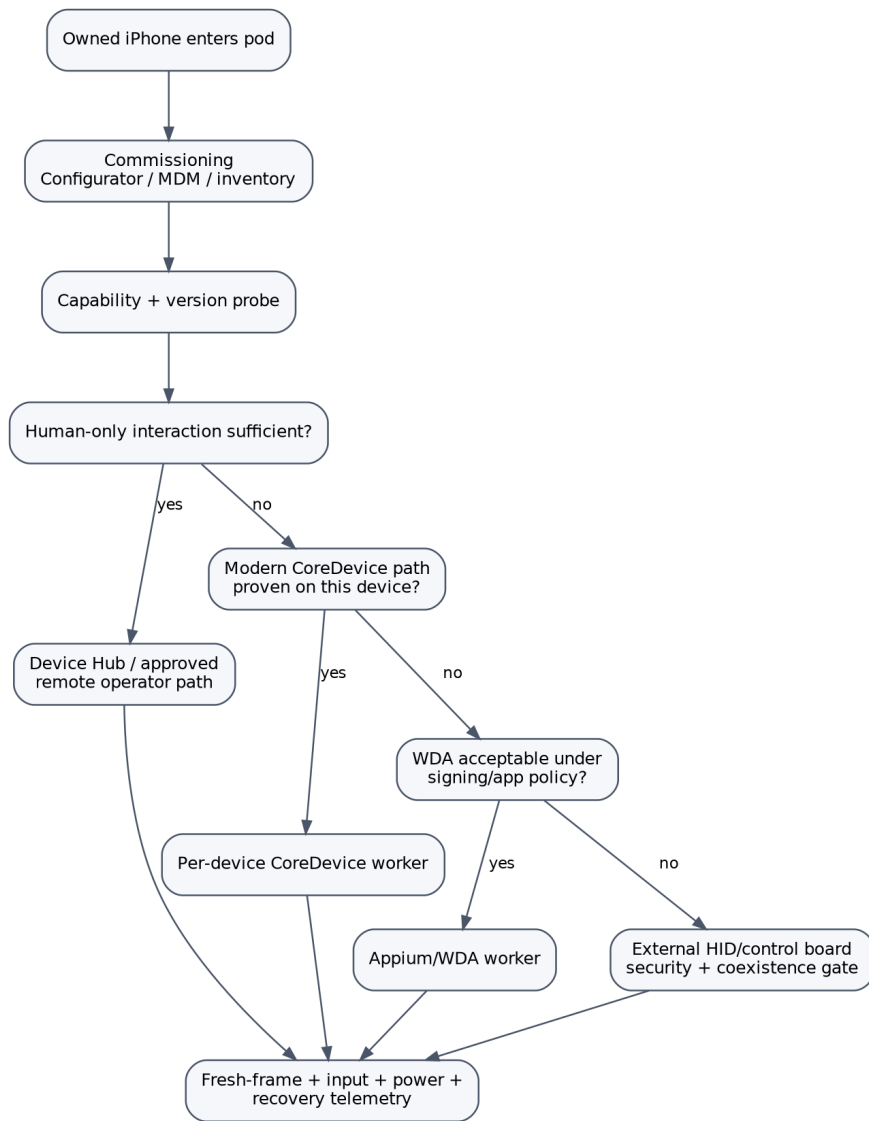


Figure: ARCH-FINAL-CONTROL-DECISION-001 - rendered from the canonical Mermaid topology preserved in the LLM Markdown.

Canonical ID: ARCH-FINAL-CONTROL-DECISION-001

V15-FINAL-10 - Research saturation versus bench validation

Status: INFERENCE

Web and documentary research is now near saturation for the current design. The largest remaining unknowns are physical and should not be converted into facts by additional searching:

- simultaneous 5/10/20-device display + input stability;
- exact M4 Mac mini USB branch behavior with the selected powered hubs;
- battery slope and rack thermal behavior under sustained load;
- unplug/replug and host-reboot recovery;
- the incoming iMouse/control-board pass-through/coexistence result;
- Dim's exact tested software/version/failure evidence;
- final helper-app/no-helper media policy.

Self-assessment - not a scientific metric

Measurement	V1.5 final research cut
Research coverage	99.80%
Conceptual understanding	99.70%
Actual XYZ/Dim bench validation	74.4%

The research and understanding percentages are qualitative engineering self-assessments. The bench figure is intentionally unchanged because this web pass did not produce local XYZ/Dim measurements.

V1.5 final source additions

ID	Source	Class	Supports
S-V15F-01	Apple Device Hub documentation	VERIFIED-FIRST-PARTY	physical/simulated device management and Mac-control interaction
S-V15F-02	Apple WWDC26 Device Hub session	VERIFIED-FIRST-PARTY	Device Hub control/configure workflow and devicectl context
S-V15F-03	Xcode 27 release notes	VERIFIED-FIRST-PARTY	devicectl JSON output; Device Hub current known/resolved issues
S-V15F-04	pymobiledevice3 iOS 17+ tunnels guide	MAINTAINED-PROJECT	iOS 17.4+ USB userspace RSD, 17.0-17.3.1 tunnel path, AV/HID support
S-V15F-05	Appium RemoteXPC tunnels guide	MAINTAINED-PROJECT	current iOS18+ driver tunnel use and multi-device registry
S-V15F-06	Apple AVCaptureDevice.uniqueID	VERIFIED-FIRST-PARTY	persistent capture-device identity on one Mac
S-V15F-07	Apple Device Management Restrictions	VERIFIED-FIRST-PARTY	allowUSBRestrictedMode behavior
S-V15F-08	Apple ReturnToServiceResponse	VERIFIED-FIRST-PARTY	PreserveDataPlan on supported versions
S-V15F-09	Apple StatusDeviceBatteryHealth	VERIFIED-FIRST-PARTY	managed battery-health values
S-V15F-10	Apple PhotoKit addOnly + asset creation	VERIFIED-FIRST-PARTY	narrow true Photos-library ingestion path
S-V15F-11	Cambrionix SuperSync15 scalability	VENDOR-MEASURED	15 iPhone 15 baseline and endpoint/hub-depth warning
S-V15F-12	RustDesk macOS + self-host docs	VENDOR-DOC	macOS privacy gates; hbbs/hbbr architecture
S-V15F-13	Some3C iMouse current product docs	VENDOR-DOC	Windows-only, one board per phone, OS/model support claims

URLs - V1.5 final additions

- S-V15F-01: <https://developer.apple.com/documentation/xcode/device-hub>
- S-V15F-02: <https://developer.apple.com/videos/play/wwdc2026/260/>

- S-V15F-03: <https://developer.apple.com/documentation/xcode-release-notes/xcode-27-release-notes>
- S-V15F-04: <https://github.com/doronz88/pymobiledevice3/blob/master/docs/guides/ios17-tunnels.md>
- S-V15F-05: <https://github.com/appium/appium-xcuitest-driver/blob/master/docs/guides/remotexpc-tunnels-real-devices.md>
- S-V15F-06: <https://developer.apple.com/documentation/avfoundation/avcapturedevice/uniqueid>
- S-V15F-07: <https://developer.apple.com/documentation/devicemanagement/restrictions>
- S-V15F-08: <https://developer.apple.com/documentation/devicemanagement/returntoserviceresponse>
- S-V15F-09: <https://developer.apple.com/documentation/devicemanagement/statusdevicebatteryhealth>
- S-V15F-10a: <https://developer.apple.com/documentation/photos/phaccesslevel/addonly>
- S-V15F-10b: [https://developer.apple.com/documentation/photos/phassetchangerequest/creationrequestforassetfromimage\(atfileurl:\)](https://developer.apple.com/documentation/photos/phassetchangerequest/creationrequestforassetfromimage(atfileurl:))
- S-V15F-11: <https://knowledge.cambrionix.com/Content/Products/SS15/User-Manual/Scalability.htm>
- S-V15F-12a: <https://rustdesk.com/docs/en/client/mac/>
- S-V15F-12b: <https://rustdesk.com/docs/en/self-host/>
- S-V15F-13: <https://some3c.com/products/iphone-ios-automation-control-board-otg>

V1.6 RESEARCH UPDATE

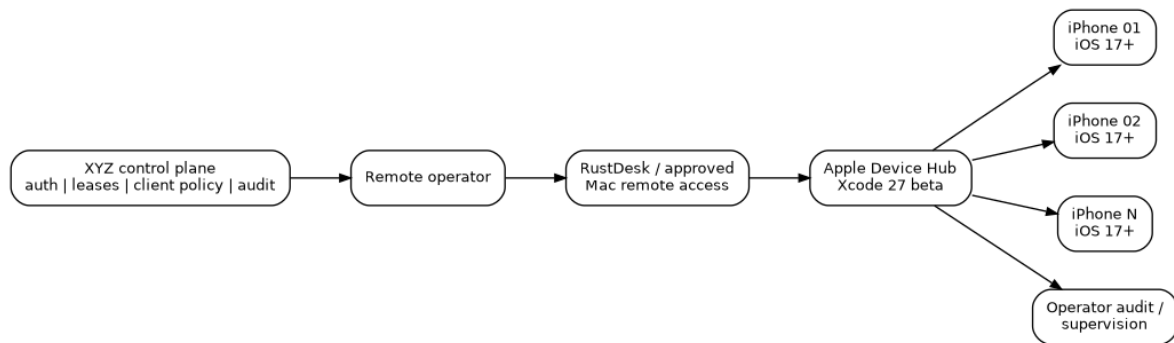
Apple Device Hub changes the human-control branch

This update adds a new first-party control candidate for modern iPhones. It does not replace the existing CoreDevice, WDA, iMouse, commissioning, networking, power, security, and control-plane material in V1.5; those sections remain authoritative unless this appendix explicitly refines them.

Current status: Xcode 27 is still beta software. Xcode 27 beta 6 was released 24 August 2026 and supports on-device debugging on iOS 17 and later. Therefore Device Hub is a high-value bench candidate, not yet a stable production dependency.

1. Native Apple human-control lane

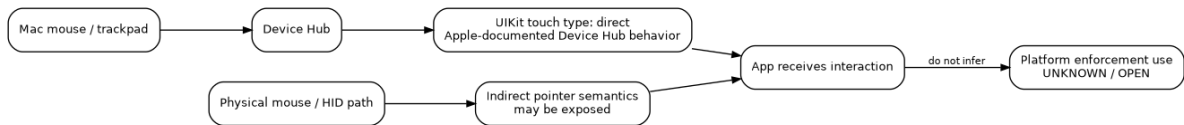
Apple describes Device Hub as a standalone app that ships alongside Xcode 27 and works with both simulated and physical devices. A paired physical device can expose a live screen in the canvas and be driven from Mac controls. Apple demonstrates click, drag, scroll, trackpad gestures, keyboard capture, Home, screenshots, rotation, device information, app/profile management, and diagnostics. The WWDC26 Device Hub session also says operators can open any number of devices in tabs or stand-alone compact windows. That is a UI organization capability, not a 20-device performance specification.



Canonical diagram ID: ARCH-NATIVE-HUMAN-CONTROL-001

2. Input semantics: Device Hub is not the same as physical HID

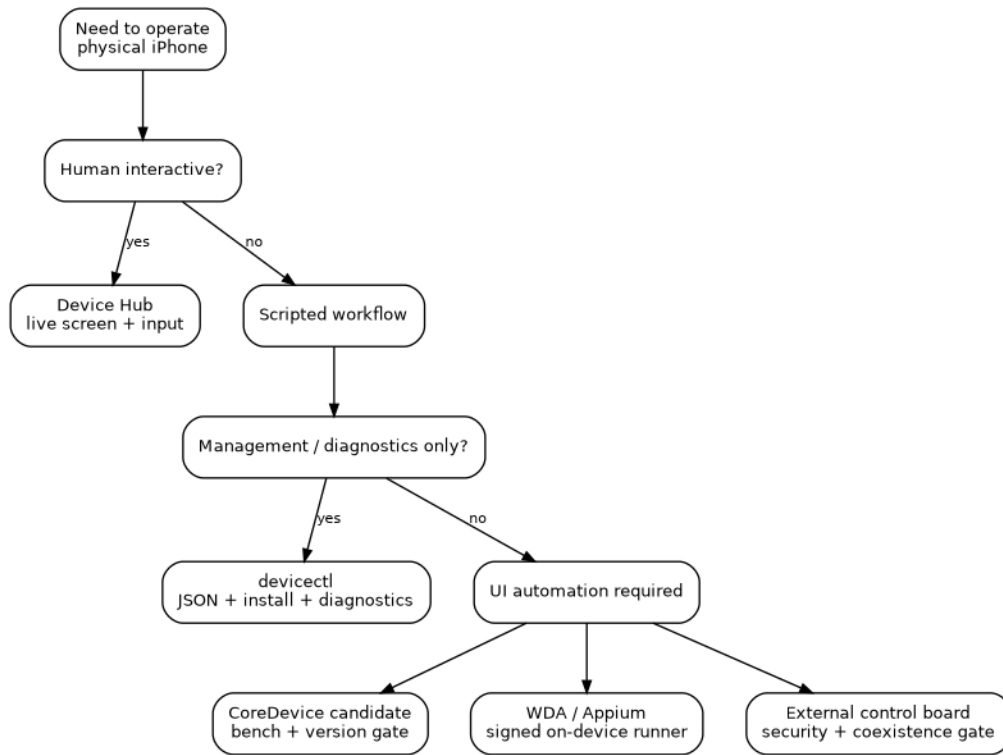
Xcode 27 beta release notes state that clicking with a Mac mouse or trackpad inside Device Hub produces a simulated finger touch of type `UITouch.TouchType.direct` rather than `UITouch.TouchType.indirectPointer`. Apple explicitly says this hybrid Device Hub behavior does not reflect a physical pointing device. This is materially different from an external mouse/HID route. It does not establish how any third-party platform classifies or enforces such input.



Canonical diagram ID: ARCH-INPUT-SEMANTICS-002

3. Human control versus scripted automation

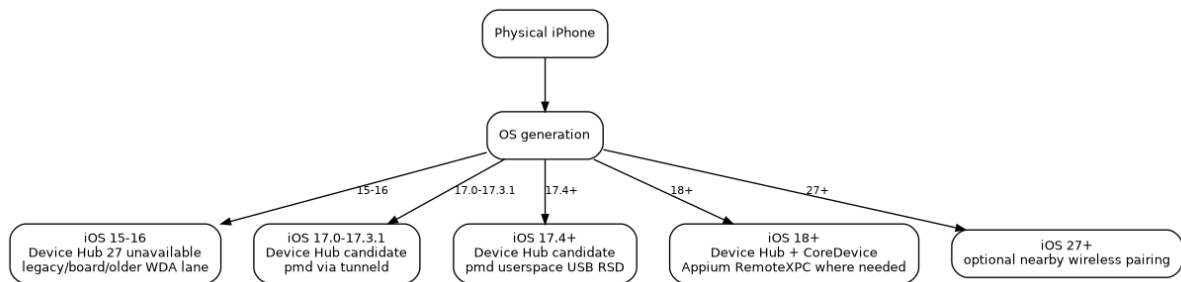
Device Hub is the graphical human-control lane. Apple positions devicectl for scripts and CI workflows such as device management, app installation, diagnostics, and structured JSON output. Current first-party documentation does not document arbitrary scripted touch/swipe injection through devicectl. If scripted third-party UI automation is required, the CoreDevice, WDA/Appium, or external-board branches remain separate engineering choices.



Canonical diagram ID: ARCH-HUMAN-VS-AUTOMATION-001

4. iOS generation split

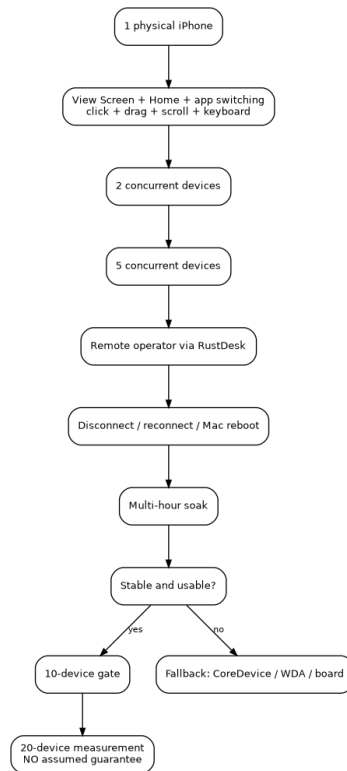
Xcode 27 beta 6 supports on-device debugging on iOS 17+. pymobiledevice3 currently treats iOS 17.4+ as the clean USB userspace CoreDevice/RSD case and 17.0-17.3.1 as a tunneled case. Appium currently uses its RemoteXPC tunnel integration for real iOS/tvOS 18+ features. Xcode 27 additionally adds nearby wireless pairing for iOS/iPadOS 27+. This makes a single universal control stack less attractive than version-aware capability probing.



Canonical diagram ID: ARCH-IOS-GENERATION-002

5. Device Hub acceptance gate

Because Device Hub is beta and Apple publishes no 5/10/20-phone latency or resource ceiling, it needs the same staged bench discipline as every other control path. The pilot should prove whole-device navigation, multiple simultaneous views, remote operation through the team's approved Mac access layer, disconnect/reconnect, reboot recovery, and multi-hour stability before scale is assumed.



Canonical diagram ID: TEST-DEVICE-HUB-SCALE-001

6. Architecture decision update

For compatible durable iPhones, the Mac-first pilot should now test Device Hub before custom control code. If Apple's own human-control console satisfies the operator workflow, the project can reserve CoreDevice/WDA/board engineering for automation or unsupported OS generations.

Need	Preferred first candidate	Why / caveat
Human remote operation, iOS 17+	Device Hub	First-party live screen and Mac controls; beta; scale OPEN
Scripted device/app management	devicectl / MDM / Configurator	Documented management/diagnostics; not arbitrary scripted touch
Scripted UI automation	CoreDevice or WDA/Appium	Requires bench/version gates; WDA installs signed runner
Older iOS 15-16	Legacy/WDA/board route	Xcode 27 Device Hub support floor is iOS 17
Bulk commissioning	Apple Configurator / MDM	Still stronger for supervision and repeatable provisioning

Key operational inference: the remote team may be able to use RustDesk only to reach the Mac, then use Apple Device Hub as the phone interaction console. This would keep phone interaction on an Apple tool while XYZ retains authorization, client isolation, leasing and auditing in its own control plane.

7. Current limitations and tests

- Xcode 27 / Device Hub is beta as of this research cut; treat every behavior as version-pinned and regression-tested.
- Apple states Device Hub can open any number of devices in windows/tabs, but publishes no simultaneous physical-iPhone performance ceiling.
- Device Hub does not support arbitrary two-finger touch injection; some gestures have Device Hub-specific handling.
- Keyboard capture and disconnect behavior have beta-specific known issues; include keyboard-mode cleanup in soak/reconnect testing.
- Device Hub direct-touch semantics must not be converted into claims about third-party platform detection or policy.
- Older iOS 15-16 devices fall outside Xcode 27 beta 6 on-device-debugging support and need another control path.
- Whole-device navigation on the exact social-app workflow must be proven on the bench; do not assume an app-under-development example covers every operational path.

8. Current research and validation score

Measurement	V1.5	V1.6
Research coverage	99.80%	99.90%
Conceptual understanding	99.70%	99.85%
Actual XYZ/Dim bench validation	74.4%	74.4%

These percentages are engineering self-assessments, not scientific measurements. Bench validation is intentionally unchanged because this pass produced documentation and external evidence, not measurements on XYZ hardware.

9. V1.6 source register

Apple - Device Hub

<https://developer.apple.com/documentation/xcode/device-hub>

Apple WWDC26 - Get the most out of Device Hub

<https://developer.apple.com/videos/play/wwdc2026/260/>

Apple - Xcode 27 beta 6 release notes

<https://developer.apple.com/documentation/xcode-release-notes/xcode-27-release-notes>

Apple - Xcode 27 beta 6 release page

<https://developer.apple.com/news/releases/?id=08102026g>

Apple - Managing devices in Device Hub

<https://developer.apple.com/documentation/xcode/managing-your-simulated-and-physical-devices-in-device-hub>

Apple - Content caching deployment

<https://support.apple.com/guide/deployment/intro-to-content-caching-depde72e125f/web>

pymobiledevice3 - iOS 17+ tunnels

<https://github.com/doronz88/pymobiledevice3/blob/master/docs/guides/ios17-tunnels.md>

pymobiledevice3 - releases

<https://github.com/doronz88/pymobiledevice3/releases>

Appium XCUITest - RemoteXPC tunnels

<https://github.com/appium/appium-xcuitest-driver/blob/master/docs/guides/remotexp-tunnels-real-devices.md>

Appium XCUITest - real device setup

<https://github.com/appium/appium-xcuitest-driver/blob/master/docs/getting-started/device-setup.md>

Appium XCUITest - parallel tests

<https://github.com/appium/appium-xcuitest-driver/blob/master/docs/guides/parallel-tests.md>

V1.7 - Final Adversarial Web Audit / Research Freeze

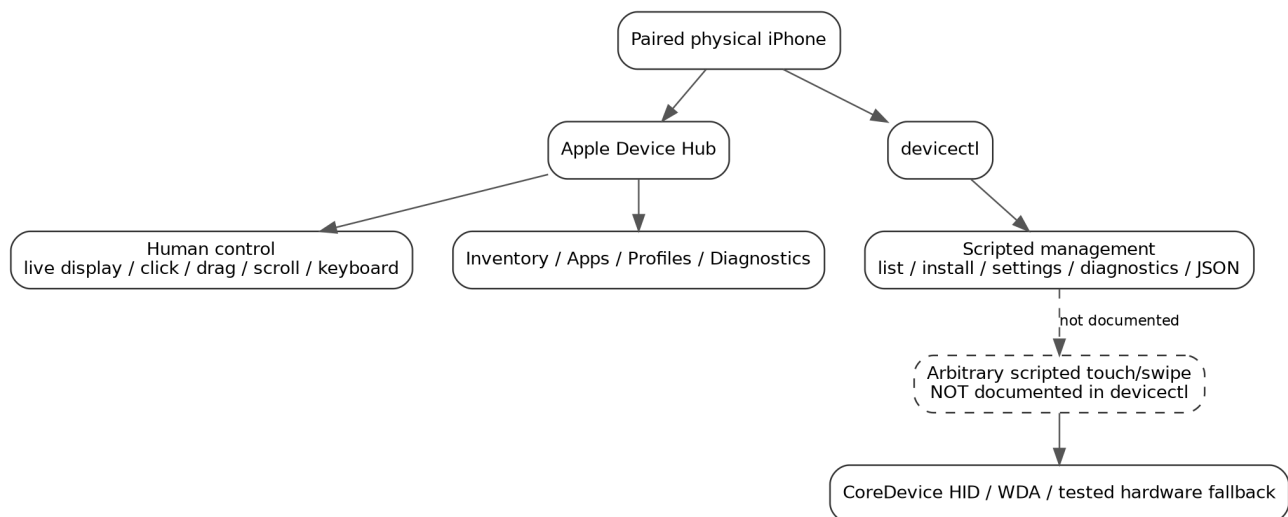
This appendix preserves V1.0-V1.6 and adds only corrections, adversarial findings, final architecture gates, and source-freeze rules.

Research conclusion

Broad web research is now saturated enough to freeze for this cut. The remaining uncertainty is predominantly local hardware behavior, not missing architecture categories. Xcode 27 / Device Hub is still beta, so upstream release changes remain a formal trigger to reopen targeted research.

1. Device Hub scope: human control versus scripted management

- Apple documents Device Hub as a standalone app that ships with Xcode 27 and can be opened without launching Xcode.
- Paired physical devices can present View Screen; Device Hub provides live interaction from the Mac and Apple says any number of devices can be viewed in tabs or compact windows.
- The phrase "any number" is a UI organization capability, not a performance guarantee for 20 live physical iPhones.
- Apple documents devicectl for device/app/settings/diagnostic automation and structured JSON. This audit found no first-party arbitrary coordinate tap/swipe API in devicectl.
- Production decision: Device Hub first for human operation; CoreDevice HID/WDA/tested hardware only when scripted UI or an unsupported lane is actually required.



ARCH-DEVICE-HUB-SCOPE-003 - First-party human control is separated from documented scripted device management.

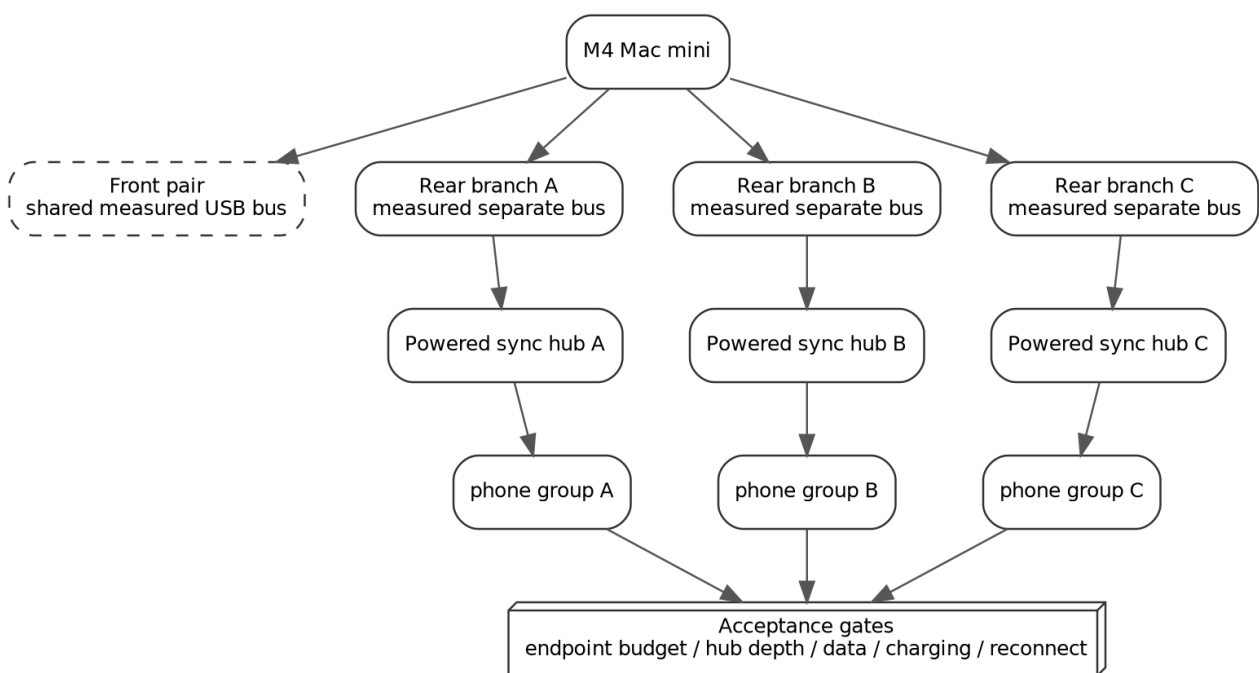
2. Apple pairing state and beta caveats

- Device Hub cable pairing can require Trust and Developer Mode; iOS/iPadOS 27 adds nearby wireless pairing.
- Xcode 27 beta 6 still has Device Hub input/keyboard limitations, including no arbitrary two-finger touch and temporary hardware-keyboard-mode edge cases after disconnect.

- Device Hub must therefore be measured on the exact third-party App Store navigation workflow. Documentation is not substituted for the farm bench.

3. USB endpoint budget and host topology

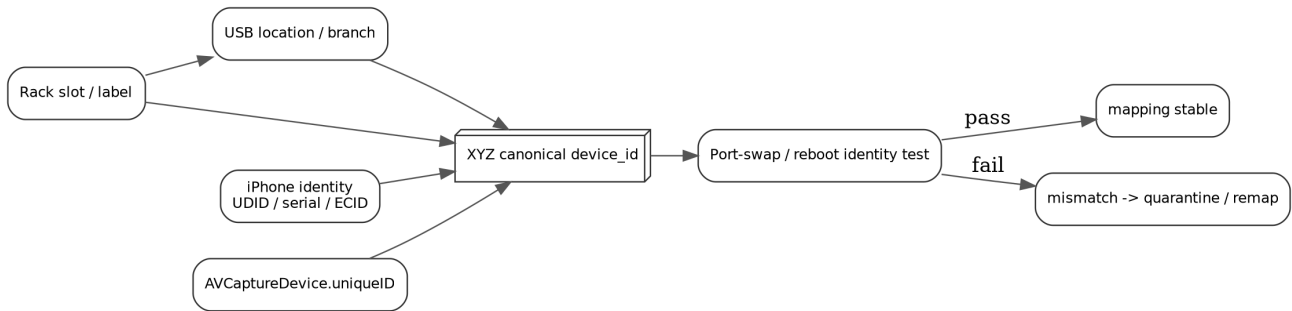
- Cambrionix documents endpoint memory and hub depth as scaling constraints. Its current table lists iPhone 13/15/16 at up to 11 endpoints and gives a baseline of 15 iPhone 15 devices over USB 2 through one standard host controller.
- Twocanoes measured the M4 Mac mini front pair on a shared USB 3.1 bus and the three rear ports on separate USB 3.1 buses. ChargerLAB teardown evidence is consistent with multiple controller paths and a front-port hub.
- These are external engineering observations, not Apple guarantees. The purchased Mac must be mapped with local System Information/IORegistry evidence.
- 20-device design implication: split load across known host branches and test enumeration, endpoint resources, charging and reconnect on every scale gate.



ARCH-USB-ENDPOINT-BUDGET-002 - Endpoint resources and hub depth are acceptance gates, not afterthoughts.

4. Capture identity correction: use a composite identity ledger

- Apple documents AVCaptureDevice.uniqueID as persistent across disconnects, app restarts and Mac reboots.
- A 2026 Apple Developer Forums reproduction reports some UVC capture devices whose uniqueID followed the USB port. It is not an Apple resolution and is not specific to iPhone screen capture.
- This is enough to reject uniqueID as the sole inventory key. Canonical device identity should cross-reference UDID/serial/ECID, capture uniqueID, USB location and physical rack slot.
- Add a deliberate port-swap/reboot test before trusting capture-to-inventory mapping.



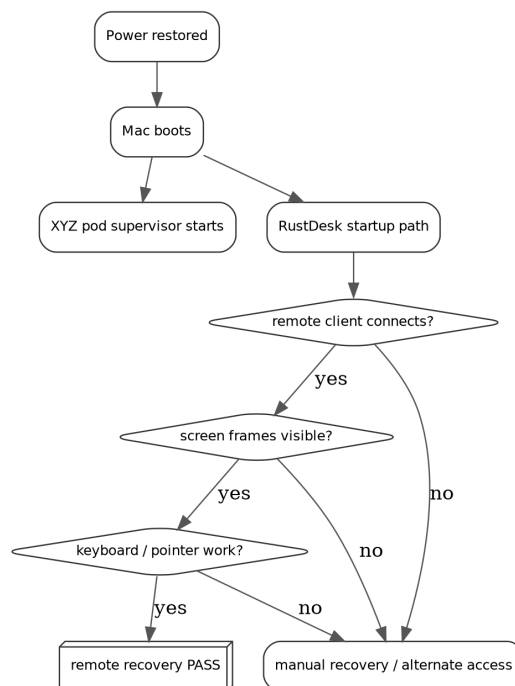
ARCH-IDENTITY-CROSSCHECK-003 - Business identity is XYZ-owned and cross-checks several hardware/OS identities.

5. Native capture health: fresh frames, not session state

- Quern reports a multi-AVCaptureSession race that it mitigated by staggering starts by about one second.
- WireView documents trust/reconnect and no-frame conditions where device discovery does not imply a healthy stream.
- Therefore every capture worker needs last-frame age, frame count/bytes and bounded reconnect/restart logic. Startup should be queued rather than blasting all sessions simultaneously.

6. RustDesk unattended recovery remains a bench gate

- A June 2026 RustDesk project discussion reports macOS 26.5 TCC behavior where LaunchAgent/LaunchDaemon startup connected but failed screen capture. This is community/project evidence, not an Apple platform guarantee.
- Final acceptance test: cold power cycle -> unattended Mac boot -> RustDesk connection -> visible frames -> keyboard/pointer -> Device Hub/pod access. Do not infer success from the fact that RustDesk works after a manual login.



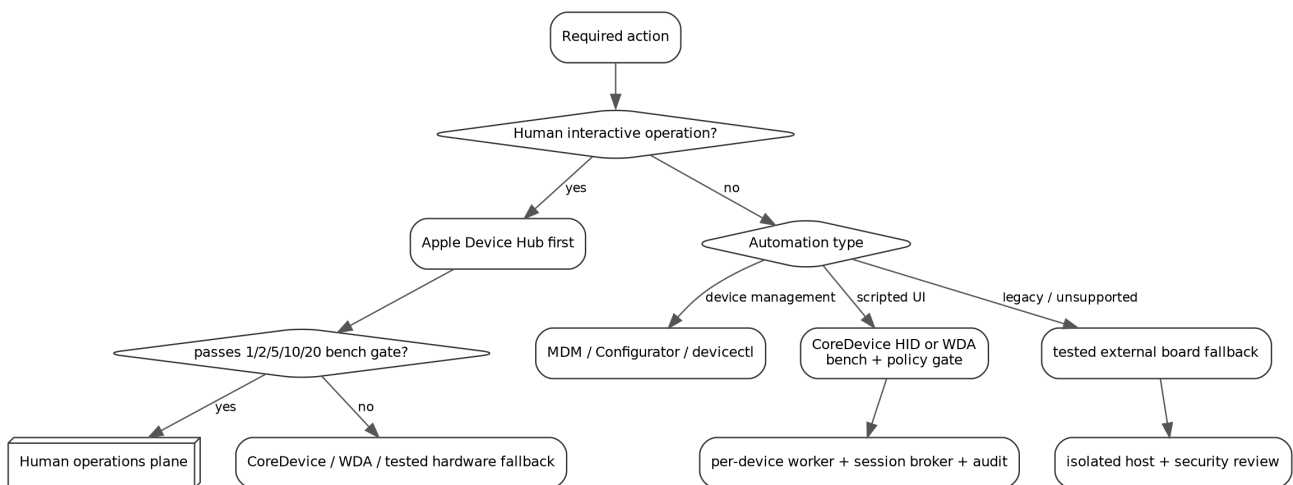
TEST-REMOTE-BOOT-TCC-002 - Unattended remote recovery must be proven end-to-end.

7. iMouse remains security-quarantined fallback

- Some3C documents Windows 10+, a persistent iMouse kernel service, AirPlay/same-network mirroring, up to 10 phones per mirroring service, and a phone-side Shortcut for richer album/file/clipboard/connectivity functions.
- The vendor also recommends Windows Security exclusions / antivirus trust. That does not prove malicious software, but it raises the security-review threshold.
- Keep the iMouse host isolated and test-only until binary/network behavior is reviewed. Its coexistence with Mac-native USB capture remains an OPEN hardware bench question.

8. Final control-lane architecture

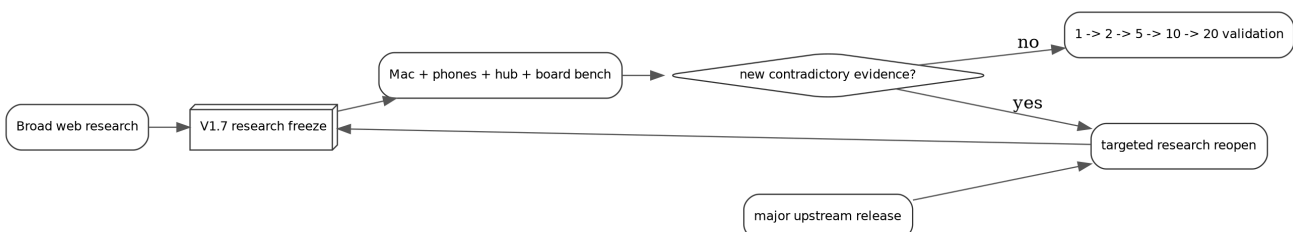
- Human remote operation: Apple Device Hub first, if the exact workflow and scale pass bench tests.
- Device management: Apple Configurator / MDM / devicectl.
- Scripted UI automation: CoreDevice HID or WDA only when justified and policy-accepted, behind per-device workers/session brokers.
- Legacy/unsupported lane: tested external board after security and coexistence review.



ARCH-FINAL-CONTROL-LANES-001 - Final architecture separates human operation, management automation, scripted UI, and fallback hardware.

9. Research freeze and reopening rules

- Broad web research is frozen for V1.7 because architecture categories are saturated and further generic searching has diminishing returns.
- Reopen only on a major Xcode/Device Hub/iOS/macOS/pymobiledevice3/Appium/iMouse/RustDesk change, a selected-hardware change, Dim first-party evidence, or a bench failure that contradicts the current model.
- No web source can move the local bench-validation score without local XYZ/Dim measurements.



ARCH-RESEARCH-FREEZE-001 - After source freeze, the Mac bench becomes the primary evidence generator.

V1.7 evidence score and remaining bench gates

Measurement	V1.6	V1.7 research freeze
Research coverage	99.90%	[Inference] 99.95%
Conceptual understanding	99.85%	[Inference] 99.90%
Actual XYZ/Dim bench validation	74.4%	74.4%

The 74.4% bench figure is intentionally unchanged: this run added web evidence, not XYZ/Dim hardware measurements.

- Device Hub 1/2/5/10/20 physical-iPhone scale, latency and third-party App Store navigation.
- Native CoreMediaIO capture stagger/reconnect/fresh-frame behavior.
- Actual M4 Mac mini USB branch map and selected powered-hub endpoint/charging behavior.
- 5-device multi-hour soak, then 10- and 20-device scale gates.
- Power-loss -> remote RustDesk -> Device Hub/pod recovery.
- iMouse board + Mac USB visibility/capture coexistence on the actual control board.
- Dim evidence capture: exact product/version, phones, chassis, hub/cabling, failures and reproducible logs.

V1.7 source ledger

Source	Class	Used for
Apple Device Hub docs	First-party	physical inventory / control surface
Apple Device Hub management docs	First-party	pairing / Developer Mode / View Screen
WWDC26 Device Hub session	First-party	standalone app / live interaction / any-number windows / devicectl split
Xcode 27 beta 6 notes	First-party	beta limitations / input behavior
pymobiledevice3 iOS17 tunnels	Maintained project	17.4+ userspace RSD / display & HID paths
Appium RemoteXPC guide	Maintained project	multi-device tunnel isolation
Cambionix SS15 + endpoint notes	Vendor engineering	endpoint budget / hub depth / 15-iPhone baseline
Twocanoes M4 port guide	Third-party measured	front shared / rear separate USB buses
ChargerLAB M4 teardown	Third-party teardown	controller / front hub evidence
Apple AVCaptureDevice.uniqueID	First-party	documented persistence semantics
Apple Developer Forums UVC report	Community reproduction	identity-adversarial caveat
Quern discussion	Maintained project	staggered CoreMediaIO multi-device start
WireView bug ledger	Third-party engineering	trust / no-frame capture failures
RustDesk macOS 26 discussion	Community reproduction	unattended TCC hypothesis
Some3C iMouse docs	Vendor claim	Windows kernel / mirroring / shortcut / security posture
Apple device-management restrictions	First-party	host pairing / USB restricted mode
Apple Configurator connection docs	First-party	simultaneous powered-hub operation
Apple PhotoKit PHAssetChangeRequest	First-party	file-based Photos asset creation

Research-freeze rule: external observations stay labeled. Only current first-party OS/platform documentation and reproducible local XYZ/Dim measurements can close architecture-blocking unknowns without qualification.